

第6章: Tailwind CSS & shadcn/ui

6.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どのファイルに何を入力し、何を確認すればよい**か 分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象ファイル（例）	完了時に確認すること
Tailwind 基本・クラス	<code>*.tsx</code> , <code>globals.css</code>	ユーティリティクラスで見た目が変わる
レスポンシブ・ダークモード	<code>tailwind.config.*</code> , <code>*.tsx</code>	画面幅・テーマで表示が切り替わる
shadcn/ui 導入・コンポーネント	<code>components/ui/*</code>	Button, Card, Dialog 等が使える
BON-LOG 実例・3カラムレイアウト	<code>components/layout/*</code>	実際の画面にスタイルが適用される

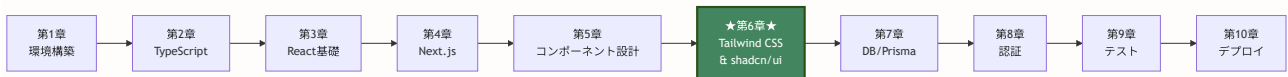
各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するとこうなる・このあとと変わること・確認方法** を確認しながら進めてください。

この章で学ぶこと

- Tailwind CSSの「ユーティリティファースト」という考え方と従来のCSSとの違い
- よく使うTailwind CSSクラス（余白、色、タイポグラフィ、Flexbox、Grid等）
- モバイルファーストのレスポンシブデザインの実装方法
- ダークモード対応の仕組みと実装手順
- BON-LOGの和風カスタムテーマの設定方法
- shadcn/uiの特徴と従来のUIライブラリとの違い

- shadcn/uiの主要コンポーネント（Button、Card、Dialog等）の使い方
- BON-LOGの実際の画面（投稿カード、ログイン画面等）でのスタイリング実装
- CVA（class-variance-authority）の応用パターンとAlert/Inputの高度なセレクト
- tailwind-merge + clsx のcn()関数による安全なクラス名管理
- Tailwind CSS v4のCSS-firstアーキテクチャとglobals.cssの完全な読み方
- oklch()カラースペースの実践的な操作とダークモード色変換
- 3カラムレイアウトの各コンポーネント（Sidebar、RightSidebar、MobileNav、Header）の詳細
- アニメーション・トランジションパターンとアクセシビリティ対応

この章の位置づけ



この章の位置づけ: この章では、画面の「見た目」を効率的に作る方法を学びます。第5章で学んだコンポーネントに美しいスタイルを適用し、次の第7章でデータベースと連携する準備を整えます。

目次

- [6.1 Tailwind CSSとは](#)
- [6.2 よく使うクラス](#)
- [6.3 レスポンシブデザイン](#)
- [6.4 ダークモード対応](#)
- [6.5 カスタムテーマ](#)
- [6.6 shadcn/uiとは](#)
- [6.7 shadcn/uiのインストールと使い方](#)
- [6.8 主要コンポーネント](#)
- [6.9 BON-LOGでの実例](#)
- [6.10 演習問題](#)
- [6.11 CVA（class-variance-authority）詳細](#)
- [6.12 tailwind-merge + clsx（cn\(\)関数）](#)
- [6.13 Tailwind CSS v4固有機能](#)
- [6.14 和風デザインパターン](#)
- [6.15 3カラムレイアウト実装](#)
- [6.16 CVA応用: 実際のコンポーネントを深く読む](#)

- [6.17 Tailwind CSS v4 詳細: globals.cssを完全に読む](#)
- [6.18 和風タイポグラフィとカラーの深掘り](#)
- [6.19 3カラムレイアウト詳細: 各コンポーネントの実装を深く読む](#)
- [6.20 アニメーションとトランジションパターン](#)
- [6.21 asChildパターンとSlotコンポーネント](#)
- [6.22 よくある質問 \(FAQ\)](#)
- [6.23 デザインシステム チートシート](#)

6.1 Tailwind CSSとは

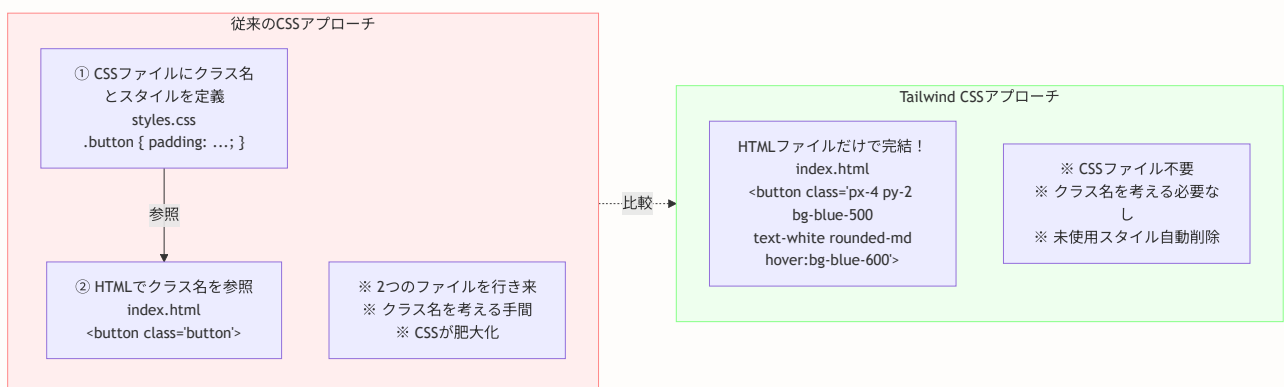
このセクションで学ぶこと

- 従来のCSSアプローチとユーティリティファーストアプローチの違い
- Tailwind CSSが開発効率を高める理由
- クラスの基本的な命名規則（プロパティ-値のパターン）

ユーティリティファーストの考え方

Tailwind CSSは「ユーティリティファースト」のCSSフレームワークです。事前定義された小さなユーティリティクラスを組み合わせることでスタイリングします。

従来のCSS vs Tailwind CSS の比較図



従来のCSS

```
/* styles.css */
.button {
  padding: 0.5rem 1rem;
  background-color: #3b82f6;
  color: white;
  border-radius: 0.375rem;
  font-weight: 600;
}

.button:hover {
  background-color: #2563eb;
}
```

```
<button class="button">ボタン</button>
```

Tailwind CSS

```
<button class="px-4 py-2 bg-blue-500 text-white rounded-md font-semibold hover:bg-blue-600">
  ボタン
</button>
```

画面表示 上記の両方のコードは同じボタンを表示します:

- 左右に16pxの余白 (`px-4`)、上下に8pxの余白 (`py-2`) が入った青い背景色のボタン
- テキストは白色 (`text-white`) で、やや太めのフォント (`font-semibold`)
- 角がわずかに丸い (`rounded-md` = 6px)
- マウスを重ねると、青色が少し濃くなる (`hover:bg-blue-600`)
- 違いは「CSSファイル + HTMLファイルの2つ」か「HTMLだけで完結」かの記述方法のみ

Tailwind CSSの利点

1. **高速な開発**: CSSファイルとHTMLを行き来する必要がない
2. **一貫性**: デザインシステムに基づいた値 (spacing、color等)
3. **パフォーマンス**: 未使用のスタイルは自動削除 (PurgeCSS)
4. **保守性**: コンポーネント単位でスタイルが完結

未使用クラスの自動削除の仕組み Tailwind CSSは全クラス（数千個）を定義していますが、ビルド時にプロジェクト内のファイル（`.tsx`、`.ts`、`.html`等）をスキャンし、**実際に使われているクラス名だけ**を最終CSSに含めます。そのため、本番CSSは通常10KB以下と非常に小さくなります。

⚠ **注意:** クラス名を動的に組み立てると検出されません：

```
// ❌ 検出されない（ビルド時に文字列が確定しない）
const color = 'blue'
className={`bg-${color}-500`}

// ✅ 検出される（完全なクラス名が文字列に含まれる）
className={color === 'blue' ? 'bg-blue-500' : 'bg-red-500'}
```

Tailwind CSSの基本構造

```
<!-- 構造: プロパティ値 -->
<div class="
  text-lg      <!-- font-size: 1.125rem -->
  text-gray-700 <!-- color: #374151 -->
  bg-white     <!-- background-color: #ffffff -->
  p-4         <!-- padding: 1rem -->
  rounded-lg   <!-- border-radius: 0.5rem -->
  shadow-md    <!-- box-shadow: ... -->
">
  コンテンツ
</div>
```

画面表示 このコードを適用すると：

- 白い背景（`bg-white`）の上に、やや大きめのフォント（`text-lg` = 18px）でダークグレーのテキスト（`text-gray-700`）が表示される
- 全方向に16pxの内側余白（`p-4`）があり、テキストが枠線ギリギリにならない
- 角が丸い（`rounded-lg` = 8px）カード形状になる
- 中程度の影（`shadow-md`）が付き、カードが浮いているような奥行き感が生まれる

よくあるトラブルと解決法

トラブル	原因	解決法
クラスを書いたのにスタイルが適用されない	Tailwind CSS v4ではCSSファイルでの <code>@import "tailwindcss"</code> が必要	<code>app/globals.css</code> に <code>@import "tailwindcss";</code> があるか確認。v4ではcontentの自動検出が行われるため、v3のような <code>content</code> 配列は不要
動的なクラス名が効かない	Tailwindはビルド時にクラスを検出するため、動的生成は不可	<code>bg-red-500</code> のように完全なクラス名を書く (<code>bg-\${color}-500</code> はNG)
カスタムの値を使いたい	デフォルトのスケールにない値を使いたい	<code>w-[320px]</code> のように角括弧で任意の値を指定 (JIT)

BON-LOGでの使用箇所: `app/globals.css`

実装しない場合の影響: Tailwind CSSが正しく読み込まれず、すべてのユーティリティクラスが適用されません。ページは装飾のない素のHTMLとして表示されてしまいます。

理解度チェック

▶ Q1: Tailwind CSSの「ユーティリティファースト」とは何ですか？

▶ Q2: 従来のCSSと比較して、Tailwind CSSの最大のメリットは何ですか？

▶ Q3: `bg-${color}-500` のような動的クラス名が動かないのはなぜですか？

補足A: 初心者のための専門用語ガイド

この章では多くのCSS・フロントエンド関連の専門用語が登場します。初めて出会う用語に戸惑わないよう、ここでまとめて解説します。

スタイリング基本用語

用語	読み方	意味
ユーティリティファースト	-	1つのCSSプロパティに対して1つのクラスを割り当て、それらを組み合わせでUIを構築するアプローチ。「まずユーティリティ（道具）を使う」という意味
レスポンシブデザイン	-	スマートフォン、タブレット、デスクトップなど、さまざまな画面サイズに合わせてレイアウトを自動調整するデザイン手法
ブレイクポイント	breakpoint	画面幅の区切りとなるピクセル値のこと。例: 768px以上ならタブレット向けレイアウトに切り替える、のように使う
モバイルファースト	mobile first	まずスマートフォン向けのスタイルを基本として定義し、そこから画面幅が大きくなるにつれて追加のスタイルを上書きする設計手法
CSS変数（カスタムプロパティ）	CSS variables	<code>--primary: #42855e;</code> のように <code>--</code> で始まる名前で定義し、 <code>var(--primary)</code> で参照できるCSSの値の仕組み。JavaScriptの変数と同じように「値に名前を付けて再利用する」ためのもの
バリエーション	variant	コンポーネントの見た目のバリエーション（種類）のこと。例: ボタンの「デフォルト」「アウトライン」「ゴースト」など
コンポーネントライブラリ	component library	Button、Card、Dialogなどの再利用可能なUIパーツ（コンポーネント）をまとめたもの。自分で一から作る手間を省ける

レイアウト関連用語

用語	読み方	意味
Flexbox	フレックスボックス	CSSのレイアウト方式の1つ。要素を横並びまたは縦並びに配置し、間隔や位置を柔軟に調整できる。「flex = 柔軟な」の意味
Grid	グリッド	CSSのレイアウト方式の1つ。要素を行と列の格子状（グリッド）に配置する。Excelのセルのようなイメージ
sticky	スティッキー	<code>position: sticky</code> のこと。通常は他の要素と一緒にスクロールするが、画面の端に達すると「くっついて」固定される配置方法
fixed	フィクスト	<code>position: fixed</code> のこと。常に画面の特定位置に固定される配置方法。スクロールしても動かない

デザインシステム用語

用語	読み方	意味
デザイントークン	design token	色、余白、角丸などの値に意味のある名前を付けて管理する仕組み。例: <code>--primary</code> (メインカラー)、 <code>--radius-lg</code> (大きい角丸) など
アクセシビリティ	accessibility (a11y)	障がいのある方やさまざまな環境のユーザーが問題なく利用できるようにすること。キーボード操作、スクリーンリーダー対応などが含まれる
ヘッドレスUI	headless UI	機能（動作やアクセシビリティ）だけを提供し、見た目（スタイル）は自分で自由に付けられるUIライブラリのこと。「頭（見た目）がない」という意味
PurgeCSS	パージCSS	実際にコード内で使われていないCSSクラスを自動的に検出・除去する仕組み。Tailwind CSSに組み込まれており、ファイルサイズの削減に貢献する
バンドルサイズ	bundle size	Webサイトをブラウザに読み込む際に必要なファイルの合計サイズ。小さいほどページの読み込みが速い
DX (Developer Experience)	ディーエックス	開発者体験のこと。開発ツールの使いやすさ、ドキュメントの充実度、エラーメッセージの分かりやすさなどを含む

CSS-in-JS関連用語

用語	読み方	意味
CSS-in-JS	シーエスエス・イン・ジェーエス	JavaScriptファイルの中でCSSスタイルを記述するアプローチの総称。Styled Components、Emotionなどが代表例
型安全	かたあんぜん	TypeScriptの型チェック機能により、間違った値を指定するとコンパイル時にエラーが表示されること。実行前にミスを防げる
ゼロランタイム	zero runtime	スタイルの処理がビルド時に完了し、ブラウザ実行時にはJavaScriptが不要であること。ページの表示速度が速くなる

補足B: ユーティリティファーストCSSを従来のCSSと比較して理解する

Tailwind CSSの「ユーティリティファースト」という考え方が初めての方に向けて、従来のCSS記法との違いを段階的に解説します。

従来のCSS: クラス名を考えて、別ファイルにスタイルを書く

従来のCSS開発では、以下のような流れでスタイリングを行います。

- 【Step 1】 HTMLに意味のあるクラス名を付ける
- 【Step 2】 CSSファイルでそのクラス名にスタイルを定義する
- 【Step 3】 2つのファイルを行き来しながら調整する

例: ユーザープロフィールカード

```

/* styles/profile.css */
.profile-card {
  background-color: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 0.5rem;
  padding: 1.5rem;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}

.profile-card__name {
  font-size: 1.25rem;
  font-weight: 700;
  color: #1a1a1a;
  margin-bottom: 0.5rem;
}

.profile-card__bio {
  font-size: 0.875rem;
  color: #6b7280;
  line-height: 1.625;
}

.profile-card__button {
  background-color: #6b8e23;
  color: #ffffff;
  padding: 0.5rem 1rem;
  border-radius: 0.375rem;
  font-weight: 600;
  margin-top: 1rem;
}

.profile-card__button:hover {
  background-color: #2d5016;
}

```

```

<!-- pages/profile.html -->
<div class="profile-card">
  <h2 class="profile-card__name">山田太郎</h2>
  <p class="profile-card__bio">五葉松を育てて20年。盆栽の魅力を発信しています。</p>
  <button class="profile-card__button">フォローする</button>
</div>

```

この方法の課題:

- クラス名を考える手間がある（`profile-card__name`のような命名ルールを覚える必要がある）
- HTMLとCSSの2つのファイルを常に行き来する

- CSSファイルが肥大化しやすい（使われなくなったクラスが残りがち）
- 「この色はどのクラスで定義してたっけ？」と探す手間がかかる

Tailwind CSS: HTMLの中で直接スタイルを書く

Tailwind CSSでは、同じUIを以下のように記述します。

```
<!-- 従来のCSSで5つのクラス定義 + CSSファイルが必要だったものが、HTMLだけで完結 -->
<div class="bg-white border border-gray-200 rounded-lg p-6 shadow-sm">
  <h2 class="text-xl font-bold text-sumi mb-2">山田太郎</h2>
  <p class="text-sm text-gray-500 leading-relaxed">
    五葉松を育てて20年。盆栽の魅力を発信しています。
  </p>
  <button class="bg-bonsai-moss text-white px-4 py-2 rounded-md font-semibold mt-4 hover:b
    フォローする
  </button>
</div>
```

画面表示 このBON-LOGのプロフィールカードは以下のように表示されます：

- 白い背景（`bg-white`）に薄いグレーの枠線（`border-gray-200`）で囲まれたカード
- 角が丸く（`rounded-lg`）、内側に24pxの余白（`p-6`）、薄い影（`shadow-sm`）で浮遊感
- **名前:** 「山田太郎」が大きめ（`text-xl` = 20px）の太字（`font-bold`）で墨色（`text-sumi`）表示
- **自己紹介:** 小さめ（`text-sm` = 14px）のグレー文字（`text-gray-500`）で、行間がゆったり（`leading-relaxed`）
- **フォローボタン:** 苔色（`bg-bonsai-moss` = 黄緑がかった緑）の背景に白文字、角がやや丸い
- ボタンにマウスを重ねると松葉色（`bg-bonsai-pine` = 濃い深緑）に変化

対応関係:

従来のCSS	Tailwind CSS
<code>background-color: #ffffff;</code>	<code>bg-white</code>
<code>border: 1px solid #e5e7eb;</code>	<code>border border-gray-200</code>
<code>border-radius: 0.5rem;</code>	<code>rounded-lg</code>
<code>padding: 1.5rem;</code>	<code>p-6</code>
<code>box-shadow: ... ;</code>	<code>shadow-sm</code>
<code>font-size: 1.25rem;</code>	<code>text-xl</code>
<code>font-weight: 700;</code>	<code>font-bold</code>
<code>color: #1a1a1a;</code>	<code>text-sumi</code>
<code>margin-bottom: 0.5rem;</code>	<code>mb-2</code>
<code>:hover background-color</code>	<code>hover:bg-bonsai-pine</code>

「クラスが長すぎる」問題はReactコンポーネントが解決する

Tailwind CSSの最大の批判は「HTMLのクラスが長すぎて読みにくい」というものです。しかし、Reactのコンポーネント化と組み合わせることで、この問題は解消されます。

```
// components/ui/ProfileCard.tsx
// 1回定義すれば、使う側はシンプルなコンポーネントとして呼び出すだけ
export function ProfileCard({ name, bio, onFollow }: ProfileCardProps) {
  return (
    <div className="bg-white border border-gray-200 rounded-lg p-6 shadow-sm">
      <h2 className="text-xl font-bold text-sumi mb-2">{name}</h2>
      <p className="text-sm text-gray-500 leading-relaxed">{bio}</p>
      <button
        onClick={onFollow}
        className="bg-bonsai-moss text-white px-4 py-2 rounded-md font-semibold mt-4 hover:"
      >
        フォローする
      </button>
    </div>
  )
}

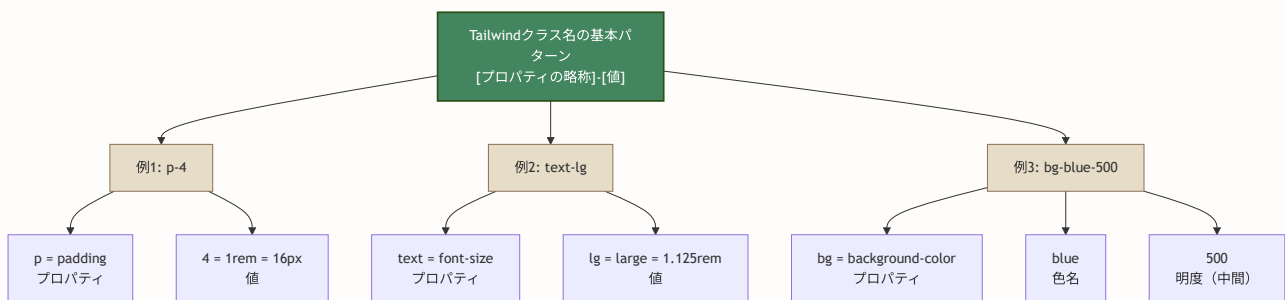
// 使う側はこれだけ
<ProfileCard name="山田太郎" bio="五葉松を育てて20年" onFollow={handleFollow} />
```

スタイルの詳細はコンポーネント内部に隠蔽され、使う側はpropsを渡すだけで済みます。これは従来のCSSでクラス名を別ファイルに定義するのと同質的に同じ役割です。

補足C: Tailwindクラス名の読み方ガイド

Tailwind CSSのクラス名は一見すると暗号のように見えますが、実は「プロパティの略称-値」という規則的なパターンで構成されています。このガイドでは、クラス名を見ただけで何のスタイルか分かるようになることを目指します。

基本パターン: プロパティ-値



方向の略語

Tailwindでは方向を1文字で表します。

```

t (top)
↑
l (left) ← → r (right)   x = 左右 (horizontal)
↓                       y = 上下 (vertical)
b (bottom)               (なし) = 全方向
  
```

具体例:

```

p-4   → padding: 1rem          (全方向に1rem)
pt-4  → padding-top: 1rem       (上だけ1rem)
pb-4  → padding-bottom: 1rem    (下だけ1rem)
pl-4  → padding-left: 1rem      (左だけ1rem)
pr-4  → padding-right: 1rem     (右だけ1rem)
px-4  → padding-left/right: 1rem (左右に1rem)
py-4  → padding-top/bottom: 1rem (上下に1rem)

m-4   → margin: 1rem           (marginも同様のパターン)
mt-4  → margin-top: 1rem
mx-auto → margin-left/right: auto (中央寄せ)

```

数値スケールの読み方

Tailwindの数値は **0.25rem(4px)** を1単位としています。

数値	rem値	px値	覚え方
0	0	0px	
1	0.25rem	4px	「1 = 4px」と覚える
2	0.5rem	8px	1の2倍
3	0.75rem	12px	1の3倍
4	1rem	16px	★ よく使う基準値
5	1.25rem	20px	
6	1.5rem	24px	4の1.5倍
8	2rem	32px	4の2倍
10	2.5rem	40px	
12	3rem	48px	4の3倍
16	4rem	64px	4の4倍
20	5rem	80px	
24	6rem	96px	

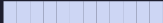
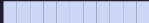
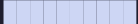
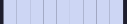
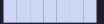
よく使う目安:

- **p-2** (8px): テキスト周りの最小パディング
- **p-4** (16px): カード内のパディング、要素間の標準間隔
- **p-6** (24px): セクション内のゆったりしたパディング
- **p-8** (32px): ページセクション間の大きな余白

色の読み方

色の指定: [プロパティ]-[色名]-[明度]

明度スケール:

50		極めて薄い (背景用)
100		とても薄い
200		薄い
300		やや薄い
400		やや薄め
500		標準 (中間)
600		やや濃い
700		濃い
800		とても濃い
900		極めて濃い
950		最も濃い

例:

text-gray-500	→ 中間のグレーの文字色 (サブテキストに最適)
text-gray-900	→ 濃いグレーの文字色 (メインテキストに最適)
bg-blue-100	→ 薄い青の背景色 (情報メッセージの背景に最適)
bg-red-500	→ 標準の赤の背景色 (エラー表示に最適)
border-gray-200	→ 薄いグレーのボーダー (カードの枠線に最適)

修飾プレフィックスの読み方

Tailwindでは **状態:クラス名** の形式で、特定の状態でのみスタイルを適用できます。

<code>hover:bg-blue-600</code>	→ マウスホバー時に背景色を青600に
<code>focus:ring-2</code>	→ フォーカス時にリング（枠線）を2px表示
<code>active:scale-95</code>	→ クリック中にサイズを95%に縮小
<code>disabled:opacity-50</code>	→ 無効状態で透明度を50%に
<code>dark:bg-gray-900</code>	→ ダークモード時に背景色をグレー900に
<code>dark:text-white</code>	→ ダークモード時に文字色を白に
<code>sm:text-lg</code>	→ 640px以上でフォントサイズをlgに
<code>md:grid-cols-2</code>	→ 768px以上で2カラムグリッドに
<code>lg:flex</code>	→ 1024px以上でFlexbox表示に
<code>xl:block</code>	→ 1280px以上でブロック表示に

実践: クラスの逆引きリファレンス

以下に、BON-LOGでよく使われるスタイルの「やりたいこと→Tailwindクラス」対応表を示します。

やりたいこと	Tailwindクラス
画面全体の高さを確保したい	<code>min-h-screen</code>
要素を横並びにしたい	<code>flex</code>
要素を縦並びにしたい	<code>flex flex-col</code>
要素を上下左右中央に配置したい	<code>flex items-center justify-center</code>
要素間に均等な余白を入れたい	<code>flex gap-4</code>
残りのスペースを埋めたい	<code>flex-1</code>
テキストを1行で省略表示したい	<code>truncate</code>
テキストを複数行で省略したい	<code>line-clamp-2</code>
画像をコンテナに合わせてトリミングしたい	<code>object-cover</code>
角を丸くしたい	<code>rounded-lg</code>
完全な円にしたい	<code>rounded-full</code>
要素を非表示にしたい	<code>hidden</code>
大画面でのみ表示したい	<code>hidden lg:block</code>
マウスカーソルを指に変えたい	<code>cursor-pointer</code>
アニメーションで変化させたい	<code>transition-all duration-200</code>

補足D: レスポンシブデザインの考え方（モバイルファースト）

なぜモバイルファーストなのか

現在、Webサイトのアクセスの50%以上がスマートフォンからです。そのため、まずスマートフォンでの表示を基本として設計し、画面が大きくなるにつれて追加のレイアウトを適用するのが合理的です。

【デスクトップファースト（従来の考え方）】

- まずデスクトップ向けに作る → スマホ対応を後から追加
- スマホで「崩れる」「はみ出す」問題が起きやすい
- 「引き算」の設計（デスクトップから要素を減らす）

【モバイルファースト（Tailwind CSSの考え方）】

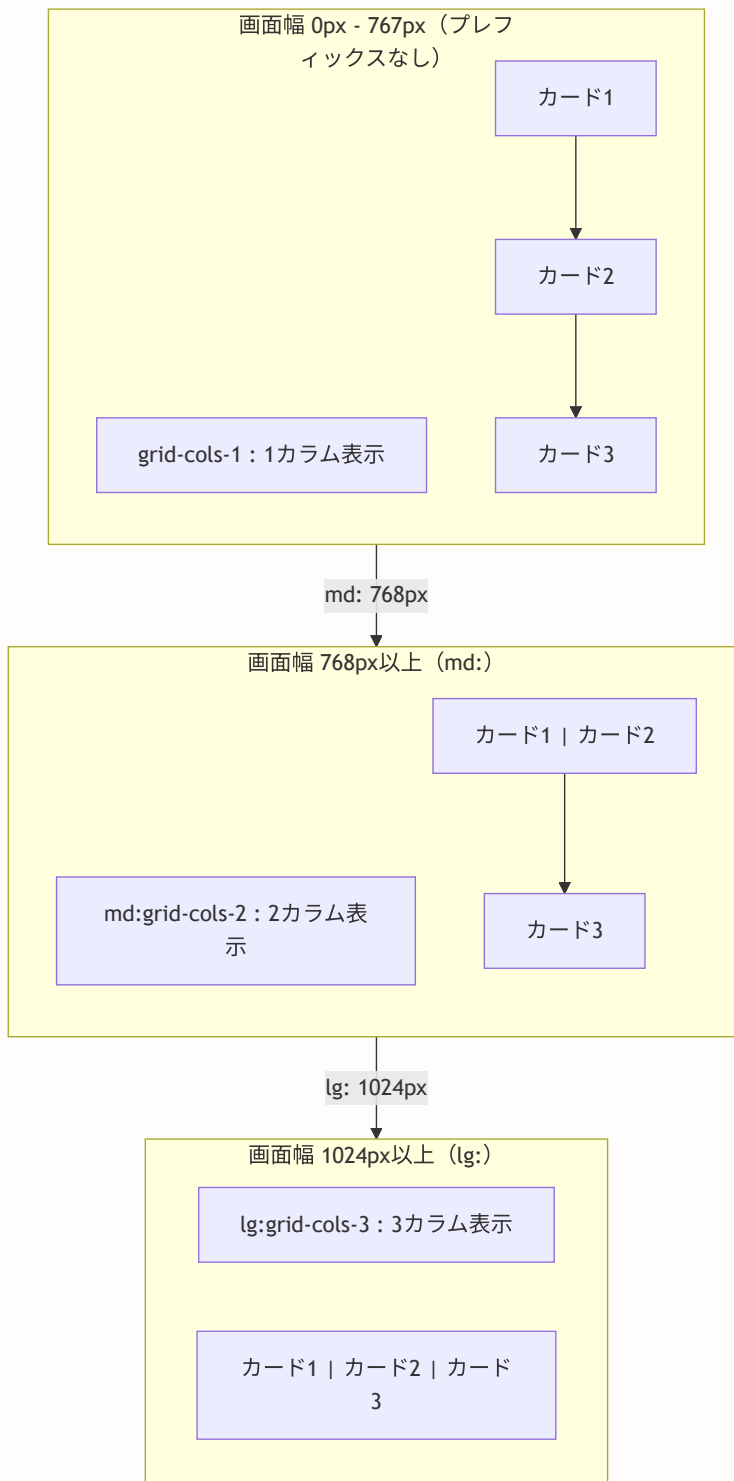
- まずスマホ向けに作る → 大画面向けを後から追加
- 基本がシンプルなのでレイアウト崩れが起きにくい
- 「足し算」の設計（画面が広がったら要素を追加）

Tailwind CSSでのモバイルファースト

Tailwindのレスポンシブの仕組みを、具体例で理解しましょう。

```
<!-- この1行に3つのスタイルが定義されている -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3">
```

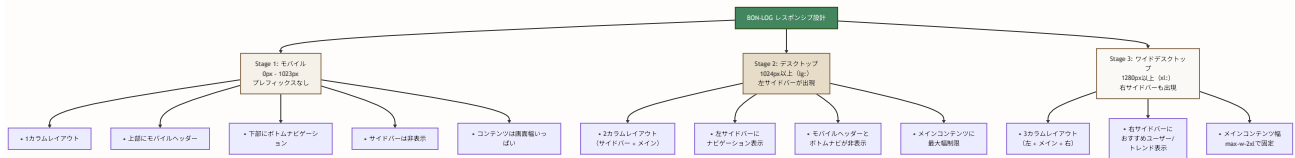
このクラスは以下のように解釈されます:



重要なポイント: プレフィックスなしのクラスは「全画面サイズ」で適用されるのではなく、最小画面（モバイル）からのベースラインとして機能します。 `md:` や `lg:` のプレフィックスは「そのサイズ以上で上書き」を意味します。

BON-LOGでの実例

BON-LOGのレスポンス設計は、以下の3段階で構成されています。



このように「モバイルで基本形を作り、画面幅が広がるたびにサイドバーを追加していく」のがモバイルファーストの設計です。

6.2 よく使うクラス

このセクションで学ぶこと

- Spacing（余白）の仕組みとスケール
- Color（色）の指定方法とカラースケール
- Typography（文字）、Flexbox、Gridレイアウトのクラス
- Border、Shadow、Width/Height、Position、Displayの使い方

Spacing（余白）

Tailwind CSSのスペーシングは `0.25rem`（4px）を1単位とします。

```

<!-- Padding -->
<div class="p-4">           <!-- padding: 1rem (16px) -->
<div class="px-4">          <!-- padding-left/right: 1rem -->
<div class="py-2">          <!-- padding-top/bottom: 0.5rem -->
<div class="pt-8">          <!-- padding-top: 2rem -->
<div class="pl-2">          <!-- padding-left: 0.5rem -->

<!-- Margin -->
<div class="m-4">           <!-- margin: 1rem -->
<div class="mx-auto">       <!-- margin-left/right: auto (中央寄せ) -->
<div class="my-8">          <!-- margin-top/bottom: 2rem -->
<div class="-mt-4">         <!-- margin-top: -1rem (負のマージン) -->

<!-- Space Between (子要素間の余白) -->
<div class="space-y-4">    <!-- 子要素の縦方向に1remの余白 -->
  <div>項目1</div>         <!-- → 項目1と項目2の間に16pxの余白 -->
  <div>項目2</div>         <!-- → 項目2と項目3の間に16pxの余白 -->
  <div>項目3</div>
</div>

```

画面表示 `space-y-4` を使うと、各項目の間に均等な16pxの縦方向余白が自動的に入る。フォーム要素の縦並びやカードリストでよく使うパターン。

スペーシングスケール

```

0   → 0
1   → 0.25rem (4px)
2   → 0.5rem (8px)
4   → 1rem (16px)
6   → 1.5rem (24px)
8   → 2rem (32px)
12  → 3rem (48px)
16  → 4rem (64px)

```

なぜ4pxの倍数？ Tailwindのスペーシングは**8ポイントグリッドシステム**に基づいています。4px刻みにすることで、デザイン全体に一貫したリズムが生まれ、要素間の余白が自然に揃います。デザイナーとの協業でも「p-4 (16px)」のように共通言語で会話できます。

Color (色)

```
<!-- Text Color → テキストの色を変える →  
<p class="text-gray-900">濃いグレーのテキスト</p> <!-- → メインテキストに最適 →  
<p class="text-blue-500">青いテキスト</p> <!-- → リンクや情報表示に →  
<p class="text-red-600">赤いテキスト</p> <!-- → エラーメッセージに →  
  
<!-- Background Color → 背景色を変える →  
<div class="bg-white">白背景</div> <!-- → カード、入力欄に →  
<div class="bg-gray-100">薄いグレー背景</div> <!-- → セクション区切りに →  
<div class="bg-blue-500">青背景</div> <!-- → ボタン、バッジに（白文字と組み合わせる） →  
  
<!-- Border Color → 枠線の色を変える →  
<div class="border border-gray-300">グレーのボーダー</div> <!-- → カードの枠線に →
```

カラースケール

```
50   → 最も薄い  
100  →  
200  →  
...  
800  →  
900  → 最も濃い
```

Typography (タイポグラフィ)

```

<!-- Font Size → 文字が段階的に大きくなる →
<p class="text-xs">極小 (0.75rem)</p>      <!-- → 12px: 注釈や日時表示に →
<p class="text-sm">小 (0.875rem)</p>        <!-- → 14px: サブテキスト、説明文に →
<p class="text-base">標準 (1rem)</p>        <!-- → 16px: 本文のデフォルト →
<p class="text-lg">大 (1.125rem)</p>        <!-- → 18px: 強調テキストに →
<p class="text-xl">特大 (1.25rem)</p>        <!-- → 20px: カードタイトルに →
<p class="text-2xl">2XL (1.5rem)</p>        <!-- → 24px: セクション見出しに →
<p class="text-3xl">3XL (1.875rem)</p>      <!-- → 30px: ページタイトルに →

<!-- Font Weight → 文字の太さが段階的に変わる →
<p class="font-normal">通常 (400)</p>        <!-- → 本文テキスト →
<p class="font-medium">中 (500)</p>          <!-- → ラベル、ナビリンク →
<p class="font-semibold">半太字 (600)</p>    <!-- → ボタンテキスト →
<p class="font-bold">太字 (700)</p>          <!-- → 見出し、ユーザー名 →

<!-- Line Height → 行と行の間隔が変わる →
<p class="leading-tight">密な行間</p>        <!-- → 見出し向き (行間が詰まる) →
<p class="leading-normal">標準行間</p>        <!-- → 一般テキスト向き →
<p class="leading-relaxed">ゆったり行間</p>    <!-- → 長文・説明文向き (読みやすい) →

<!-- Text Align →
<p class="text-left">左寄せ</p>
<p class="text-center">中央寄せ</p>
<p class="text-right">右寄せ</p>

<!-- Text Decoration →
<a class="underline">下線</a>
<a class="no-underline">下線なし</a>
<p class="line-through">打ち消し線</p>

<!-- Text Transform →
<p class="uppercase">大文字</p>
<p class="lowercase">小文字</p>
<p class="capitalize">先頭大文字</p>

<!-- Text Overflow →
<p class="truncate">長いテキストは省略される ... </p>
<p class="overflow-ellipsis">同様に省略 ... </p>

```

Flexbox

```

<!-- 基本的なFlexコンテナ -->
<div class="flex">
  <div>項目1</div>
  <div>項目2</div>
</div>

<!-- 縦方向 -->
<div class="flex flex-col">
  <div>項目1</div>
  <div>項目2</div>
</div>

<!-- 主軸の配置 (justify-content) -->
<div class="flex justify-start">      <!-- 左寄せ -->
<div class="flex justify-center">    <!-- 中央 -->
<div class="flex justify-end">        <!-- 右寄せ -->
<div class="flex justify-between">    <!-- 両端寄せ -->
<div class="flex justify-around">    <!-- 均等配置 -->

<!-- 交差軸の配置 (align-items) -->
<div class="flex items-start">        <!-- 上寄せ -->
<div class="flex items-center">      <!-- 中央 -->
<div class="flex items-end">          <!-- 下寄せ -->
<div class="flex items-stretch">     <!-- 伸ばす -->

<!-- 折り返し -->
<div class="flex flex-wrap">          <!-- 折り返す -->

<!-- ギャップ -->
<div class="flex gap-4">              <!-- 子要素間に1remの余白 -->
<div class="flex gap-x-2 gap-y-4">   <!-- 横2、縦4の余白 -->

<!-- 実用例: ヘッダー -->
<header class="flex items-center justify-between p-4 bg-white border-b">
  <div class="flex items-center gap-2">
    
    <h1 class="text-xl font-bold">BON-LOG</h1>
  </div>
  <nav class="flex gap-4">
    <a href="/feed">タイムライン</a>
    <a href="/search">検索</a>
  </nav>
</header>

```

画面表示 このヘッダーは白い背景に下線付き (`border-b`) で横一列に配置されます:

- 左端: ロゴアイコン (高さ32px) と「BON-LOG」テキスト (太字、20px) が横並び (`gap-2` で8px 間隔)
- 右端: 「タイムライン」「検索」のリンクが16px間隔 (`gap-4`) で並ぶ
- `justify-between` により左右に分かれ、`items-center` で縦方向の中央に揃う

Grid

←!— 基本的なグリッド →

```
<div class="grid grid-cols-3 gap-4">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
</div>
```

←!— 列数の指定 →

```
<div class="grid grid-cols-2">      ←!— 2列 →
<div class="grid grid-cols-4">      ←!— 4列 →
<div class="grid grid-cols-12">     ←!— 12列 (Bootstrapスタイル) →
```

←!— 行数の指定 →

```
<div class="grid grid-rows-3">      ←!— 3行 →
```

←!— 列のスパン →

```
<div class="grid grid-cols-3">
  <div class="col-span-2">2列分</div>
  <div>1列</div>
</div>
```

←!— 実用例: カードグリッド →

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  <div class="bg-white rounded-lg shadow p-4">カード1</div>
  <div class="bg-white rounded-lg shadow p-4">カード2</div>
  <div class="bg-white rounded-lg shadow p-4">カード3</div>
</div>
```

画面表示 レスポンシブなカードグリッドの表示:

- **モバイル**: 3枚のカードが縦1列 (`grid-cols-1`) に並ぶ。各カード間に24pxの余白 (`gap-6`)
- **タブレット (768px以上)**: 2列 (`md:grid-cols-2`) に並び、3枚目は次の行に回り込む
- **デスクトップ (1024px以上)**: 3列 (`lg:grid-cols-3`) に横一列で3枚並ぶ
- 各カードは白背景、角丸、影付きで浮いたように見える

Border (ボーダー)

```

<!-- ボーダーの太さ -->
<div class="border">           <!-- 1px -->
<div class="border-2">        <!-- 2px -->
<div class="border-4">        <!-- 4px -->

<!-- 方向指定 -->
<div class="border-t">         <!-- 上のみ -->
<div class="border-b">         <!-- 下のみ -->
<div class="border-l">         <!-- 左のみ -->
<div class="border-r">         <!-- 右のみ -->

<!-- 角丸 -->
<div class="rounded">          <!-- border-radius: 0.25rem -->
<div class="rounded-md">       <!-- border-radius: 0.375rem -->
<div class="rounded-lg">       <!-- border-radius: 0.5rem -->
<div class="rounded-full">     <!-- border-radius: 9999px (完全な円) -->
<div class="rounded-t-lg">     <!-- 上のみ角丸 -->

<!-- 実用例: カード -->
<div class="border border-gray-200 rounded-lg p-4">
  カードコンテンツ
</div>

```

Shadow (影)

```

<div class="shadow-sm">          <!-- 薄い影 -->
<div class="shadow">             <!-- 標準の影 -->
<div class="shadow-md">          <!-- 中程度の影 -->
<div class="shadow-lg">          <!-- 大きい影 -->
<div class="shadow-xl">          <!-- 特大の影 -->

<!-- 実用例: ホバー時に影を強調 -->
<div class="shadow hover:shadow-lg transition-shadow duration-200">
  ホバーしてみてください
</div>

```

画面表示 通常時は控えめな影 (`shadow`) が付いた要素。マウスを重ねると200ミリ秒かけて (`duration-200`) 影がなめらかに大きくなり (`hover:shadow-lg`)、カードが浮き上がるような視覚効果が生まれる。`transition-shadow` により影の変化だけがアニメーションする。

Width & Height

```

<!-- 固定サイズ -->
<div class="w-64">                <!-- width: 16rem (256px) -->
<div class="h-32">                <!-- height: 8rem (128px) -->

<!-- パーセント -->
<div class="w-full">              <!-- width: 100% -->
<div class="w-1/2">               <!-- width: 50% -->
<div class="w-1/3">              <!-- width: 33.333% -->
<div class="w-2/3">              <!-- width: 66.666% -->

<!-- ビューポート -->
<div class="w-screen">           <!-- width: 100vw -->
<div class="h-screen">           <!-- height: 100vh -->

<!-- 最小・最大 -->
<div class="min-w-0">            <!-- min-width: 0 -->
<div class="max-w-md">           <!-- max-width: 28rem -->
<div class="max-w-lg">           <!-- max-width: 32rem -->
<div class="max-w-xl">           <!-- max-width: 42rem -->
<div class="max-w-screen-xl">    <!-- max-width: 1280px -->

```

Position

```

<!-- 配置 -->
<div class="static">           <!-- position: static -->
<div class="relative">        <!-- position: relative -->
<div class="absolute">        <!-- position: absolute -->
<div class="fixed">           <!-- position: fixed -->
<div class="sticky">          <!-- position: sticky -->

<!-- 位置指定 -->
<div class="absolute top-0 left-0">    <!-- 左上 -->
<div class="absolute top-0 right-0">    <!-- 右上 -->
<div class="absolute bottom-0 left-0">  <!-- 左下 -->
<div class="absolute inset-0">         <!-- 全方向 -->

<!-- 実用例: 通知バッジ -->
<div class="relative">
  <button>通知</button>
  <span class="absolute -top-2 -right-2 bg-red-500 text-white rounded-full w-5 h-5 text-xs">
    3
  </span>
</div>

```

画面表示 「通知」ボタンの右上に赤い丸バッジ（直径20px）が少しはみ出して表示される。バッジ内には白文字で「3」という件数が中央揃えで表示される。`relative`の親要素を基準に`absolute`で配置し、`-top-2 -right-2`（負のオフセット）でボタンの外側にはみ出す位置に調整している。

Display

```

<div class="block">           <!-- display: block -->
<div class="inline">          <!-- display: inline -->
<div class="inline-block">    <!-- display: inline-block -->
<div class="flex">            <!-- display: flex -->
<div class="grid">           <!-- display: grid -->
<div class="hidden">         <!-- display: none -->

```

Opacity & Visibility

```
<div class="opacity-0">      <!-- opacity: 0 (完全透明)  -->
<div class="opacity-50">    <!-- opacity: 0.5 (半透明)  -->
<div class="opacity-100">   <!-- opacity: 1 (不透明)  -->

<div class="visible">        <!-- visibility: visible  -->
<div class="invisible">      <!-- visibility: hidden  -->
```

理解度チェック

▶ Q1: `px-4` と `p-4` の違いは何ですか？

▶ Q2: `flex justify-between items-center` を使うとどのようなレイアウトになりますか？

▶ Q3: `space-y-4` と `gap-4` の違いは何ですか？

6.3 レスポンシブデザイン

このセクションで学ぶこと

- Tailwind CSSのブレイクポイントの種類と意味
- 「モバイルファースト」の考え方とプレフィックスの使い方
- BON-LOGの3カラムレイアウトの実装方法

Tailwind CSSでは、プレフィックスを付けることでブレイクポイントごとのスタイルを指定できます。

ブレイクポイント

```
sm: 640px以上
md: 768px以上
lg: 1024px以上
xl: 1280px以上
2xl: 1536px以上
```

レスポンシブブレイクポイントの図解

画面幅(px)	0	640	768	1024	1280	1536
	(なし)	sm:	md:	lg:	xl:	2xl:
スマートフォン						
[=====]						
タブレット						
[=====]						
小型ノートPC						
[=====]						
デスクトップ						
[=====]						
大画面						
[=====]						

※ 各プレフィックスは「そのサイズ以上」で適用される
 ※ プレフィックスなし = すべてのサイズで適用

モバイルファーストアプローチ

Tailwind CSSは**モバイルファースト**です。プレフィックスなしのクラスはすべての画面サイズに適用され、プレフィックス付きはそのサイズ以上で適用されます。

```

<!-- モバイル: 1列、タブレット: 2列、デスクトップ: 3列 -->
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  <div>カード1</div>
  <div>カード2</div>
  <div>カード3</div>
</div>
<!-- → モバイル: カードが縦1列 / タブレット: 2列 / PC: 3列横並び -->

<!-- モバイル: テキスト小、デスクトップ: テキスト大 -->
<h1 class="text-2xl lg:text-4xl font-bold">
  BON-LOG
</h1>
<!-- → モバイル: 24pxの見出し / PC: 36pxの大見出しに拡大 -->

<!-- モバイル: 縦並び、デスクトップ: 横並び -->
<div class="flex flex-col lg:flex-row gap-4">
  <div>左側</div>
  <div>右側</div>
</div>
<!-- → モバイル: 上下に積み重なる / PC: 左右に横並びになる -->

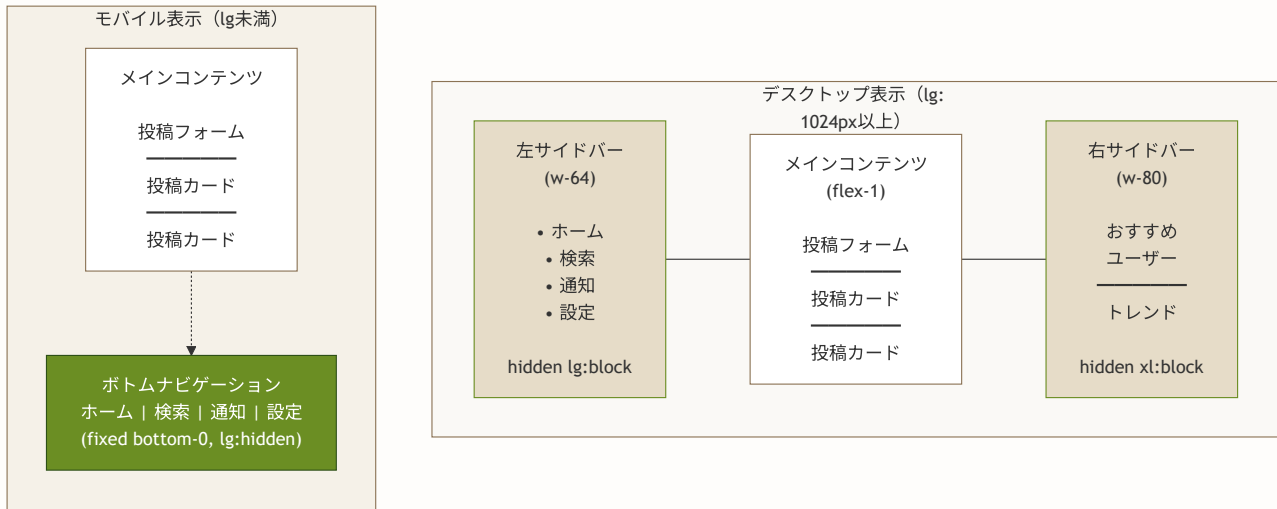
<!-- モバイル: 非表示、デスクトップ: 表示 -->
<aside class="hidden lg:block">
  サイドバー (PCのみ)
</aside>
<!-- → モバイル: 完全に非表示 / PC: ブロック要素として出現 -->

<!-- モバイル: 表示、デスクトップ: 非表示 -->
<nav class="block lg:hidden">
  モバイルメニュー
</nav>
<!-- → モバイル: 表示される / PC: 完全に非表示 (代わりにサイドバーが機能) -->

```

実用例: 3カラムレイアウト

BON-LOGの3カラムレイアウト構成図



```
<!-- BON-LOGのメインレイアウト -->
<div class="flex min-h-screen">
  <!-- 左サイドバー（モバイルでは非表示） -->
  <aside class="hidden lg:block w-64 border-r border-gray-200 p-4">
    <nav class="space-y-2">
      <a href="/feed" class="block p-2 rounded hover:bg-gray-100">
        タイムライン
      </a>
      <a href="/search" class="block p-2 rounded hover:bg-gray-100">
        検索
      </a>
    </nav>
  </aside>

  <!-- メインコンテンツ -->
  <main class="flex-1 max-w-full lg:max-w-2xl border-x border-gray-200">
    <!-- コンテンツ -->
  </main>

  <!-- 右サイドバー（モバイル・タブレットでは非表示） -->
  <aside class="hidden xl:block w-80 p-4">
    <div class="bg-gray-50 rounded-lg p-4">
      おすすめユーザー
    </div>
  </aside>
</div>

<!-- モバイル用ボトムナビゲーション -->
<nav class="fixed bottom-0 left-0 right-0 bg-white border-t border-gray-200 lg:hidden">
  <div class="flex justify-around p-2">
    <a href="/feed">ホーム</a>
    <a href="/search">検索</a>
    <a href="/notifications">通知</a>
    <a href="/settings">設定</a>
  </div>
</nav>
```

画面表示 この3カラムレイアウトは画面幅に応じて大きく変化します:

モバイル（1024px未満）:

- 左右のサイドバーは非表示（`hidden`）。メインコンテンツが画面幅いっぱいに表示
- 画面下部に白い帯（ボトムナビ）が固定表示。「ホーム」「検索」「通知」「設定」が均等配置（`justify-around`）
- ボトムナビは常に画面の最下部に固定（`fixed bottom-0`）されスクロールしても追従する

デスクトップ（1024px以上）:

- 左サイドバー（幅256px）が出現し、ナビゲーションリンクが縦に並ぶ
- ボトムナビは非表示になる（`lg:hidden`）

ワイドデスクトップ（1280px以上）:

- 右サイドバー（幅320px）も出現し、おすすめユーザーなどの補助情報が表示される

よくあるトラブルと解決法

トラブル	原因	解決法
モバイルでサイドバーが表示されてしまう	<code>hidden lg:block</code> ではなく <code>block</code> と書いている	デフォルトを <code>hidden</code> にし、 <code>lg:block</code> で大画面のみ表示
レスポンスが逆に動く（大画面で非表示）	<code>lg:hidden</code> と書いてしまっている	モバイルファーストで考え直す。「モバイルで非表示→大画面で表示」なら <code>hidden lg:block</code>
ボトムナビが他のコンテンツに重なる	<code>fixed</code> でコンテンツの上に重なっている	メインコンテンツに <code>pb-16</code> 等の下部パディングを追加

理解度チェック

▶ Q1: `hidden lg:block` と `lg:hidden` の違いは何ですか？

▶ Q2: BON-LOGの3カラムレイアウトで、右サイドバーがxl以上でのみ表示される理由は何ですか？

6.4 ダークモード対応

このセクションで学ぶこと

- Tailwind CSS v4でのダークモード設定方法（`@custom-variant`）
- `dark:` プレフィックスの使い方
- JavaScriptを使ったダークモード切り替えの実装

Tailwind CSSでは`dark:` プレフィックスでダークモードのスタイルを指定できます。

設定

Tailwind CSS v4では、従来の`tailwind.config.ts`の`darkMode: 'class'`に代わり、`@custom-variant`ディレクティブを使用してダークモードを設定します。

```
/* app/globals.css (BON-LOGの実際のコード) */
@import "tailwindcss";

/* ダークモードのカスタムバリエーション定義 */
/* .dark クラスの子孫要素に dark: プレフィックスのスタイルを適用 */
@custom-variant dark (&:is(.dark *));
```

Tailwind CSS v3との違い: v3では`tailwind.config.ts`に`darkMode: 'class'`と書いていましたが、v4ではCSSファイル内の`@custom-variant`で定義します。設定ファイル（JavaScript/TypeScript）ではなくCSSファイルで完結するのがv4のCSS-firstアーキテクチャの特徴です。

BON-LOGでの使用箇所: `app/globals.css`（3-4行目）

実装しない場合の影響: `dark:` プレフィックスのクラスが一切動作せず、ダークモードに切り替えてもスタイルが変わりません。ライトモードの色がそのまま表示され、暗い環境での閲覧が目には辛くなります。

使用例

```

<!-- ライト：白背景、ダーク：黒背景 -->
<div class="bg-white dark:bg-gray-900 text-gray-900 dark:text-white">
  <h1 class="text-2xl font-bold">BON-LOG</h1>
  <p class="text-gray-600 dark:text-gray-400">
    盆栽愛好家のためのSNS
  </p>
</div>

<!-- カード -->
<div class="bg-white dark:bg-gray-800 border border-gray-200 dark:border-gray-700 rounded-
  <p class="text-gray-900 dark:text-white">カードコンテンツ</p>
</div>

<!-- ボタン -->
<button class="bg-blue-500 hover:bg-blue-600 dark:bg-blue-600 dark:hover:bg-blue-700 text-
  クリック
</button>

```

画面表示 ライトモードとダークモードの見え方の違い:

ライトモード（通常時）：

- 背景: 白 (`bg-white`)、テキスト: 濃いグレー (`text-gray-900`)、サブテキスト: 中間グレー (`text-gray-600`)
- カード: 白背景に薄いグレーの枠線 (`border-gray-200`)
- ボタン: 青い背景 (`bg-blue-500`) に白文字

ダークモード（`<html class="dark">` 時）：

- 背景: ほぼ黒 (`dark:bg-gray-900`)、テキスト: 白 (`dark:text-white`)、サブテキスト: 薄めのグレー (`dark:text-gray-400`)
- カード: 暗いグレー背景 (`dark:bg-gray-800`) にさらに暗い枠線 (`dark:border-gray-700`)
- ボタン: やや暗い青 (`dark:bg-blue-600`) に白文字
- 全体的にコントラストが保たれつつ、目に優しい暗色系に切り替わる

ダークモードのちらつき（FOUC）対策 `useEffect` でテーマを読み込むと、初回表示時に一瞬ライトモードが表示されてからダークモードに切り替わる「ちらつき」が発生します。BON-LOGでは `ThemeProvider` が `<html>` タグの `class` 属性を制御し、`next-themes` ライブラリがこの問題を自動的に解決しています。

ダークモード切り替え実装

```
// components/ThemeToggle.tsx
'use client'

import { useEffect, useState } from 'react'

export function ThemeToggle() {
  const [isDark, setIsDark] = useState(false)

  useEffect(() => {
    // ローカルストレージから設定を読み込み
    const theme = localStorage.getItem('theme')
    if (theme === 'dark') {
      document.documentElement.classList.add('dark')
      setIsDark(true)
    }
  }, [])

  const toggleTheme = () => {
    if (isDark) {
      document.documentElement.classList.remove('dark')
      localStorage.setItem('theme', 'light')
      setIsDark(false)
    } else {
      document.documentElement.classList.add('dark')
      localStorage.setItem('theme', 'dark')
      setIsDark(true)
    }
  }

  return (
    <button
      onClick={toggleTheme}
      className="p-2 rounded-lg bg-gray-200 dark:bg-gray-700"
    >
      {isDark ? '☀️' : '🌙'}
    </button>
  )
}
```

画面表示 ライトモード時: 薄いグレー背景（`bg-gray-200`）のボタンに月のアイコンが表示される。クリックするとダークモードに切り替わる。ダークモード時: 暗いグレー背景（`dark:bg-gray-700`）に太陽のアイコンが表示される。クリックでライトモードに戻る。

理解度チェック

▶ Q1: ``darkMode: 'class'`` と ``darkMode: 'media'`` の違いは何ですか？

▶ Q2: ダークモード切り替えで ``localStorage`` を使う理由は何ですか？

6.5 カスタムテーマ

このセクションで学ぶこと

- `tailwind.config.ts`でカスタムカラーを定義する方法
- BON-LOGの和風カラーパレットの意味と使い方
- カスタムスペーシングやフォントの追加方法

BON-LOGでは和風の配色を使用します。

カスタムカラーの追加

```
// tailwind.config.ts
import type { Config } from 'tailwindcss'

const config: Config = {
  content: [
    './pages/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
    './app/**/*.{js,ts,jsx,tsx,mdx}',
  ],
  theme: {
    extend: {
      colors: {
        // 和風カラーパレット
        'bonsai-cream': '#f5f1e8', // クリーム色（背景）
        'bonsai-moss': '#6b8e23', // 苔色（アクセント）
        'bonsai-earth': '#8b7355', // 土色（サブカラー）
        'bonsai-pine': '#2d5016', // 松葉色（濃い緑）
        'sumi': '#1a1a1a', // 墨色（テキスト）
        'sumi-light': '#4a4a4a', // 薄墨色（サブテキスト）
        'washi': '#faf9f6', // 和紙色（カード背景）
        'kinari': '#e6dcc8', // 生成り色（ボーダー）
      },
      fontFamily: {
        sans: ['Noto Sans JP', 'sans-serif'],
      },
    },
  },
  plugins: [],
}

export default config
```

カスタムカラーの使用

```

<!-- 和風配色を使用したカード -->
<div class="bg-washi border border-kinari rounded-lg p-6 shadow-md">
  <h2 class="text-xl font-bold text-sumi mb-2">
    五葉松の育成記録
  </h2>
  <p class="text-sumi-light mb-4">
    今年の春、新芽が美しく伸びました。
  </p>
  <button class="bg-bonsai-moss hover:bg-bonsai-pine text-white px-4 py-2 rounded transition-colors">
    詳細を見る
  </button>
</div>

<!-- 和風背景 -->
<div class="min-h-screen bg-bonsai-cream">
  <!-- ページコンテンツ -->
</div>

```

画面表示 このカードは和風の色調で以下のように表示されます:

- 背景: 和紙色 (`bg-washi` = わずかに黄みを帯びた白 `#faf9f6`) で温かみのある印象
- 枠線: 生成り色 (`border-kinari` = 薄いベージュ `#e6dcc8`) で柔らかな境界線
- 影: 中程度の影 (`shadow-md`) でカードが浮いた立体感
- タイトル: 墨色 (`text-sumi` = 温かみのある黒 `#1a1a1a`) で凛とした印象。純粋な黒ではない
- 本文: 薄墨色 (`text-sumi-light` = 暗いグレー `#4a4a4a`) で落ち着いた読みやすさ
- ボタン: 苔色 (`bg-bonsai-moss` = 黄緑がかった緑) で和の自然を感じる。ホバーで松葉色に滑らかに変化 (`transition-colors`)
- ページ全体の背景はクリーム色 (`bg-bonsai-cream` = `#f5f1e8`) で、白とは異なる温かい紙のような質感

カスタムスペーシング

```
// tailwind.config.ts
const config: Config = {
  theme: {
    extend: {
      spacing: {
        '18': '4.5rem', // 72px
        '88': '22rem', // 352px
        '128': '32rem', // 512px
      },
    },
  },
},
}
```

理解度チェック

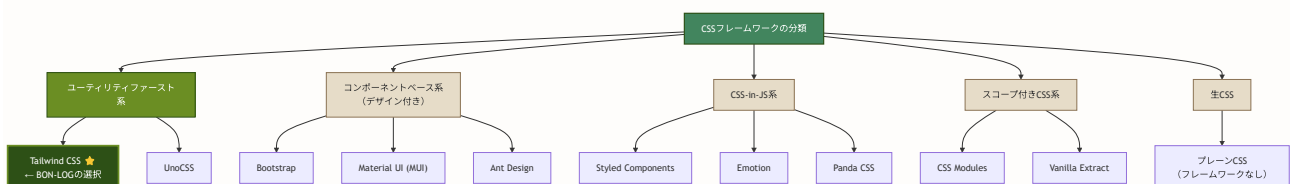
▶ Q1: `theme.extend.colors` に色を追加するのと、`theme.colors` に色を追加するのでは何が違いますか？

▶ Q2: カスタムカラー名に `bonsai-moss` のようにハイフンを含めた場合、使い方は？

補足E: CSSフレームワークの選択肢と比較

Webアプリケーションのスタイリングにはさまざまなアプローチがあります。BON-LOGではTailwind CSSを採用していますが、なぜこの選択をしたのかを他の選択肢と比較して理解しましょう。

主要なCSSフレームワーク / アプローチ一覧



比較表

特性	Tailwind CSS	Bootstrap	Material UI	CSS Modules	Styled Components	生 CSS
学習コスト	中	低	中	低	中	低
カスタマイズ性	高	中	低～中	高	高	高
バンドルサイズ	小	中	大	小	中	小
開発速度	高	高	高	中	中	低
デザインの自由度	高	低～中	低	高	高	高
型安全性	中	低	高	中	中	低
SSR対応	完全	完全	要設定	完全	要設定	完全
React Server Components対応	完全	完全	制限あり	完全	非対応	完全

各選択肢の特徴

Bootstrap

メリット: 豊富な既製デザイン、ドキュメントが充実、学習コストが低い
 デメリット: 「Bootstrap臭い」既視感のあるデザインになりがち、
 カスタマイズが難しい、未使用CSSが多く含まれる
 向いている場面: 管理画面、プロトタイプ、デザインにこだわらない場合

Material UI (MUI)

メリット: Googleのデザインガイドラインに準拠、豊富なコンポーネント、
 テーマシステムが強力
 デメリット: バンドルサイズが大きい、Material Designの見た目に引きずられる、
 Next.js App Router/Server Componentsとの相性に課題がある
 向いている場面: Googleライクなデザインが求められるBtoB製品

CSS Modules

メリット：標準的なCSSの知識だけで使える、クラス名の衝突を防げる、バンドルサイズが小さい
 デメリット：ファイル数が増える（各コンポーネントにCSSファイル）、デザインの統一性を自分で管理する必要がある
 向いている場面：CSS知識がある中～大規模チーム

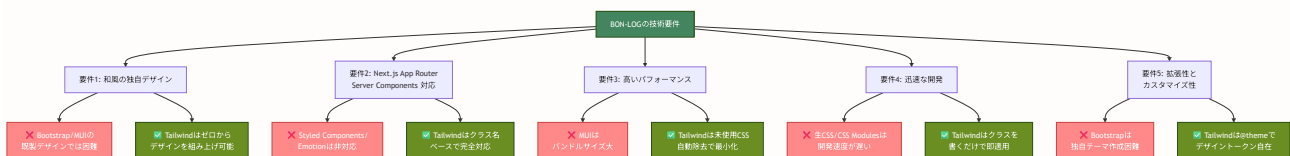
Styled Components / Emotion

メリット：JavaScriptの中でCSSが完結、動的なスタイルが書きやすい、コンポーネントのスコープが自然に効く
 デメリット：ランタイムでCSSを生成するためパフォーマンスに影響、React Server Componentsでは使えない（大きな制約）
 向いている場面：Client Componentsが中心の小～中規模アプリ

生CSS（フレームワークなし）

メリット：追加ライブラリが不要、CSSの仕様をそのまま活用できる、バンドルサイズへの影響が最小
 デメリット：開発速度が遅い、一貫性の維持が難しい、大規模プロジェクトでの管理が大変
 向いている場面：非常に小規模なサイト、CSSの学習目的

BON-LOGがTailwind CSSを選んだ理由



補足F: UIコンポーネントライブラリの選択肢と比較

主要なUIコンポーネントライブラリ

ライブラリ	アプローチ	スタイル	アクセシビリティ
shadcn/ui	コピー&ペースト	Tailwind CSS	Radix UIベースで高い
Radix UI	ヘッドレス（スタイルなし）	自分で付ける	非常に高い
Headless UI	ヘッドレス（スタイルなし）	自分で付ける	高い
Material UI (MUI)	フルパッケージ	MUI独自 + Emotion	高い
Ant Design	フルパッケージ	Less/CSS-in-JS	中程度
daisyUI	Tailwindプラグイン	Tailwind CSS	中程度
Chakra UI	フルパッケージ	Emotion / Panda CSS	高い

比較のポイント



BON-LOGがshadcn/uiを選んだ理由

- コピー&ペースト方式:** コンポーネントのソースコードが完全に手元にあるため、和風テーマへの大幅なカスタマイズが可能。MUIやAnt Designではライブラリの制約内でのカスタマイズに限られる
- Radix UIベースのアクセシビリティ:** shadcn/uiの内部ではRadix UIが使われており、キーボード操作、スクリーンリーダー対応、フォーカス管理などが標準で組み込まれている。自前で実装すると膨大な工数がかかる部分
- Tailwind CSSとの完全な統合:** shadcn/uiのスタイルはすべてTailwind CSSのクラスで定義されているため、BON-LOGのカスタムテーマ（和風カラーパレット）とシームレスに統合できる
- Next.js App Routerとの互換性:** Server Componentsでもclient Componentsでも問題なく動作する。CSS-in-JS系ライブラリ（Emotion等）を使うMUIやChakra UIでは、Server Componentsでの利用に制限がある

5. **軽量**: 必要なコンポーネントだけをプロジェクトに追加するため、使わないコンポーネントがバンドルに含まれない

補足G: CSS-in-JSツール（CVA）の選択肢と比較

CVAの位置づけ

CVA（class-variance-authority）は厳密にはCSS-in-JSライブラリではありませんが、「バリエーション管理」という同じ問題領域を扱うツールとして、関連するライブラリと比較します。

ツール	アプローチ	ランタイム	Tailwind連携	型安全
CVA	クラス名のバリエーション管理	ゼロランタイム	最適	高い
Stitches	CSS-in-JSでバリエーション管理	ランタイムあり	不可	高い
Vanilla Extract	ビルド時CSS生成	ゼロランタイム	限定的	非常に高い
Panda CSS	ビルド時CSS生成 + ユーティリティ	ゼロランタイム	不可（代替）	非常に高い

BON-LOGがCVAを選んだ理由

CVAが解決する問題：

コンポーネントの「variant(見た目の種類)」と「size(サイズ)」の組み合わせを型安全に管理すること

他のツールとの違い：

- ・ Stitches: 独自のCSS-in-JSシステムを必要とする。
Tailwind CSSと併用できない
- ・ Vanilla Extract: 強力だが学習コストが高い。
Tailwind CSSとの統合が自然ではない
- ・ Panda CSS: Tailwind CSSの代替として設計されている。
Tailwind CSSと併用する設計ではない

CVAが最適な理由：

- ・ Tailwind CSSのクラス名をそのまま使える
- ・ shadcn/uiが内部で採用しており、統一的な設計になる
- ・ ゼロランタイムで追加のパフォーマンスコストがない
- ・ TypeScriptのVariantProps型でバリエーションの型を自動生成

補足H: アイコンライブラリの選択肢と比較

主要なアイコンライブラリ

ライブラリ	アイコン数	サイズ	スタイル	Tree-shaking
Lucide React	1,500+	小	線画（アウトライン）	対応
React Icons	40,000+	大	複数スタイル混在	部分対応
Heroicons	300+	小	アウトライン/塗り	対応
Font Awesome	2,000+(無料)	中～大	複数スタイル	部分対応
Phosphor Icons	7,000+	中	6種類のスタイル	対応

※ Tree-shaking: 使わないアイコンをビルド時に自動除去する仕組み

BON-LOGがLucide Reactを選んだ理由

1. shadcn/uiの標準アイコン
 - shadcn/uiが公式にLucide Reactを推奨しており、コンポーネント例やドキュメントでも使われている
 - 同じアイコンを使うことで統一感が保たれる
2. 一貫したデザイン言語
 - 全アイコンが同じ線幅(stroke-width)、角丸、サイズ感で設計されており、統一感のあるUIが作りやすい
 - React Iconsは複数のアイコンセットの寄せ集めのため、スタイルにばらつきが出やすい
3. 軽量のバンドルサイズ
 - Tree-shakingに完全対応しており、使ったアイコンだけがバンドルに含まれる
 - React Iconsはパッケージ全体で数MBになることがある
4. 和風デザインとの相性
 - 繊細な線画スタイルがBON-LOGの和風テイストと調和する
 - 太い線や塗りつぶしスタイルのアイコンは和のテイストに合いにくい
5. カスタマイズ性
 - size、color、strokeWidthなどのpropsで細かく調整可能
 - className経由でTailwind CSSのクラスも適用できる

使用例:

```
import { Heart, MessageCircle, Bookmark, Share2 } from 'lucide-react'

// 投稿カードのアクションボタン
<div className="flex gap-4">
  <button className="flex items-center gap-1 text-gray-500 hover:text-red-500">
    <Heart className="h-5 w-5" />    {/* → 20x20pxのハートアイコン（線画） */}
    <span>12</span>
  </button>
  <button className="flex items-center gap-1 text-gray-500 hover:text-blue-500">
    <MessageCircle className="h-5 w-5" />  {/* → 20x20pxの吹き出しアイコン（線画） */}
    <span>3</span>
  </button>
</div>
```

画面表示 いいねボタンとコメントボタンが16px間隔で横並び。通常はグレー（`text-gray-500`）の線画アイコン + 数字。ハートアイコンにホバーすると赤色（`hover:text-red-500`）に、吹き出しアイコンにホバーすると青色（`hover:text-blue-500`）に変化する。

6.6 shadcn/uiとは

このセクションで学ぶこと

- shadcn/uiの基本的な考え方（コピー&ペーストアプローチ）
- 従来のUIライブラリ（Material-UI等）との違い
- shadcn/uiが採用するRadix UI + Tailwind CSSのアーキテクチャ

shadcn/uiは、Radix UIとTailwind CSSを組み合わせた**コンポーネントコレクション**です。

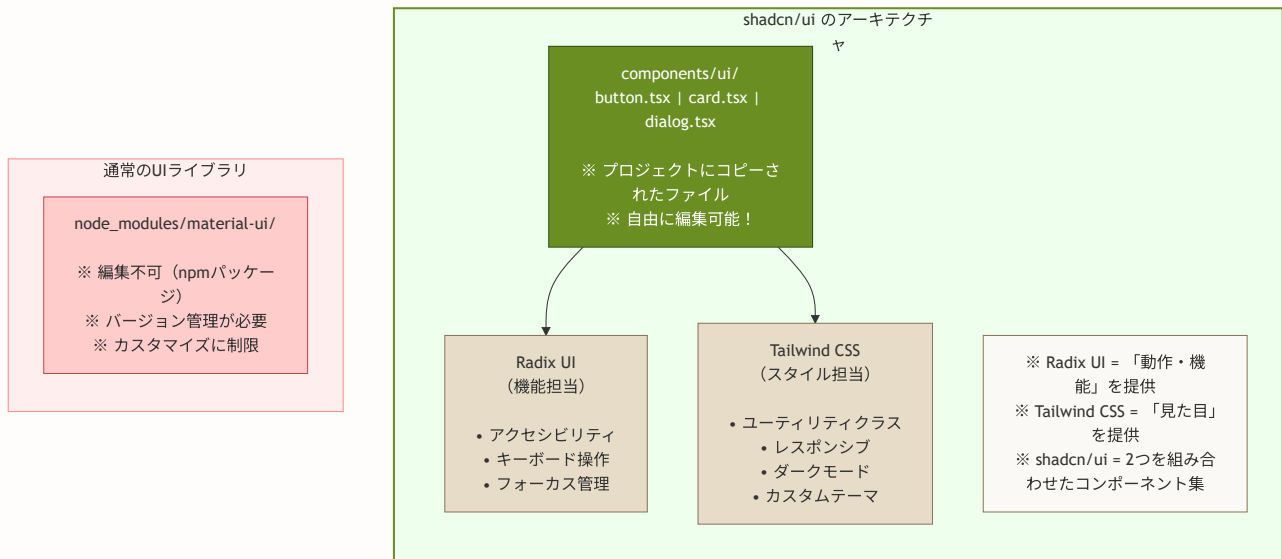
特徴

1. **コピー&ペースト**: npm installではなく、コンポーネントをプロジェクトにコピー
2. **完全なカスタマイズ性**: コードが自分のプロジェクトに入るので自由に編集可能
3. **アクセシビリティ**: Radix UIベースで高いアクセシビリティ
4. **Tailwind CSS**: スタイリングはすべてTailwind

ライブラリとの違い

項目	通常のライブラリ（Material-UI等）	shadcn/ui
インストール方法	npm installで追加	CLIでコンポーネントをコピー
保存場所	node_modulesに保存	componentsディレクトリに保存
カスタマイズ性	カスタマイズが難しい	完全にカスタマイズ可能
バージョン管理	バージョン管理が必要	自分で管理（バージョンアップは手動）

shadcn/uiのアーキテクチャ図



理解度チェック

▶ Q1: shadcn/uiのコンポーネントはどこに保存されますか？

▶ Q2: shadcn/uiが「Radix UI」を使っている理由は何ですか？

6.7 shadcn/uiのインストールと使い方

このセクションで学ぶこと

- shadcn/uiの初期セットアップ手順
- コンポーネントの追加方法 (CLIの使い方)
- 追加されたコンポーネントの使い方

初期セットアップ

```
# shadcn/uiの初期化
npx shadcn@latest init

# 質問に答える
# ✓ Would you like to use TypeScript? ... yes
# ✓ Which style would you like to use? > Default
# ✓ Which color would you like to use as base color? > Slate
# ✓ Where is your global CSS file? ... app/globals.css
# ✓ Would you like to use CSS variables for colors? ... yes
# ✓ Where is your tailwind.config.js located? ... tailwind.config.ts
# ✓ Configure the import alias for components? ... @/components
# ✓ Configure the import alias for utils? ... @/lib/utils
```

コンポーネントの追加

```
# Buttonコンポーネントを追加
npx shadcn@latest add button

# 複数まとめて追加
npx shadcn@latest add button input card dialog
```

追加されたコンポーネントは `components/ui/` に保存されます。

```
components/
├─ ui/
│   ├─ button.tsx
│   ├─ input.tsx
│   ├─ card.tsx
│   └─ dialog.tsx
```

コンポーネントの使用

```
// app/page.tsx
import { Button } from '@components/ui/button'

export default function Home() {
  return (
    <div>
      <Button>クリック</Button>
      <Button variant="destructive">削除</Button>
      <Button variant="outline">アウトライン</Button>
      <Button variant="ghost">ゴースト</Button>
    </div>
  )
}
```

よくあるトラブルと解決法

トラブル	原因	解決法
<code>npx shadcn@latest add</code> でエラー	<code>components.json</code> が存在しない	先に <code>npx shadcn@latest init</code> を実行する
コンポーネントが見つからないエラー	importパスが間違っている	<code>@/components/ui/button</code> のようにエイリアスパスを使用
スタイルが適用されない	globals.cssにTailwindのディレクティブがない	<code>@tailwind base; @tailwind components; @tailwind utilities;</code> を確認

理解度チェック

▶ Q1: `npx shadcn@latest add button` を実行すると何が起きますか？

▶ Q2: shadcn/uiで追加したコンポーネントを更新したい場合はどうしますか？

6.8 主要コンポーネント

このセクションで学ぶこと

- Button、Input、Card、Dialog等の基本的なコンポーネントの使い方
- 各コンポーネントのvariant（バリエーション）やsize（サイズ）の指定方法
- 実際のUIでどのように組み合わせるかのパターン

Button

```
import { Button } from '@components/ui/button'

export function ButtonDemo() {
  return (
    <div className="space-y-4">
      {/* バリエント */}
      <Button variant="default">デフォルト</Button>
      <Button variant="destructive">削除</Button>
      <Button variant="outline">アウトライン</Button>
      <Button variant="secondary">セカンダリ</Button>
      <Button variant="ghost">ゴースト</Button>
      <Button variant="link">リンク</Button>

      {/* サイズ */}
      <Button size="sm">小</Button>
      <Button size="default">標準</Button>
      <Button size="lg">大</Button>

      {/* 無効化 */}
      <Button disabled>無効</Button>

      {/* アイコン付き */}
      <Button>
        <svg className="mr-2 h-4 w-4" /* ... */>
          メール送信
        </svg>
      </Button>

      {/* ローディング */}
      <Button disabled>
        <svg className="mr-2 h-4 w-4 animate-spin" /* ... */>
          送信中 ...
        </svg>
      </Button>
    </div>
  )
}
```

画面表示 各バリエーションのボタンは以下のように表示されます:

- **default:** 松葉色（プライマリカラー）の塗りつぶし背景に白文字。最も目立つメインアクション用
- **destructive:** 朱色（赤系）の塗りつぶし背景に白文字。「削除」「通報」など危険な操作用
- **outline:** 白背景に枠線のみ。背景色が透明で控えめ。補助的なアクション用
- **secondary:** 亜麻色（淡いベージュ）の塗りつぶし背景。「キャンセル」など副次的な操作用
- **ghost:** 背景なし・枠線なし。テキストだけが表示される。ホバーすると薄い背景色が出現。ナビゲーションやアイコンボタン向き
- **link:** テキストのみ表示（下線なし）。ホバーすると下線が出現。テキストリンクの代替

サイズは3段階:

- **sm:** 高さ32px、小さめのパディング。カード内のアクションボタン向き
- **default:** 高さ36px、標準パディング。一般的なフォーム送信ボタン向き
- **lg:** 高さ40px、大きめのパディング。ログインやCTAなど目立たせたいボタン向き

disabled状態では半透明（`opacity-50`）になり、クリック不可に。ローディング中はスピナーアイコンが回転する。

Input

```
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'

export function InputDemo() {
  return (
    <div className="space-y-4">
      { /* 基本 */ }
      <Input type="text" placeholder="メールアドレス" />

      { /* ラベル付き */ }
      <div>
        <Label htmlFor="email">メールアドレス</Label>
        <Input id="email" type="email" />
      </div>

      { /* 無効化 */ }
      <Input disabled placeholder="無効" />

      { /* ファイル */ }
      <Input type="file" />
    </div>
  )
}
```

画面表示 4つの入力欄が16px間隔で縦に並びます:

- **基本:** 枠線付きの入力欄。プレースホルダー「メールアドレス」がグレーで表示。フォーカスすると枠線が松葉色に変化しリング（輪郭線）が表示される
- **ラベル付き:** 「メールアドレス」のラベルが入力欄の上に小さめの太字で表示される
- **無効化:** グレーアウトして操作不可。テキストと背景が薄くなる
- **ファイル:** 「ファイルを選択」ボタンが入力欄内に表示される

Card

```
import {
  Card,
  CardContent,
  CardDescription,
  CardFooter,
  CardHeader,
  CardTitle,
} from '@components/ui/card'
import { Button } from '@components/ui/button'

export function CardDemo() {
  return (
    <Card className="w-full max-w-md">
      <CardHeader>
        <CardTitle>五葉松の育成記録</CardTitle>
        <CardDescription>2026年2月8日</CardDescription>
      </CardHeader>
      <CardContent>
        <p>今年の春、新芽が美しく伸びました。針金かけも順調です。</p>
      </CardContent>
      <CardFooter className="flex justify-between">
        <Button variant="outline">編集</Button>
        <Button>詳細を見る</Button>
      </CardFooter>
    </Card>
  )
}
```

画面表示 このCardコンポーネントは以下のように表示されます：

- 最大幅448px (`max-w-md`) の白いカードで、枠線・角丸・薄い影がある
- **CardHeader:** 上部にタイトル「五葉松の育成記録」(太字・大きめ)と日付「2026年2月8日」(小さめ・グレー)
- **CardContent:** 中央に本文テキストが表示される
- **CardFooter:** 下部に2つのボタンが左右に分かれて配置 (`flex justify-between`)
 - 左側: 「編集」ボタン (枠線のためのoutlineスタイル)
 - 右側: 「詳細を見る」ボタン (松葉色の塗りつぶし)
- カード全体は和紙色 (`bg-card`) の背景で、BON-LOGの落ち着いた色調に馴染む

Dialog (モーダル)

```
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
  DialogTrigger,
} from '@components/ui/dialog'
import { Button } from '@components/ui/button'

export function DialogDemo() {
  return (
    <Dialog>
      <DialogTrigger asChild>
        <Button>投稿する</Button>
      </DialogTrigger>
      <DialogContent>
        <DialogHeader>
          <DialogTitle>新しい投稿</DialogTitle>
          <DialogDescription>
            今日の盆栽について語りましょう（500文字まで）
          </DialogDescription>
        </DialogHeader>
        <textarea className="w-full border rounded p-2" rows={5} />
        <DialogFooter>
          <Button variant="outline">キャンセル</Button>
          <Button>投稿する</Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  )
}
```

画面表示 Dialogは以下のように動作・表示されます:

- 「投稿する」 ボタンをクリックすると、画面中央にモーダルが表示される
- 背後は半透明の黒いオーバーレイで覆われ、モーダルに注目が集まる
- モーダル内部:
 - **ヘッダー**: 「新しい投稿」というタイトルと説明文
 - **本文**: テキストエリア（5行分の高さ）が配置され、すぐに入力可能
 - **フッター**: 「キャンセル」(outline) と「投稿する」(primary) の2つのボタン
- モーダルの外側をクリックするか、Escキーを押すと閉じる
- フォーカスがモーダル内に閉じ込められ（フォーカストラップ）、Tabキーでモーダル外の要素にフォーカスが移らない（Radix UIのアクセシビリティ機能）

DropDownMenu

```
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from '@components/ui/dropdown-menu'
import { Button } from '@components/ui/button'

export function DropdownMenuDemo() {
  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="outline">メニュー</Button>
      </DropdownMenuTrigger>
      <DropdownMenuContent>
        <DropdownMenuLabel>マイアカウント</DropdownMenuLabel>
        <DropdownMenuSeparator />
        <DropdownMenuItem>プロフィール</DropdownMenuItem>
        <DropdownMenuItem>設定</DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuItem className="text-red-600">
          ログアウト
        </DropdownMenuItem>
      </DropdownMenuContent>
    </DropdownMenu>
  )
}
```

画面表示 DropdownMenuは以下のように動作・表示されます:

- 「メニュー」 ボタンをクリックすると、ボタンの下にドロップダウンリストが表示される
- リスト上部に「マイアカウント」のラベル（グレー・小さめ）、その下に区切り線
- 「プロフィール」「設定」の項目がリスト状に並び、ホバーすると背景が薄く変化
- 再度区切り線の後に「ログアウト」が赤文字（ `text-red-600` ）で表示され、危険な操作であることが視覚的に伝わる
- メニューの外側をクリックするか、Escキーで閉じる。上下矢印キーでの移動にも対応

Avatar

```
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar'

export function AvatarDemo() {
  return (
    <div className="flex gap-4">
      {/* 画像あり */}
      <Avatar>
        <AvatarImage src="/avatars/user1.jpg" alt="@user1" />
        <AvatarFallback>U1</AvatarFallback>
      </Avatar>

      {/* フォールバックのみ */}
      <Avatar>
        <AvatarFallback>佐藤</AvatarFallback>
      </Avatar>

      {/* サイズ指定 */}
      <Avatar className="h-16 w-16">
        <AvatarImage src="/avatars/user2.jpg" />
        <AvatarFallback>U2</AvatarFallback>
      </Avatar>
    </div>
  )
}
```

画面表示 3つのアバターが横に16px間隔（`gap-4`）で並びます：

- **1つ目:** 丸い画像（デフォルト40x40px）。画像読み込み失敗時は「U1」の2文字がグレー背景の円に表示
- **2つ目:** 画像がないため「佐藤」という文字がフォールバックとして表示される
- **3つ目:** `h-16 w-16` で大きめ（64x64px）の丸い画像。プロフィールページのアバター等に使用

Select

```
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from '@components/ui/select'

export function SelectDemo() {
  return (
    <Select>
      <SelectTrigger className="w-64">
        <SelectValue placeholder="ジャンルを選択" />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="pine">松柏類</SelectItem>
        <SelectItem value="deciduous">雑木類</SelectItem>
        <SelectItem value="flowering">花物</SelectItem>
        <SelectItem value="fruit">実物</SelectItem>
        <SelectItem value="tools">用品・道具</SelectItem>
      </SelectContent>
    </Select>
  )
}
```

画面表示 幅256px (`w-64`) のセレクトボックスが表示される。初期状態では「ジャンルを選択」のプレースホルダーがグレーで表示。クリックするとドロップダウンリストが開き、5つのジャンルが縦に並ぶ。各項目にマウスを重ねると背景色が変わり、クリックで選択される。選択後はトリガー部分に選択したジャンル名が表示される。

Textarea

```
import { Textarea } from '@components/ui/textarea'

export function TextareaDemo() {
  return (
    <div className="space-y-2">
      <label htmlFor="content" className="text-sm font-medium">
        投稿内容
      </label>
      <Textarea
        id="content"
        placeholder="今日の盆栽について語りましょう"
        rows={5}
        maxLength={500}
      />
      <p className="text-sm text-gray-500">500文字まで</p>
    </div>
  )
}
```

Badge

```
import { Badge } from '@components/ui/badge'

export function BadgeDemo() {
  return (
    <div className="flex gap-2">
      <Badge>デフォルト</Badge>          { /* → 松葉色の塗りつぶし丸型タグ */ }
      <Badge variant="secondary">セカンダリ</Badge>    { /* → 亜麻色の控えめな丸型タグ */ }
      <Badge variant="destructive">削除</Badge>         { /* → 朱色の警告用丸型タグ */ }
      <Badge variant="outline">アウトライン</Badge>     { /* → 枠線のための軽い丸型タグ */ }
    </div>
  )
}
```

画面表示 4つの小さな丸型タグ（`rounded-full`）が横に8px間隔で並ぶ。各バリエントは背景色と文字色の組み合わせが異なり、ジャンルタグ、ステータス表示、カテゴリラベルなどに使い分ける。BON-LOGでは投稿のジャンル表示（「松柏類」「雑木類」等）にsecondaryバリエントを使用。

理解度チェック

▶ Q1: Buttonコンポーネントの`variant="ghost"`はどのような見た目ですか？

▶ Q2: DialogコンポーネントのDialogTriggerに`asChild`を付ける理由は何ですか？

6.9 BON-LOGでの実例

このセクションで学ぶこと

- PostCard（投稿カード）コンポーネントの実際のスタイリング
- 認証ページ（ログイン画面）のレイアウト実装
- 投稿フォームのインタラクティブなUI実装

PostCardのスタイリング

```
// components/post/PostCard.tsx
import { Card, CardContent, CardFooter, CardHeader } from '@components/ui/card'
import { Avatar, AvatarFallback, AvatarImage } from '@components/ui/avatar'
import { Badge } from '@components/ui/badge'
import { Button } from '@components/ui/button'
import Link from 'next/link'
import Image from 'next/image'

interface PostCardProps {
  post: {
    id: string
    content: string
    createdAt: Date
    user: {
      id: string
      nickname: string
      avatarUrl: string | null
    }
    media: Array<{
      url: string
      type: string
    }>
    genres: Array<{
      genre: {
        id: string
        name: string
      }
    }>
    _count: {
      likes: number
      comments: number
    }
  }
}

export function PostCard({ post }: PostCardProps) {
  return (
    <Card className="mb-4 hover:shadow-md transition-shadow">
      <CardHeader>
        <div className="flex items-center gap-3">
          <Link href={`\users/${post.user.id}`}>
            <Avatar>
              <AvatarImage src={post.user.avatarUrl || undefined} />
              <AvatarFallback>{post.user.nickname[0]}</AvatarFallback>
            </Avatar>
          </Link>
          <div className="flex-1">
            <Link
              href={`\users/${post.user.id}`}

```

```

        className="font-semibold hover:underline"
      >
        {post.user.nickname}
      </Link>
      <p className="text-sm text-gray-500">
        {new Date(post.createdAt).toLocaleDateString('ja-JP')}
      </p>
    </div>
  </div>
</CardHeader>

<CardContent>
  {/* 投稿内容 */}
  <p className="text-gray-900 whitespace-pre-wrap mb-4">
    {post.content}
  </p>

  {/* メディア */}
  {post.media.length > 0 && (
    <div className="grid grid-cols-2 gap-2 mb-4">
      {post.media.map((media, index) => (
        <div key={index} className="relative aspect-square">
          <Image
            src={media.url}
            alt=""
            fill
            className="object-cover rounded-lg"
          />
        </div>
      ))}
    </div>
  )}

  {/* ジャンル */}
  <div className="flex flex-wrap gap-2">
    {post.genres.map(({ genre }) => (
      <Badge key={genre.id} variant="secondary">
        {genre.name}
      </Badge>
    ))}
  </div>
</CardContent>

<CardFooter className="flex justify-between">
  <div className="flex gap-4">
    <Button variant="ghost" size="sm">
      💬 {post._count.comments}
    </Button>
    <Button variant="ghost" size="sm">
      ❤️ {post._count.likes}
    </Button>
  </div>
  <Link href={`\posts/${post.id}`}>

```

```

        <Button variant="ghost" size="sm">
            詳細
        </Button>
    </Link>
</CardFooter>
</Card>
)
}

```

画面表示 PostCardコンポーネントはSNSの投稿カードとして以下のように表示されます:

ヘッダー部分 (CardHeader) :

- 左側に丸いアバター画像 (40x40px)。画像がない場合はニックネームの頭文字がグレー背景に表示される
- アバターの右側にユーザーのニックネーム (太字) と投稿日時 (小さめのグレー文字) が縦に並ぶ
- ニックネームをクリック/ホバーすると下線が表示され、ユーザーページへリンク

コンテンツ部分 (CardContent) :

- 投稿テキストが墨色 (`text-gray-900`) で表示。改行は保持される (`whitespace-pre-wrap`)
- 画像がある場合は2列のグリッドで正方形にトリミングされたサムネイルが並ぶ (`grid-cols-2`)
- ジャンルタグ (「松柏類」「雑木類」等) がセカンダリBadgeとして横並びに表示

フッター部分 (CardFooter) :

- 左側にコメント数 (吹き出しアイコン) といいね数 (ハートアイコン) がghostボタンで横並び
- 右側に「詳細」リンクボタン
- カード全体にマウスを重ねると影が強まる (`hover:shadow-md`) アニメーション付き

認証ページのレイアウト

```
// app/(auth)/login/page.tsx
import { Card, CardContent, CardDescription, CardFooter, CardHeader, CardTitle } from '@d
import { Input } from '@components/ui/input'
import { Label } from '@components/ui/label'
import { Button } from '@components/ui/button'
import Link from 'next/link'

export default function LoginPage() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-bonsai-cream p-4">
      <Card className="w-full max-w-md bg-washi">
        <CardHeader className="text-center">
          <div className="mb-4">
            <h1 className="text-3xl font-bold text-sumi">BON-LOG</h1>
            <p className="text-sm text-sumi-light mt-1">
              盆栽愛好家のためのSNS
            </p>
          </div>
          <CardTitle className="text-2xl">ログイン</CardTitle>
          <CardDescription>
            アカウント情報を入力してログインしてください
          </CardDescription>
        </CardHeader>

        <CardContent>
          <form className="space-y-4">
            <div>
              <Label htmlFor="email">メールアドレス</Label>
              <Input
                id="email"
                type="email"
                placeholder="your@email.com"
                required
              />
            </div>

            <div>
              <Label htmlFor="password">パスワード</Label>
              <Input
                id="password"
                type="password"
                placeholder="....."
                required
              />
            </div>

            <Button
              type="submit"
              className="w-full bg-bonsai-moss hover:bg-bonsai-pine">
```

```
        >
        ログイン
      </Button>
    </form>
  </CardContent>

  <CardFooter className="flex flex-col gap-2">
    <Link
      href="/forgot-password"
      className="text-sm text-bonsai-moss hover:underline"
    >
      パスワードを忘れた場合
    </Link>
    <div className="text-sm text-gray-600">
      アカウントをお持ちでない方は{' '}
      <Link
        href="/register"
        className="text-bonsai-moss font-semibold hover:underline"
      >
        新規登録
      </Link>
    </div>
  </CardFooter>
</Card>
</div>
)
}
```

画面表示 ログインページは以下のように表示されます:

- 画面全体が生成り色 (`bg-bonsai-cream`) の背景で覆われ、和の温かみを感じる
- 画面の中央に和紙色 (`bg-washi`) のカードが配置される (`flex items-center justify-center` で上下左右中央)
- カード上部:
 - 「BON-LOG」のロゴテキスト (`text-3xl font-bold text-sumi` = 30px、太字、墨色)
 - 「盆栽愛好家のためのSNS」のサブタイトル (小さめ、薄墨色)
 - 「ログイン」の見出しと説明文
- カード中部:
 - メールアドレスとパスワードの入力欄 (ラベル付き) が縦に並ぶ (`space-y-4` で16px間隔)
 - 「ログイン」ボタンが全幅 (`w-full`) で表示。苔色 (`bg-bonsai-moss`) でホバー時に松葉色に変化
- カード下部:
 - 「パスワードを忘れた場合」のリンク (松葉色テキスト)
 - 「新規登録」のリンク (松葉色、太字)
- モバイルでも `p-4` により画面端からの余白が確保される

投稿フォーム

```
// components/post/PostForm.tsx
'use client'

import { useState } from 'react'
import { Card, CardContent } from '@components/ui/card'
import { Textarea } from '@components/ui/textarea'
import { Button } from '@components/ui/button'
import { Badge } from '@components/ui/badge'
import { createPost } from '@lib/actions/post'

const GENRES = [
  { id: 'pine', name: '松柏類' },
  { id: 'deciduous', name: '雑木類' },
  { id: 'flowering', name: '花物' },
  { id: 'fruit', name: '実物' },
  { id: 'tools', name: '用品・道具' },
]

export function PostForm() {
  const [content, setContent] = useState('')
  const [selectedGenres, setSelectedGenres] = useState<string[]>([])

  const toggleGenre = (genreId: string) => {
    if (selectedGenres.includes(genreId)) {
      setSelectedGenres(selectedGenres.filter(id => id !== genreId))
    } else if (selectedGenres.length < 3) {
      setSelectedGenres([ ... selectedGenres, genreId ])
    }
  }

  return (
    <Card className="mb-4 bg-washi">
      <CardContent className="pt-6">
        <form action={createPost}>
          <Textarea
            name="content"
            placeholder="今日の盆栽について語りましょう（500文字まで）"
            value={content}
            onChange={(e) => setContent(e.target.value)}
            maxLength={500}
            rows={4}
            className="mb-4"
          />

          <div className="mb-4">
            <p className="text-sm font-medium mb-2">
              ジャンル（最大3つ）
            </p>
            <div className="flex flex-wrap gap-2">
```

```

      {GENRES.map(genre => (
        <Badge
          key={genre.id}
          variant={selectedGenres.includes(genre.id) ? 'default' : 'outline'}
          className="cursor-pointer"
          onClick={() => toggleGenre(genre.id)}
        >
          {genre.name}
        </Badge>
      ))}
    </div>
  </div>

  <div className="flex justify-between items-center">
    <p className="text-sm text-gray-500">
      {content.length} / 500
    </p>
    <Button
      type="submit"
      disabled={!content.trim() || selectedGenres.length === 0}
      className="bg-bonsai-moss hover:bg-bonsai-pine"
    >
      投稿する
    </Button>
  </div>
</form>
</CardContent>
</Card>
)
}

```

画面表示 投稿フォームは和紙色のカードとして以下のように表示されます:

- **テキストエリア:** 4行分の高さで、プレースホルダー「今日の盆栽について語りましょう（500文字まで）」がグレーで表示
- **ジャンル選択:** 「松柏類」「雑木類」「花物」「実物」「用品・道具」の5つのバッジが横並び
 - 未選択: アウトライン（枠線のみ）で控えめに表示。カーソルは指マーク（`cursor-pointer`）
 - 選択済み: 塗りつぶしのdefaultスタイルで目立つ表示に切り替わる
 - 最大3つまで選択可能
- **フッター:** 左に「0 / 500」の文字数カウンター（グレー、小さめ）、右に「投稿する」ボタン
- テキスト未入力またはジャンル未選択時はボタンが半透明のdisabled状態

理解度チェック

▶ Q1: PostCardの画像表示部分で`aspect-square`と`object-cover`を組み合わせている理由は何ですか？

▶ Q2: ログインページで`min-h-screen flex items-center justify-center`を使う目的は何ですか？

6.10 演習問題

演習1: ユーザープロフィールカードを作成

要件

- カバー画像（16:9の比率）
- プロフィール画像（カバー画像に重ねる）
- ニックネーム、自己紹介
- フォロー/フォロワー数
- 編集ボタン（自分のプロフィールの場合のみ）
- レスポンシブ対応

ヒント

```
// components/user/ProfileCard.tsx
import { Card, CardContent } from '@components/ui/card'
import { Avatar, AvatarImage, AvatarFallback } from '@components/ui/avatar'
import { Button } from '@components/ui/button'
import Image from 'next/image'

interface ProfileCardProps {
  user: {
    id: string
    nickname: string
    bio: string | null
    avatarUrl: string | null
    headerUrl: string | null
    _count: {
      followers: number
      following: number
    }
  }
  isOwner: boolean
}

export function ProfileCard({ user, isOwner }: ProfileCardProps) {
  return (
    <Card>
      {/* カバー画像 */}
      <div className="relative w-full aspect-[16/9] bg-gray-200">
        {user.headerUrl && (
          <Image
            src={user.headerUrl}
            alt=""
            fill
            className="object-cover"
          />
        )}
      </div>

      <CardContent className="relative pt-0">
        {/* プロフィール画像（カバー画像に重ねる） */}
        <Avatar className="absolute -top-16 left-4 h-32 w-32 border-4 border-white">
          {/* ... */}
        </Avatar>

        {/* 編集ボタン */}
        {isOwner && (
          <div className="flex justify-end pt-4">
            <Button variant="outline">プロフィールを編集</Button>
          </div>
        )}

        {/* ユーザー情報 */}
        {/* ... */}
      </CardContent>
    </Card>
  )
}
```

```
        </CardContent>
      </Card>
    )
  }
```

演習2: 検索バーを作成

要件

- 検索入力欄
- 検索ボタン
- クリアボタン（入力がある場合のみ表示）
- フォーカス時に枠線を強調
- モバイルで全幅、デスクトップで最大幅を制限

ヒント

```
// components/search/SearchBar.tsx
'use client'

import { useState } from 'react'
import { Input } from '@components/ui/input'
import { Button } from '@components/ui/button'
import { useRouter } from 'next/navigation'

export function SearchBar() {
  const [query, setQuery] = useState('')
  const router = useRouter()

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault()
    if (query.trim()) {
      router.push(`/search?q=${encodeURIComponent(query)}`)
    }
  }

  return (
    <form onSubmit={handleSubmit} className="w-full max-w-2xl">
      <div className="relative flex gap-2">
        <Input
          type="search"
          placeholder="投稿、ユーザーを検索"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          className="flex-1"
        />
        {query && (
          <Button
            type="button"
            variant="ghost"
            size="sm"
            onClick={() => setQuery('')}
            className="absolute right-20 top-1/2 -translate-y-1/2"
          >
            ✕
          </Button>
        )}
        <Button type="submit">検索</Button>
      </div>
    </form>
  )
}
```

演習3: 通知リストを作成

要件

- 通知タイプごとにアイコンを表示（いいね、コメント、フォロー）
- 未読通知は背景色を変える
- 相対時間を表示（1時間前、1日前など）
- ホバー時に背景色を変える
- クリックで詳細ページに遷移

ヒント

```
// components/notification/NotificationList.tsx
import { Card } from '@components/ui/card'
import { Avatar, AvatarImage, AvatarFallback } from '@components/ui/avatar'
import Link from 'next/link'

interface Notification {
  id: string
  type: 'like' | 'comment' | 'follow'
  isRead: boolean
  createdAt: Date
  actor: {
    id: string
    nickname: string
    avatarUrl: string | null
  }
  post?: {
    id: string
    content: string
  }
}

const NOTIFICATION_CONFIG = {
  like: { icon: '❤️', text: 'があなたの投稿にいいねしました' },
  comment: { icon: '💬', text: 'があなたの投稿にコメントしました' },
  follow: { icon: '👤', text: 'があなたをフォローしました' },
}

export function NotificationList({ notifications }: { notifications: Notification[] }) {
  return (
    <div className="space-y-2">
      {notifications.map(notification => {
        const config = NOTIFICATION_CONFIG[notification.type]

        return (
          <Link
            key={notification.id}
            href={notification.post ? `/posts/${notification.post.id}` : `/users/${notification.actor.id}`}
          >
            <Card className={`p-4 hover:bg-gray-50 transition-colors ${!notification.isRead ? 'opacity-50' : ''}`>
              <div className="flex gap-3">
                <div className="text-2xl">{config.icon}</div>
                <Avatar>
                  <AvatarImage src={notification.actor.avatarUrl || undefined} />
                  <AvatarFallback>{notification.actor.nickname[0]}</AvatarFallback>
                </Avatar>
                <div className="flex-1">
                  <p className="text-sm">
                    <span className="font-semibold">{notification.actor.nickname}</span>
                    {config.text}
                  </p>
                  {notification.post && (

```

```

        <p className="text-sm text-gray-500 mt-1 line-clamp-1">
            {notification.post.content}
        </p>
    )}
    <p className="text-xs text-gray-400 mt-1">
        {formatRelativeTime(notification.createdAt)}
    </p>
</div>
</div>
</Card>
</Link>
)
}}
</div>
)
}

function formatRelativeTime(date: Date) {
    const now = new Date()
    const diff = now.getTime() - date.getTime()
    const minutes = Math.floor(diff / 60000)

    if (minutes < 1) return 'たった今'
    if (minutes < 60) return `${minutes}分前`

    const hours = Math.floor(minutes / 60)
    if (hours < 24) return `${hours}時間前`

    const days = Math.floor(hours / 24)
    if (days < 7) return `${days}日前`

    return date.toLocaleDateString('ja-JP')
}

```

6.11 CVA (class-variance-authority) 詳細

このセクションで学ぶこと

- CVA (class-variance-authority) の基本概念と役割
- `cva()` 関数の引数構造とバリエーション定義方法
- `defaultVariants` と `compoundVariants` の使い分け
- `VariantProps` 型ヘルパーによる型安全なprops定義
- BON-LOGの実際のコンポーネントで使われているCVAパターン

CVAとは何か

CVA (class-variance-authority) は、Tailwind CSSベースのコンポーネントにおいて、**バリエーション（見た目のバリエーション）を型安全に管理する**ためのライブラリです。shadcn/uiの全コンポーネントがCVAを基盤としており、BON-LOGのUI設計においても中核的な役割を果たしています。

CVAが解決する問題

【CVAなしの場合 - 条件分岐が複雑になる】

```
function Button({ variant, size }: ButtonProps) {
  // 条件分岐が増えるほどコードが複雑に ...
  let classes = 'inline-flex items-center justify-center'

  if (variant === 'default') {
    classes += ' bg-primary text-primary-foreground'
  } else if (variant === 'destructive') {
    classes += ' bg-destructive text-white'
  } else if (variant === 'outline') {
    classes += ' border border-border bg-background'
  }
  // ... size分岐も追加するとさらに複雑に

  return <button className={classes}> ... </button>
}
```

↓↓↓

【CVAを使った場合 - 宣言的で見通しが良い】

```
const buttonVariants = cva('inline-flex items-center ...', {
  variants: {
    variant: {
      default: 'bg-primary text-primary-foreground',
      destructive: 'bg-destructive text-white',
      outline: 'border border-border bg-background',
    },
    size: { ... },
  },
  defaultVariants: { variant: 'default', size: 'default' },
})
```

cva() 関数の構造

cva() 関数は2つの引数を受け取ります。

```
import { cva } from 'class-variance-authority'

const myVariants = cva(
  // 第1引数: ベーススタイル (全バリエントに共通で適用される)
  'inline-flex items-center justify-center rounded text-sm font-medium',

  // 第2引数: オプションオブジェクト (variants, defaultVariants, compoundVariants)
  {
    variants: { ... },
    defaultVariants: { ... },
    compoundVariants: [ ... ],
  }
)
```

第1引数: ベーススタイル

全てのバリエントに共通して適用されるクラス名の文字列です。コンポーネントがどのバリエントで呼ばれても必ず適用されます。

```
// BON-LOGのButtonコンポーネントのベーススタイル (実際のコード)
const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded text-sm font-medium",
  { ... }
)
```

このベーススタイルには以下が含まれています。

クラス	役割
<code>inline-flex items-center justify-center</code>	フレックスレイアウトで中央揃え
<code>gap-2</code>	子要素間にスペース
<code>rounded text-sm font-medium</code>	基本的な見た目
<code>transition-all duration-200</code>	アニメーション
<code>disabled:pointer-events-none disabled:opacity-50</code>	無効状態のスタイル
<code>active:scale-[0.98]</code>	クリック時のフィードバック
<code>focus-visible:ring-[3px]</code>	キーボードフォーカス表示

第2引数: variantsオプション

```
{
  variants: {
    // variant: 見た目のスタイルを切り替える
    variant: {
      default: 'bg-primary text-primary-foreground hover:bg-primary/85 shadow-sm',
      destructive: 'bg-destructive text-white hover:bg-destructive/90',
      outline: 'border border-border/80 bg-background shadow-xs hover:bg-muted/50',
      secondary: 'bg-secondary text-secondary-foreground hover:bg-secondary/70',
      ghost: 'hover:bg-muted/60 hover:text-foreground',
      link: 'text-primary underline-offset-4 hover:underline',
    },

    // size: サイズを切り替える
    size: {
      default: 'h-9 px-4 py-2',
      sm: 'h-8 rounded gap-1.5 px-3',
      lg: 'h-10 rounded px-6',
      icon: 'size-9',
    },
  },
}
```

variantsの使い方

```
// 関数として呼び出すと、該当するクラス名の文字列が返る
buttonVariants({ variant: 'default', size: 'default' })
// ⇒ 'inline-flex items-center ... bg-primary text-primary-foreground ... h-9 px-4 py-2'

buttonVariants({ variant: 'destructive', size: 'lg' })
// ⇒ 'inline-flex items-center ... bg-destructive text-white ... h-10 rounded px-6'

// コンポーネントでの使用
<button className={buttonVariants({ variant: 'outline', size: 'sm' })}>
  保存
</button>
```

defaultVariants

propsが指定されなかった場合に適用されるデフォルト値を定義します。

```
const buttonVariants = cva('... ', {
  variants: {
    variant: {
      default: '...',
      destructive: '...',
      outline: '...',
    },
    size: {
      default: '...',
      sm: '...',
      lg: '...',
    },
  },
  // propsが未指定の場合のデフォルト値
  defaultVariants: {
    variant: 'default',
    size: 'default',
  },
})

// これらは同じ結果になる
buttonVariants() // デフォルト適用
buttonVariants({ variant: 'default', size: 'default' }) // 明示的に指定
```

compoundVariants（複合バリエント）

複数のバリエントの組み合わせに応じて追加のスタイルを適用したい場合に使います。

```

const alertVariants = cva('rounded-lg border p-4 text-sm', {
  variants: {
    variant: {
      info: 'bg-blue-50 border-blue-200',
      success: 'bg-green-50 border-green-200',
      warning: 'bg-yellow-50 border-yellow-200',
      error: 'bg-red-50 border-red-200',
    },
    size: {
      sm: 'p-2 text-xs',
      default: 'p-4 text-sm',
      lg: 'p-6 text-base',
    },
    elevated: {
      true: 'shadow-lg',
      false: '',
    },
  },
},

// compoundVariants: 特定の組み合わせに追加スタイルを適用
compoundVariants: [
  {
    // variant=error かつ elevated=true の場合
    variant: 'error',
    elevated: true,
    // 追加のクラスを適用
    className: 'ring-2 ring-red-300 shadow-red-100',
  },
  {
    // variant=success かつ size=lg の場合
    variant: 'success',
    size: 'lg',
    className: 'border-2 font-semibold',
  },
],

defaultVariants: {
  variant: 'info',
  size: 'default',
  elevated: false,
},
})

```

compoundVariantsの使用例

variant="error" + elevated=true の場合:

レイヤー	適用されるスタイル
ベーススタイル	<code>rounded-lg border p-4 text-sm</code>
+ error	<code>bg-red-50 border-red-200</code>
+ elevated	<code>shadow-lg</code>
+ compound	<code>ring-2 ring-red-300 shadow-red-100</code> (組み合わせにのみ追加されるスタイル)

variant="info" + elevated=true の場合:

レイヤー	適用されるスタイル
ベーススタイル	<code>rounded-lg border p-4 text-sm</code>
+ info	<code>bg-blue-50 border-blue-200</code>
+ elevated	<code>shadow-lg</code>
+ compound	(適用されない - 条件が一致しないため)

VariantProps型ヘルパー

CVAで定義したバリエントの型を自動的にTypeScriptの型として抽出できます。これにより、コンポーネントのpropsに型安全性が保証されます。

```

import { cva, type VariantProps } from 'class-variance-authority'

const buttonVariants = cva('... ', {
  variants: {
    variant: {
      default: '...',
      destructive: '...',
      outline: '...',
    },
    size: {
      default: '...',
      sm: '...',
      lg: '...',
    },
  },
  defaultVariants: {
    variant: 'default',
    size: 'default',
  },
})

// VariantPropsで型を抽出
type ButtonVariantProps = VariantProps<typeof buttonVariants>
// ⇒ {
//   variant?: 'default' | 'destructive' | 'outline' | null | undefined
//   size?: 'default' | 'sm' | 'lg' | null | undefined
// }

// コンポーネントのprops型に使用
function Button({
  className,
  variant,
  size,
  ...props
}: React.ComponentProps<'button'> & VariantProps<typeof buttonVariants>) {
  return (
    <button
      className={cn(buttonVariants({ variant, size, className }))}
      { ... props}
    />
  )
}

```

VariantPropsの型安全性

```
// 正しい使い方 - 型チェックに通る
<Button variant="outline" size="lg">保存</Button>

// コンパイルエラー - 存在しないバリエーションを指定
<Button variant="large" size="lg">保存</Button>
//                ^^^^^^^^ '"large"' は割り当てられません
```

BON-LOGでのCVA実例: Badgeコンポーネント

```
// components/ui/badge.tsx (BON-LOGの実際のコード)
const badgeVariants = cva(
  // ベーススタイル
  "inline-flex items-center justify-center rounded-full border px-2 py-0.5 text-xs font-me
  {
    variants: {
      variant: {
        // デフォルト: プライマリカラーの塗りつぶし
        default:
          "border-transparent bg-primary text-primary-foreground [a&]:hover:bg-primary/90"
        // セカンダリ: グレー系の塗りつぶし
        secondary:
          "border-transparent bg-secondary text-secondary-foreground [a&]:hover:bg-seconda
        // 削除・エラー用: 赤系の塗りつぶし
        destructive:
          "border-transparent bg-destructive text-white [a&]:hover:bg-destructive/90",
        // アウトライン: 枠線のみ
        outline:
          "text-foreground [a&]:hover:bg-accent [a&]:hover:text-accent-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)
```

独自コンポーネントでのCVA活用

BON-LOG向けのカスタムコンポーネントを作る場合の例です。

```

// components/post/PostStatusBadge.tsx
import { cva, type VariantProps } from 'class-variance-authority'
import { cn } from '@lib/utils'

// 投稿ステータスバッジのバリエーション定義
const postStatusVariants = cva(
  // ベーススタイル
  'inline-flex items-center gap-1 rounded-full px-2.5 py-0.5 text-xs font-medium',
  {
    variants: {
      status: {
        draft: 'bg-muted text-muted-foreground',
        published: 'bg-primary/10 text-primary border border-primary/20',
        scheduled: 'bg-amber-50 text-amber-700 border border-amber-200',
        archived: 'bg-gray-100 text-gray-500',
      },
    },
    defaultVariants: {
      status: 'draft',
    },
  },
)

// ステータスラベルのマッピング
const STATUS_LABELS = {
  draft: '下書き',
  published: '公開中',
  scheduled: '予約済み',
  archived: 'アーカイブ',
} as const

type PostStatusBadgeProps = VariantProps<typeof postStatusVariants> & {
  className?: string
}

export function PostStatusBadge({ status, className }: PostStatusBadgeProps) {
  return (
    <span className={cn(postStatusVariants({ status }), className)}>
      {STATUS_LABELS[status ?? 'draft']}
    </span>
  )
}

// 使用例
// <PostStatusBadge status="published" />
// <PostStatusBadge status="scheduled" className="text-base" />

```

画面表示 各ステータスバッジの見え方:

- **draft（下書き）**: グレーの背景にグレーのテキスト。控えめで目立たない表示
- **published（公開中）**: 松葉色の薄い背景に松葉色のテキスト + 薄い松葉色の枠線。公開状態を緑系で表現
- **scheduled（予約済み）**: 薄い黄色の背景に黄褐色のテキスト + 黄系の枠線。未来の予定を暖色で表現
- **archived（アーカイブ）**: 薄いグレー背景に薄めのテキスト。過去のコンテンツを控えめに表示

cva()にclassNameを渡す

`cva()` で生成された関数は、`className` 引数を受け取ることができます。これにより、呼び出し側から追加のクラスを注入できます。

```
const buttonVariants = cva('px-4 py-2 rounded', {
  variants: {
    variant: { primary: 'bg-blue-500', secondary: 'bg-gray-200' },
  },
  defaultVariants: { variant: 'primary' },
})

// className引数で追加クラスを渡す
buttonVariants({ variant: 'primary', className: 'mt-4 w-full' })
// ⇒ 'px-4 py-2 rounded bg-blue-500 mt-4 w-full'
```

理解度チェック

▶ Q1: `cva()` の第1引数と第2引数の `variants` の違いは何ですか？

▶ Q2: `compoundVariants` はどのような場面で使いますか？

▶ Q3: `VariantProps` を使う利点は何ですか？

6.12 tailwind-merge + clsx (cn()関数)

このセクションで学ぶこと

- BON-LOGの `lib/utils.ts` に定義されている `cn()` 関数の内部動作
- `clsx` と `tailwind-merge` がそれぞれ解決する問題
- 条件付きクラス名の実践的なパターン
- コンポーネント設計における `cn()` の重要性

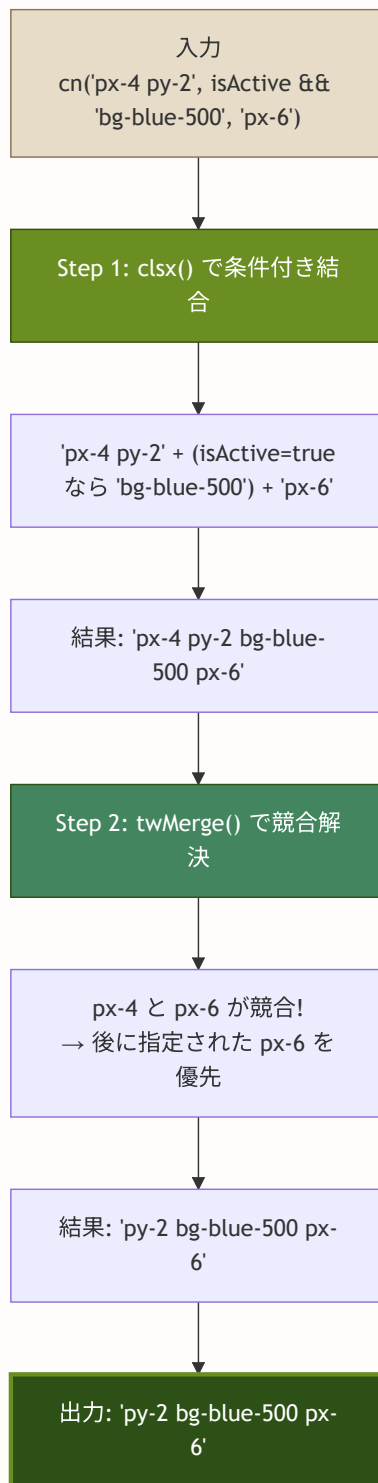
cn()関数の全体像

BON-LOGでは、`lib/utils.ts` にスタイリングの要となる `cn()` 関数が定義されています。

```
// lib/utils.ts (BON-LOGの実際のコード)
import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

この関数は非常に短いですが、2つのライブラリを組み合わせることで強力な機能を提供しています。



clsx: 条件付きクラス名の結合

`clsx` は、さまざまな形式の引数を受け取って、1つのクラス名文字列に結合するライブラリです。

```
import { clsx } from 'clsx'

// 1. 文字列の結合
clsx('foo', 'bar')
// ⇒ 'foo bar'

// 2. 条件付き (falsy値は無視される)
clsx('base', true && 'active', false && 'hidden', null, undefined)
// ⇒ 'base active'

// 3. オブジェクト形式 (値がtruthyのキーのみ含まれる)
clsx({ 'bg-red-500': hasError, 'bg-green-500': isSuccess, 'opacity-50': isDisabled })
// hasError=true, isSuccess=false, isDisabled=true の場合
// ⇒ 'bg-red-500 opacity-50'

// 4. 配列形式
clsx(['foo', 'bar'], ['baz'])
// ⇒ 'foo bar baz'

// 5. 混合形式 (文字列、オブジェクト、配列を組み合わせ可能)
clsx('base', { active: isActive }, ['extra', isLarge && 'large'])
// isActive=true, isLarge=false の場合
// ⇒ 'base active extra'
```

tailwind-merge: Tailwindクラスの競合解決

`tailwind-merge` は、Tailwind CSSのクラス名が競合した場合に、後に指定されたクラスを優先するライブラリです。

```
import { twMerge } from 'tailwind-merge'

// 同じプロパティの競合を解決
twMerge('px-4 px-6')
// ⇒ 'px-6' (後の値が優先)

twMerge('text-red-500 text-blue-500')
// ⇒ 'text-blue-500'

// ショートハンドと個別指定の競合も解決
twMerge('px-4 py-2 p-6')
// ⇒ 'p-6' (pはpx/pyを包含する)

// レスポンシブクラスは別プロパティとして扱う
twMerge('md:px-4 md:px-6')
// ⇒ 'md:px-6'

// 競合しないクラスはそのまま残る
twMerge('bg-red-500 text-white font-bold bg-blue-500')
// ⇒ 'text-white font-bold bg-blue-500'
```

なぜtailwind-mergeが必要か

【tailwind-mergeなしの問題】

```
// コンポーネント内部のベーススタイル
const baseClass = 'px-4 py-2 bg-blue-500'

// 親コンポーネントから上書きしたい
<Button className="px-8">大きい余白</Button>

// 結果: 'px-4 py-2 bg-blue-500 px-8'
//      ↑               ↑
//      px-4 と px-8 の両方が適用される！
//      CSSの詳細度が同じなので、どちらが適用されるか不確定
```

【tailwind-mergeありの場合】

```
cn('px-4 py-2 bg-blue-500', 'px-8')
// 結果: 'py-2 bg-blue-500 px-8'
//      px-4は削除され、px-8のみが適用される
```

条件付きクラス名のパターン集

パターン1: 真偽値による切り替え

```
// isActive が true なら 'bg-primary text-white' を追加
<div className={cn(
  'px-4 py-2 rounded',
  isActive && 'bg-primary text-white'
)}>
```

パターン2: 三項演算子による切り替え

```
// isActive の真偽で異なるスタイルを適用
<div className={cn(
  'px-4 py-2 rounded transition-colors',
  isActive
    ? 'bg-primary text-primary-foreground'
    : 'bg-muted text-muted-foreground'
)}>
```

パターン3: オブジェクト形式の条件指定

```
// 複数の条件を見やすく記述
<div className={cn(
  'px-4 py-2 rounded',
  {
    'bg-primary text-white': isActive,
    'opacity-50 cursor-not-allowed': isDisabled,
    'ring-2 ring-destructive': hasError,
    'shadow-lg': isElevated,
  }
)}>
```

パターン4: コンポーネントのclassNameオーバーライド

```
// 最も重要なパターン: 外部からのクラス上書きを許可
function Card({ className, children }: { className?: string; children: React.ReactNode })
  return (
    <div className={cn(
      // ベーススタイル (デフォルト)
      'rounded-lg border bg-card p-6 shadow-sm',
      // 外部から渡されたクラスで上書き可能
      className
    )}>
      {children}
    </div>
  )
}

// 使用側: paddingを上書き
<Card className="p-2">コンパクトなカード</Card>
// 結果: 'rounded-lg border bg-card shadow-sm p-2'
// p-6がp-2で上書きされる
```

パターン5: CVAと組み合わせた使用

```
// BON-LOGのButtonコンポーネントでの実際のパターン
function Button({
  className,
  variant,
  size,
  ... props
}: React.ComponentProps<'button'> & VariantProps<typeof buttonVariants>) {
  return (
    <button
      className={cn(buttonVariants({ variant, size, className })))}
      { ... props}
    />
  )
}

// 使用側: バリエント + 追加クラス
<Button variant="outline" className="w-full mt-4">
  保存
</Button>
```

ClassValue型

`cn()` が受け入れる引数の型 `ClassValue` は、以下の値を受け付けます。

```
type ClassValue =
  | string           // 'foo bar'
  | number           // 0 (falsyなので無視される)
  | boolean          // true/false
  | null             // 無視される
  | undefined        // 無視される
  | ClassValue[]     // ['foo', 'bar']
  | Record<string, boolean> // { foo: true, bar: false }
```

この柔軟な型定義により、条件分岐の結果をそのまま引数に渡すことが可能です。

よくある使い方の比較

```
// すべて同じ結果になる (isActive=true, isDisabled=false の場合)

// 方法1: 論理AND演算子
cn('base', isActive && 'active', isDisabled && 'disabled')
// ⇒ 'base active'

// 方法2: 三項演算子
cn('base', isActive ? 'active' : '', isDisabled ? 'disabled' : '')
// ⇒ 'base active'

// 方法3: オブジェクト形式
cn('base', { active: isActive, disabled: isDisabled })
// ⇒ 'base active'

// 推奨: 条件が少ない場合は方法1、多い場合は方法3
```

理解度チェック

▶ Q1: `cn('px-4', 'px-8')` の結果は何ですか？

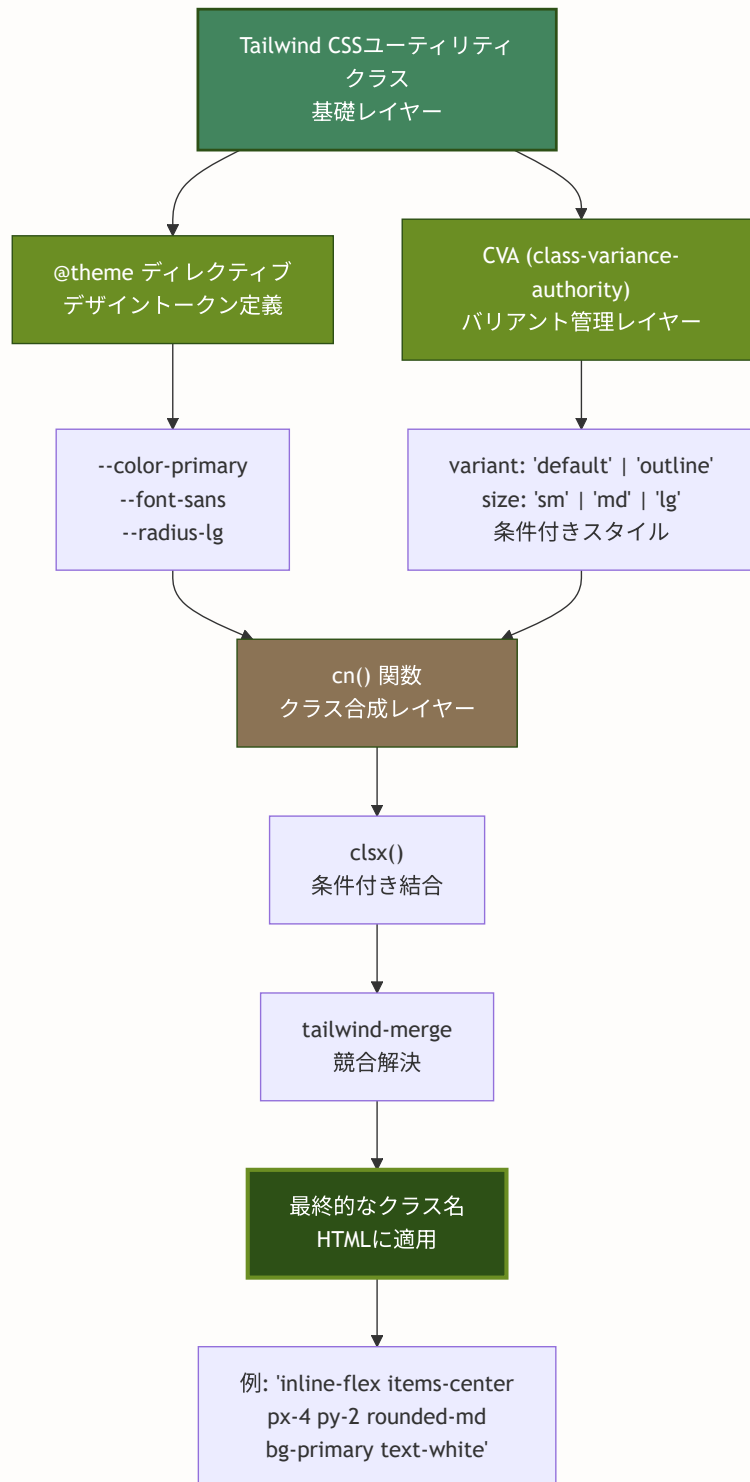
▶ Q2: `cn()`関数がコンポーネント設計で重要な理由は何ですか？

▶ Q3: `clsx({ 'bg-red': true, 'text-white': false, 'font-bold': true })` の結果は？

補足I: CSS Class Composition Flow (Tailwind → CVA → `cn()`)

BON-LOGで使われるスタイリングの流れを、全体像として図解します。

CSS クラス合成フローの全体像



レイヤー別の役割

レイヤー1: Tailwind CSS (基礎)

- `px-4`, `py-2`, `bg-blue-500` などのユーティリティクラスを提供
- すべてのスタイリングの土台

レイヤー2: @theme (デザイントークン)

- `--color-primary`, `--font-sans` など、プロジェクト共通の値を定義
- `bg-primary` のように、トークン名でクラスを使えるようにする

レイヤー3: CVA (バリエント管理)

- `variant="default"` などのpropsに応じてクラスを切り替え
- 複雑な条件分岐を型安全に管理

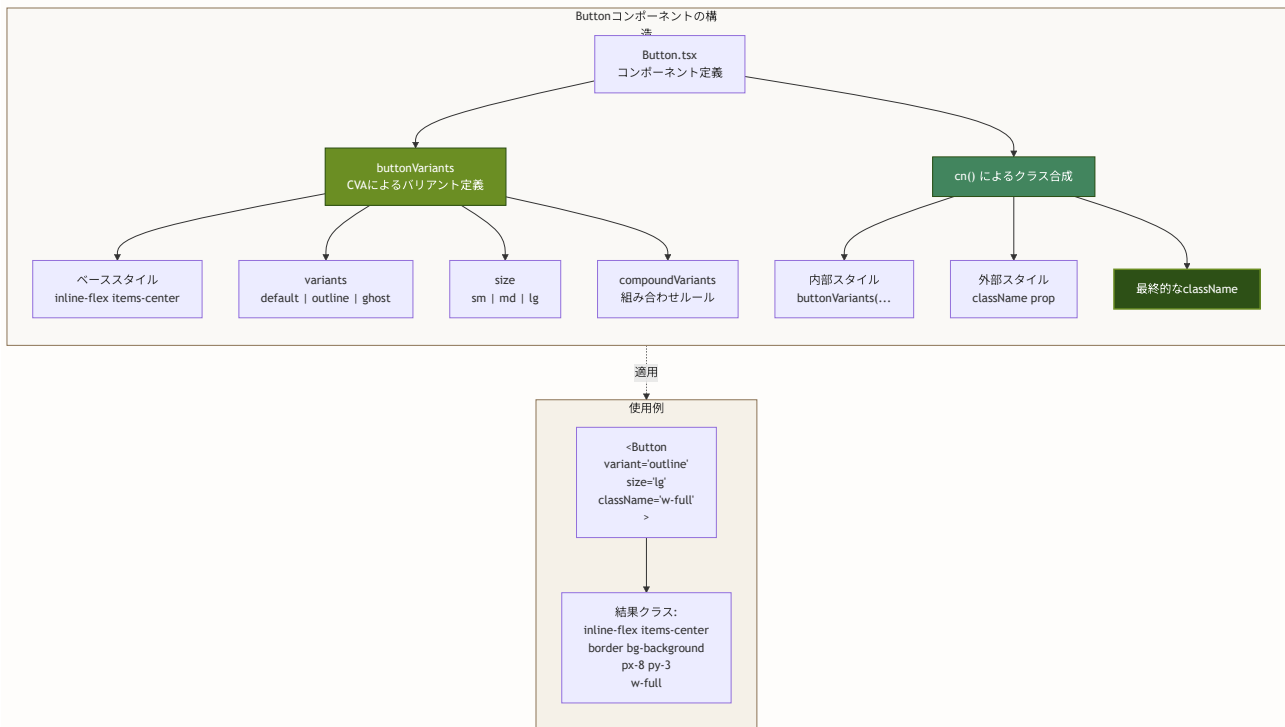
レイヤー4: cn() (クラス合成)

- 複数のクラス名を結合し、競合を解決
- コンポーネントの外部から上書き可能にする

補足J: Component Styling Architecture (コンポーネントスタイリング設計)

shadcn/uiのコンポーネントがどのようにスタイリングされているかを構造化して示します。

コンポーネントスタイリングのアーキテクチャ



スタイリング優先順位

1. **ベーススタイル**（最も低い優先度）：すべてのバリエーションで共通
2. **variants**: `variant` や `size` に応じたスタイル
3. **compoundVariants**: 複数バリエーションの組み合わせ
4. **className prop**（最も高い優先度）：外部からの上書き

```

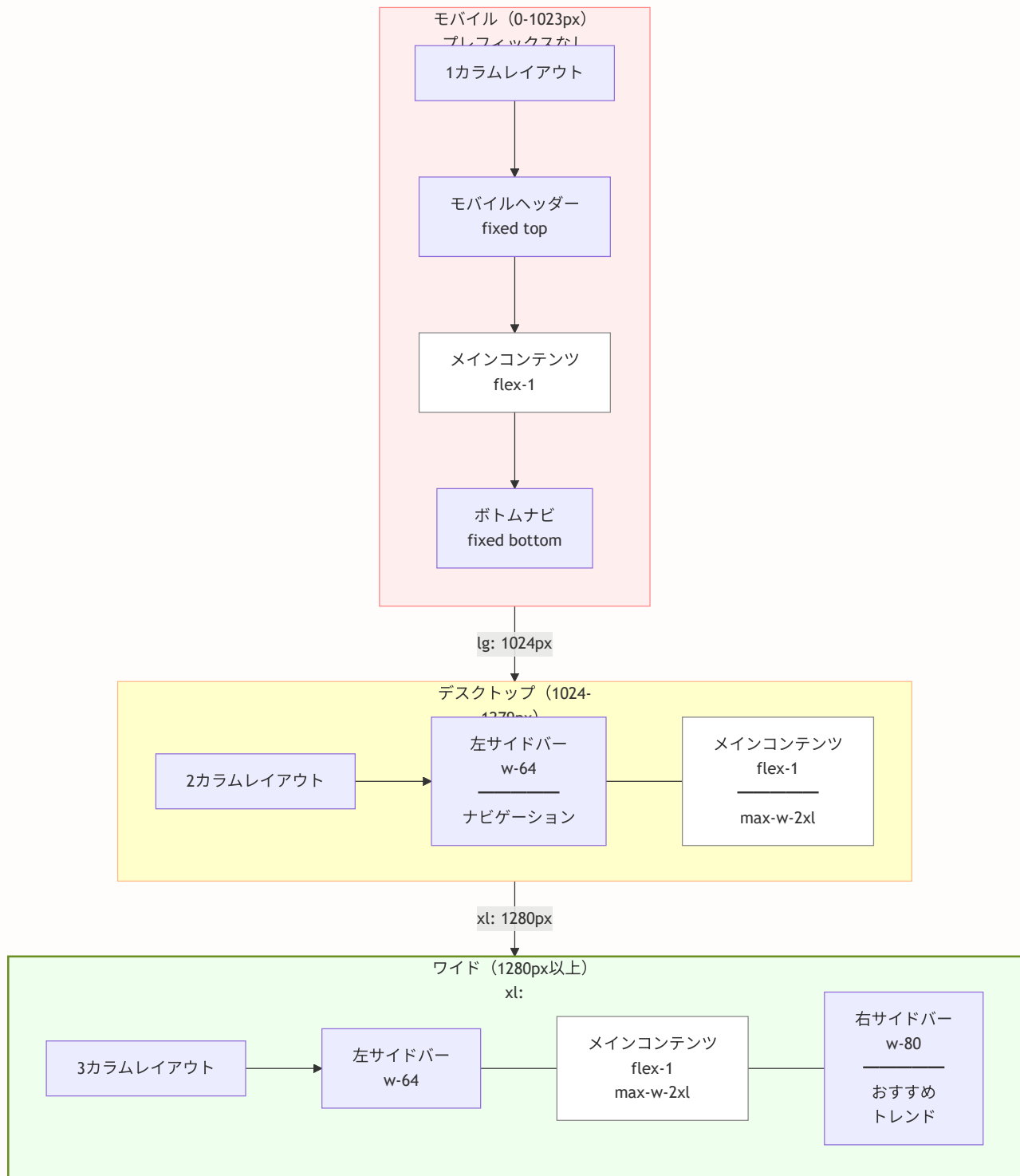
// 優先順位の実例
<Button
  variant="default"    // → bg-primary text-white
  size="lg"           // → px-8 py-3
  className="px-4"     // → px-8 を px-4 で上書き
/>
// 最終結果: inline-flex items-center bg-primary text-white py-3 px-4
  
```

補足K: Responsive Layout Breakdown（レスポンシブレイアウト）

ト詳細)

BON-LOGの3カラムレイアウトが画面サイズによってどう変化するかを段階的に示します。

レスポンシブレイアウトの段階的变化



各段階での表示/非表示クラス

```
// 左サイドバー: モバイルで非表示、lg以上で表示
<aside className="hidden lg:block w-64">

// 右サイドバー: モバイル・デスクトップで非表示、xl以上で表示
<aside className="hidden xl:block w-80">

// モバイルヘッダー: モバイルのみ、lg以上で非表示
<header className="block lg:hidden fixed top-0">

// ボトムナビ: モバイルのみ、lg以上で非表示
<nav className="block lg:hidden fixed bottom-0">
```

コンテンツ幅の制御

```
// メインコンテンツ: 画面サイズに応じて幅を調整
<main className="
  flex-1                // 残りスペースを埋める
  max-w-full            // モバイル: 画面幅いっぱい
  lg:max-w-2xl          // デスクトップ: 最大42rem
  border-x border-gray-200 // 左右にボーダー
">
```

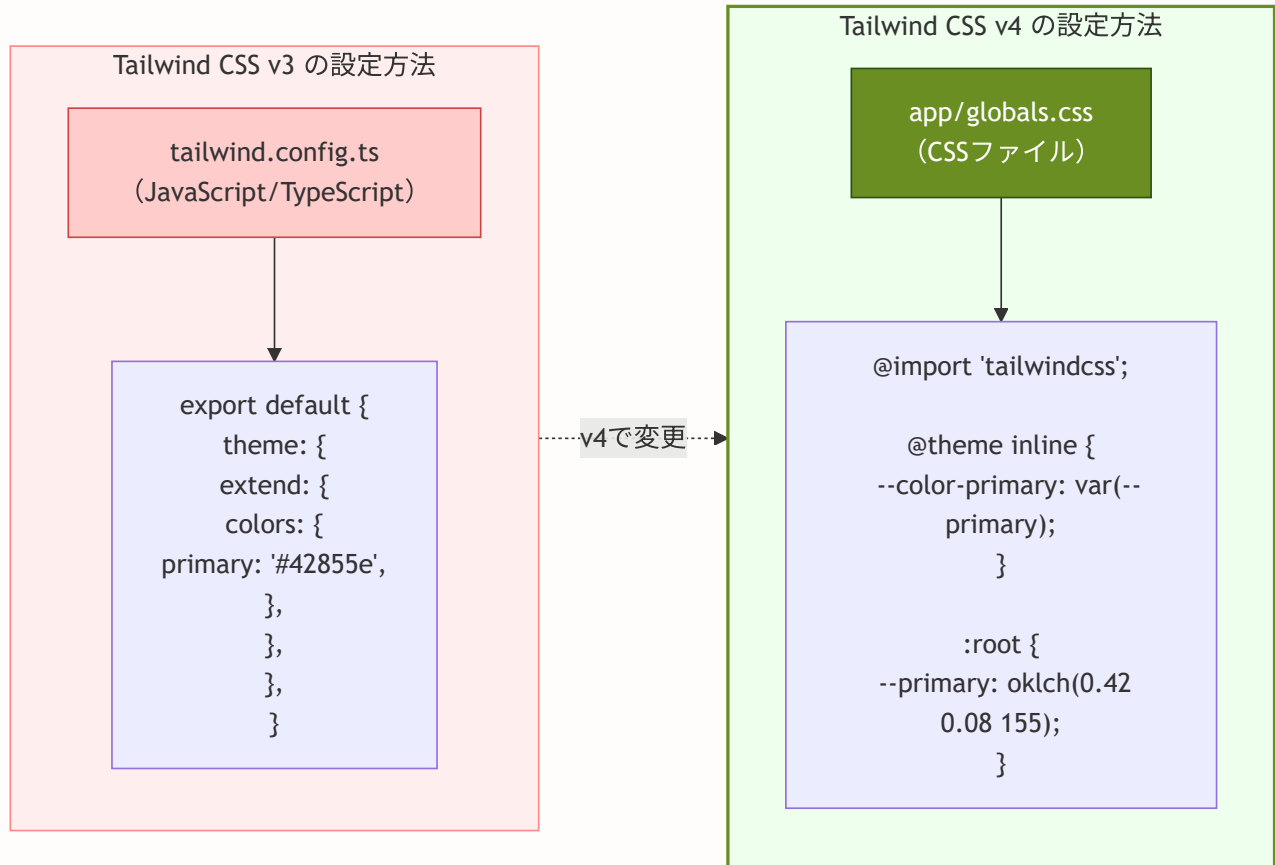
6.13 Tailwind CSS v4固有機能

このセクションで学ぶこと

- Tailwind CSS v4のCSS-firstアーキテクチャ
- `@theme` ディレクティブによるデザイントークンの定義
- CSS変数（カスタムプロパティ）を活用したテーマシステム
- `@custom-variant` によるカスタムバリエーションの定義
- `oklch()` カラースペースの利点

Tailwind CSS v4の大きな変更点

BON-LOGはTailwind CSS v4を使用しています。v3からの最大の変更点は、**設定ファイル** (`tailwind.config.ts`) から**CSS-first**への移行です。



@import "tailwindcss"

v4では、従来の3つのディレクティブ (`@tailwind base; @tailwind components; @tailwind utilities;`) が1つのインポート文に統合されました。

```
/* Tailwind CSS v3 */
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Tailwind CSS v4 */
@import "tailwindcss";
```

@themeディレクティブ

`@theme` は、Tailwind CSSのユーティリティクラスとして使えるデザイントークン（色、フォント、角丸など）を定義するディレクティブです。BON-LOGの `globals.css` では `@theme inline` として使用しています。

```
/* app/globals.css (BON-LOGの実際のコード) */
@theme inline {
  /* カラートークン: bg-background、text-foreground 等のクラスで使用可能 */
  --color-background: var(--background);
  --color-foreground: var(--foreground);

  /* フォントトークン: font-sans、font-serif 等のクラスで使用可能 */
  --font-sans: var(--font-oto-sans-jp);
  --font-mono: var(--font-geist-mono);
  --font-serif: var(--font-shippori-mincho);

  /* 和風カラートークン: bg-bonsai-green、text-sumi 等のクラスで使用可能 */
  --color-bonsai-green: var(--bonsai-green);
  --color-bonsai-brown: var(--bonsai-brown);
  --color-bonsai-beige: var(--bonsai-beige);
  --color-bonsai-cream: var(--bonsai-cream);
  --color-washi: var(--washi);
  --color-sumi: var(--sumi);
  --color-aka: var(--aka);
  --color-ai: var(--ai);
  --color-matcha: var(--matcha);
  --color-kincha: var(--kincha);

  /* 角丸トークン: rounded-sm、rounded-lg 等のクラスで使用可能 */
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
}
```

@themeの命名規則

`@theme` 内で定義する変数名は、Tailwindのユーティリティクラスに対応する特定の命名規則に従います。

@theme変数名	使えるTailwindクラス
<code>--color-primary</code>	<code>bg-primary</code> , <code>text-primary</code>
<code>--color-bonsai-green</code>	<code>bg-bonsai-green</code>
<code>--font-sans</code>	<code>font-sans</code>
<code>--font-serif</code>	<code>font-serif</code>
<code>--radius-lg</code>	<code>rounded-lg</code>
<code>--spacing-18</code>	<code>p-18</code> , <code>m-18</code> , <code>gap-18</code>
<code>--breakpoint-xl</code>	<code>xl:</code>

inlineキーワード

```

/* @theme inline: CSSの出力に @theme の内容を含めない（軽量化） */
/* CSS変数を参照するだけの場合に使用 */
@theme inline {
  --color-primary: var(--primary);
}

/* @theme (inlineなし) : テーマ値を直接定義する場合 */
@theme {
  --color-primary: oklch(0.42 0.08 155);
}

```

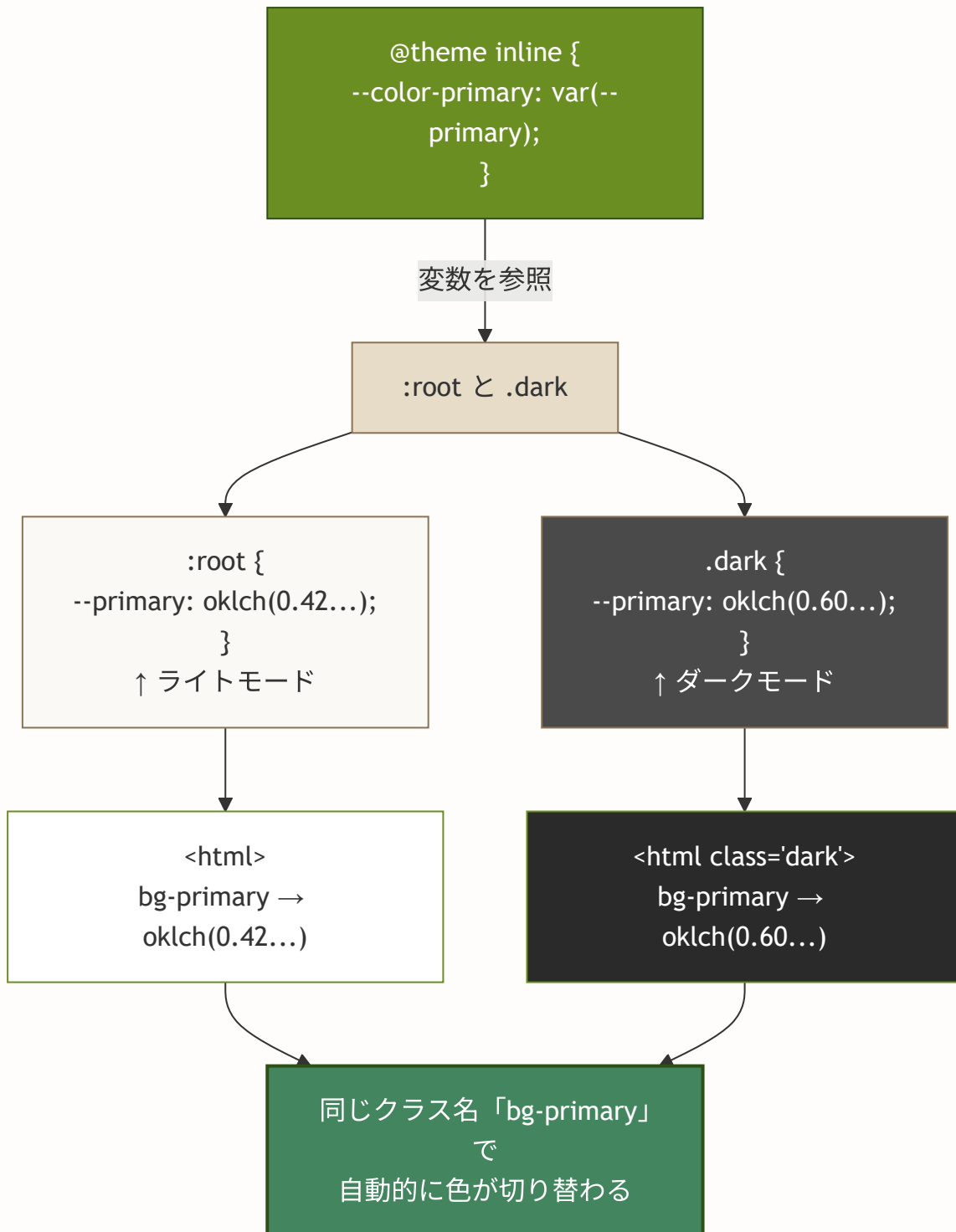
CSS変数によるテーマシステム

BON-LOGでは、`:root`と`.dark`で定義されたCSS変数を切り替えることで、ライトモードとダークモードを実現しています。

```
/* app/globals.css (BON-LOGの実際のコード) */
:root {
  /* ライトモード: 和風の明るい色調 */
  --background: oklch(0.97 0.008 85); /* 生成り - kinari */
  --foreground: oklch(0.22 0.015 50); /* 墨色 - sumi */
  --card: oklch(0.99 0.004 90); /* 和紙 - washi */
  --primary: oklch(0.42 0.08 155); /* 松葉色 - matsuba */
  --secondary: oklch(0.88 0.025 75); /* 亜麻色 - ama */
  --accent: oklch(0.55 0.12 45); /* 錆朱 - sabiaka */
  --destructive: oklch(0.55 0.20 25); /* 朱色 - shu */
}

.dark {
  /* ダークモード: 墨・漆のような暗い色調 */
  --background: oklch(0.15 0.015 50); /* 漆黒 - shikkoku */
  --foreground: oklch(0.92 0.008 85); /* 生成り */
  --card: oklch(0.20 0.015 50); /* 墨色 */
  --primary: oklch(0.60 0.10 155); /* 若松 */
  --secondary: oklch(0.30 0.02 50);
  --accent: oklch(0.60 0.15 45); /* 緋色 */
  --destructive: oklch(0.60 0.20 25);
}
```

CSS変数の仕組み図



@custom-variant

v4では `@custom-variant` ディレクティブでカスタムバリエントを定義できます。BON-LOGではダークモードのバリエントをカスタム定義しています。

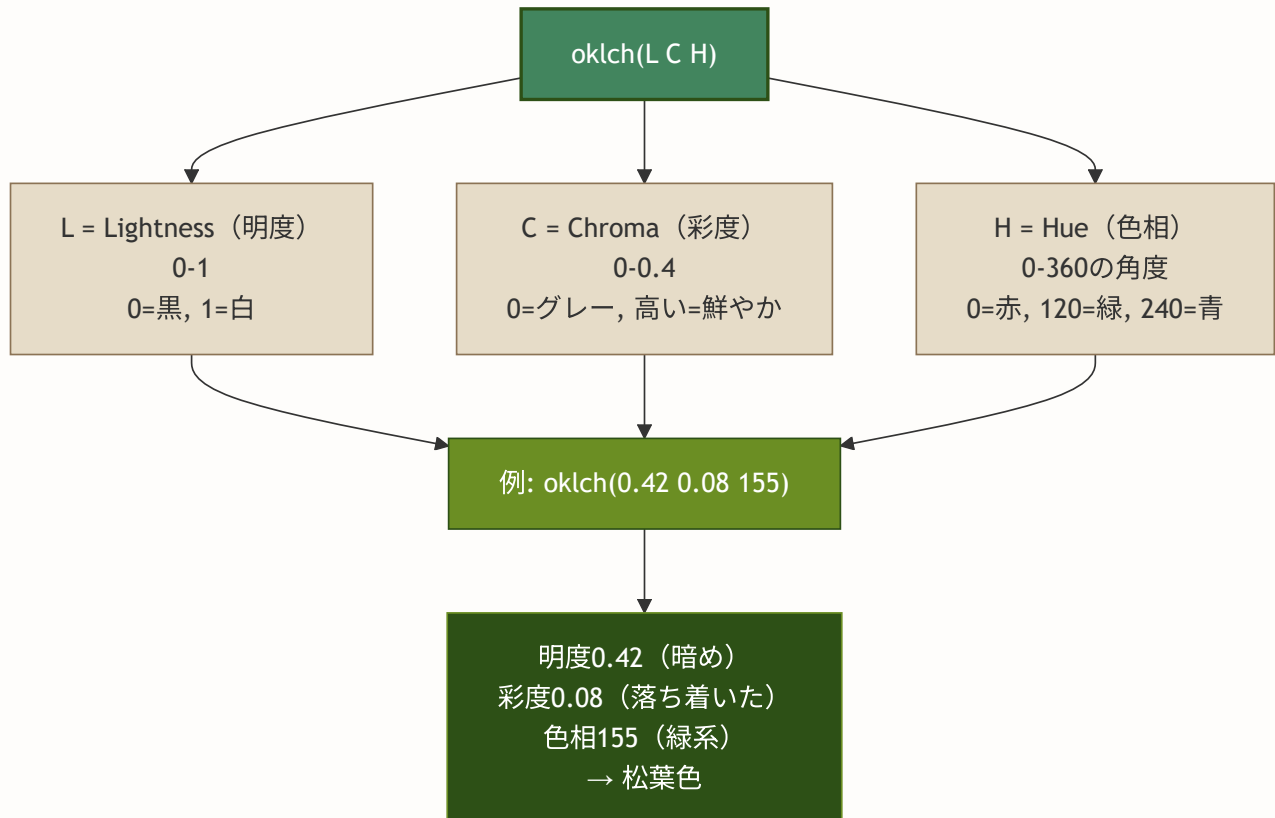
```
/* app/globals.css (BON-LOGの実際のコード) */
@custom-variant dark (&:is(.dark *));
```

これにより、`dark:` プレフィックスが `.dark` クラスの子孫要素に適用されるようになります。

```
<!-- この設定により以下が動作する -->
<html class="dark">
  <body>
    <div class="bg-white dark:bg-gray-900">
      <!-- .darkクラスの子孫なので dark:bg-gray-900 が適用される -->
    </div>
  </body>
</html>
```

oklch() カラースペース

BON-LOGのカラーパレットでは、従来のHEX/RGB/HSLではなく**oklch()**カラースペースを使用しています。



oklch()の利点

```
/* 明度を変えるだけで統一感のあるカラーバリエーションが作れる */
:root {
  --primary: oklch(0.42 0.08 155);          /* 松葉色（基本） */
}
.dark {
  --primary: oklch(0.60 0.10 155);          /* 若松色（明るくした版） */
  /*           ↑   ↑   同じ色相
     明度を上げて、彩度を少し上げるだけで
     ダークモード用の色が自然に作れる */
}
```

利点	説明
知覚的均一性	明度の値が同じなら、人間の目に同じ明るさに見える
直感的な操作	明度・彩度・色相を独立して調整できる
ダークモード対応	明度を変えるだけで自然なダークモード色が作れる
アクセシビリティ	コントラスト比の計算が正確にできる

@layerディレクティブ

BON-LOGでは `@layer base` を使って基本スタイルを定義しています。

```
/* app/globals.css (BON-LOGの実際のコード) */
@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  body {
    @apply bg-background text-foreground;
  }
}
```

`@layer` はCSSの詳細度を制御するための仕組みで、Tailwindの3つのレイヤー（`base` < `components` < `utilities`）の優先順位に従います。

理解度チェック

▶ Q1: Tailwind CSS v4で `tailwind.config.ts` の代わりに使うものは何ですか？

▶ Q2: `oklch()` カラースペースがダークモード対応に適している理由は何ですか？

▶ Q3: `@theme inline` と `@theme (inlineなし)` の違いは何ですか？

6.14 和風デザインパターン

このセクションで学ぶこと

- BON-LOGで使用している和風カラーパレット（日本の伝統色）の意味と使い分け
- 和紙テクスチャ、麻の葉、青海波などの伝統的装飾パターンの実装
- 和風タイポグラフィ（明朝体、縦書き）の活用方法
- `globals.css` で定義されている和風CSSクラスの使い方

和風カラーパレット

BON-LOGでは日本の伝統色を基調としたカラーパレットを採用しています。各色には日本語の色名が付けられ、その意味と用途が明確に定義されています。

色名	英語名	oklch値	用途	印象
■ 松葉色	matsuba (Primary)	oklch(0.42 0.08 155)	メインボタン、アクティブ状態、リンク	落ち着き、信頼、自然
■ 栗皮茶	kurikawacha (Brown)	oklch(0.45 0.08 55)	サブカラー、アクセント要素	温もり、大地、和の風格
□ 亜麻色	ama (Secondary)	oklch(0.88 0.025 75)	セカンダリボタン、タグ背景	穏やか、優しさ
□ 生成り	kinari (Background)	oklch(0.97 0.008 85)	ページ全体の背景色	自然な白、温かみ
□ 和紙	washi (Card)	oklch(0.99 0.004 90)	カード背景、ポップオーバー	清潔感、繊細さ
■ 墨	sumi (Foreground)	oklch(0.22 0.015 50)	メインテキスト	凛とした黒、文字の美しさ
■ 錆朱	sabiaka (Accent)	oklch(0.55 0.12 45)	アクセント、注目ポイント	渋み、味わい
■ 朱	shu/aka (Destructive)	oklch(0.55 0.20 25)	エラー、警告、削除	鳥居の赤、印鑑の朱
■ 藍	ai	oklch(0.35 0.10 250)	情報表示、リンク（代替）	深い青、日本の伝統染め
■ 抹茶	matcha	oklch(0.55 0.10 140)	成功状態、確認	茶道、和の緑
■ 金茶	kincha	oklch(0.50 0.12 70)	警告、注意喚起	金箔、高級感

カラーの使い分けガイド

```
// ページ背景: 生成り色 (自然な白)
<div className="bg-background">

// カード: 和紙色 (わずかに黄みがかった白)
<div className="bg-card">

// メインテキスト: 墨色 (純粹な黒ではなく温かみのある黒)
<p className="text-foreground">

// アクションボタン: 松葉色
<Button>投稿する</Button>    {/* bg-primary */}

// 危険なアクション: 朱色
<Button variant="destructive">削除</Button>

// タグ・バッジ: 亜麻色のベースに墨色のテキスト
<Badge variant="secondary">松柏類</Badge>

// リンクテキスト: 松葉色
<a className="text-primary hover:text-primary/80">詳細を見る</a>

// 和風カスタムカラーの直接使用
<div className="bg-matcha text-white">成功</div>    {/* → 抹茶色 (緑系) の背景に白文字 */}
<div className="bg-ai text-white">情報</div>        {/* → 藍色 (深い青) の背景に白文字 */}
<div className="bg-kincha text-white">警告</div>    {/* → 金茶色 (黄褐色) の背景に白文字 */}
```

和風テクスチャと装飾パターン

BON-LOGの `globals.css` には、日本の伝統的な模様をCSSで再現したクラスが定義されています。

和紙テクスチャ (`washi-texture`)

```
<!-- 和紙のような微細な質感を追加 -->
<div class="washi-texture bg-card p-6 rounded-lg">
  <p>和紙のような温かみのある質感の背景</p>
</div>
```

画面表示 カード背景色の上に、和紙の繊維のような極めて微細なノイズパターンが重なる。画面をよく見ると紙の表面のようなざらざらした質感が薄く感じられる。不透明度0.03と極めて控えめなので、テキストの可読性には影響しない。

和紙テクスチャはSVGフィルターの `feTurbulence` を使って、和紙特有の繊維感を極めて薄い不透明度（0.03）で表現しています。

麻の葉模様（asanoha-pattern）

```
<!-- 麻の葉模様の背景 -->
<div class="asanoha-pattern min-h-screen">
  <!-- BON-LOGのメインレイアウトで実際に使用 -->
</div>
```

画面表示 六角形を基調とした幾何学的な線画パターンが背景全体に繰り返し表示される。非常に薄い不透明度で描かれるため、メインコンテンツの可読性を妨げずに和の雰囲気を添える。

BON-LOGの `app/(main)/layout.tsx` では、メインレイアウトの背景に麻の葉模様を適用しています。

```
// app/(main)/layout.tsx (BON-LOGの実際のコード)
<div className="min-h-screen bg-background asanoha-pattern">
```

青海波模様（seigaiha-pattern）

```
<!-- 青海波（波のような連続模様） -->
<div class="seigaiha-pattern p-8">
  <p>海や水を象徴する伝統的な波模様</p>
</div>
```

画面表示 同心円の扇形が連なる波のような模様が背景に表示される。穏やかな海を象徴する吉祥文様で、ヘッダーやフッターの装飾に適している。

市松模様（ichimatsu-pattern）

```
<!-- 市松模様（格子状の模様） -->
<div class="ichimatsu-pattern p-8">
  <p>歌舞伎の佐野川市松に由来する格子模様</p>
</div>
```

画面表示 色の異なる正方形が交互に並ぶ格子模様（チェッカーボード）が背景に表示される。薄い不透明度で控えめに描かれ、モダンかつ伝統的な印象を与える。

和風コンポーネントクラス

カード和風装飾（card-washi）

上部にプライマリカラーのアクセントラインが入るカードスタイルです。

```
<!-- 和風カード：上部に松葉色のラインが入る -->
<div class="card-washi rounded p-4">
  <h3 class="font-medium">おすすめユーザー</h3>
  <p class="text-sm text-muted-foreground">... </p>
</div>
```

画面表示 カードの上端に松葉色（プライマリカラー）の細い横線（3px程度）が入り、和の掛け軸や料理の器の縁飾りのような上品なアクセントになる。カード本体は通常の白い背景。

BON-LOGの右サイドバーで実際に使用されています。

和風シャドウ（shadow-washi）

繊細な多層シャドウで和紙のような柔らかい影を実現します。

```

<!-- 和風の柔らかい影 -->
<div class="shadow-washi p-4 rounded-lg bg-card">
  通常の影
</div>

<!-- より大きな和風シャドウ -->
<div class="shadow-washi-lg p-4 rounded-lg bg-card">
  大きい影
</div>

```

画面表示

- **shadow-washi**: 3-4層の極めて薄い影が重なり、和紙が何枚か重なったような柔らかい影を表現。通常のTailwindの `shadow` より繊細で落ち着いた印象
- **shadow-washi-lg**: より大きな範囲に広がる多層影で、カードが少し高い位置に浮いているような奥行き感。モーダルやホバー時のカードに適している

和風タグ (tag-washi)

```

<!-- 和風のタグスタイル -->
<span class="tag-washi">松柏類</span>
<span class="tag-washi">雑木類</span>

```

画面表示 和紙のような質感を持つ小さなラベル。通常のBadgeコンポーネントと異なり、角がわずかに丸く、背景に微細なテクスチャ感がある。ジャンル分類の表示に使用。

印鑑風装飾 (hanko)

```

<!-- 印鑑風の丸い装飾 -->
<span class="hanko">盆</span>
<span class="hanko">栽</span>

```

画面表示 朱色（赤系）の丸い枠の中に「盆」「栽」の文字が表示される。実際の朱肉で押した印鑑のような見た目で、和の装飾要素として使える。

墨流し風グラデーション (suminagashi)

```
<!-- 墨流しのような微妙なグラデーション背景 -->
<div class="suminagashi p-8">
  <p>墨流し技法を模したグラデーション</p>
</div>
```

画面表示 微妙に色が変わるグラデーション背景が表示される。水面に墨を垂らして作る伝統技法「墨流し」を模した、波打つような緩やかな色調変化が見られる。ヒーローセクションや特別なページの背景装飾に適している。

和風タイポグラフィ

BON-LOGでは3つのフォントファミリーを和風デザインに活用しています。

```
/* @theme で定義されたフォント */
--font-sans: var(--font-noto-sans-jp);      /* Noto Sans JP: 本文用ゴシック体 */
--font-serif: var(--font-shippori-mincho); /* しっぽり明朝: 見出し・装飾用 */
--font-mono: var(--font-geist-mono);        /* Geist Mono: コード表示用 */
```

```
<!-- ゴシック体（デフォルト） -->
<p class="font-sans">盆栽愛好家のためのSNS</p>
<!-- → Noto Sans JP: 現代的で読みやすい。本文、ボタン、ラベルに使用 -->

<!-- 明朝体（和の雰囲気強調） -->
<h1 class="font-serif text-3xl">五葉松の育成記録</h1>
<!-- → しっぽり明朝: 筆で書いたような繊細さ。30pxの大きな見出しで和の雰囲気を演出 -->

<!-- 縦書きテキスト -->
<p class="vertical-text font-serif text-lg">盆栽の美</p>
<!-- → 右から左に縦書き表示。明朝体と組み合わせて掛け軸のような日本的な表現 -->
```

縦書きの実装

```
/* globals.css に定義済み */
.vertical-text {
  writing-mode: vertical-rl;    /* 右から左に縦書き */
  text-orientation: upright;  /* 文字を正立で表示 */
}
```

```
<!-- 縦書きの活用例：和風サイドデコレーション -->
<div class="flex items-center gap-4">
  <p class="vertical-text font-serif text-muted-foreground text-sm">
    五葉松
  </p>
  <div class="flex-1">
    
  </div>
</div>
```

和風区切り線

```
<!-- 竹のような区切り線（左右にグラデーションフェード） -->
<div class="divider-bamboo text-sm text-muted-foreground">
  <span>投稿一覧</span>
</div>
```

画面表示 「投稿一覧」テキストの左右に、中央から端に向かって徐々に薄くなるグラデーション線が表示される。竹の節のようなイメージで、セクション間の区切りとして和風の雰囲気を持つ水平線になる。

和風アニメーション

```

<!-- フェードインアップ: 下からふわっと表示 -->
<div class="animate-fade-in-up">
  投稿カード
</div>
<!-- → 要素が下から上に15pxほど移動しながら、透明から不透明にフェードイン。
      投稿カードの初回表示で「ふわっと」現れる柔らかい演出 -->

<!-- ゆるやかなバウンス: ローディング表示 -->
<div class="animate-gentle-bounce">
  読み込み中 ...
</div>
<!-- → 要素がゆっくり上下に弾むアニメーション。
      読み込み待ちの間、控えめに動くことでユーザーに処理中であることを伝える -->

```

`prefers-reduced-motion: reduce` メディアクエリにより、アニメーション酔いを起こすユーザーに対してはすべてのアニメーションが自動的に無効化されます。

和風セレクション

テキスト選択時も和風の色合いが適用されます。

```

/* globals.css に定義済み */
::selection {
  background: oklch(0.42 0.08 155 / 0.25); /* 松葉色の25%透過 */
  color: oklch(0.22 0.015 50); /* 墨色 */
}

```

理解度チェック

▶ Q1: BON-LOGのメインレイアウトの背景に使われている和柄パターンは何ですか？

▶ Q2: shadow-washiとTailwind標準のshadowクラスの違いは何ですか？

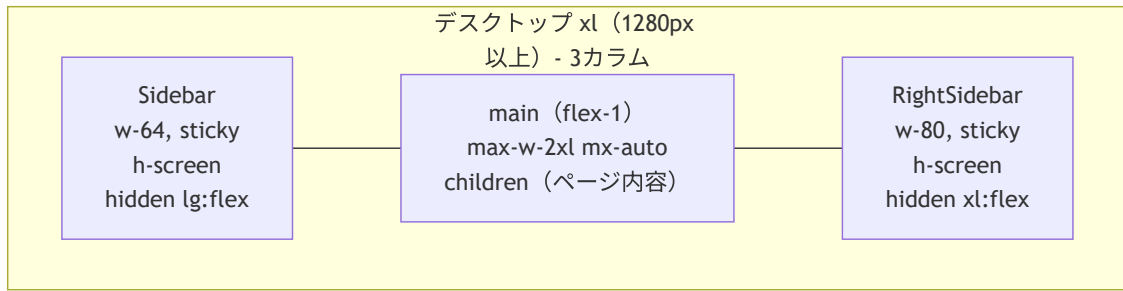
▶ Q3: prefers-reduced-motionの対応がなぜ重要ですか？

6.15 3カラムレイアウト実装

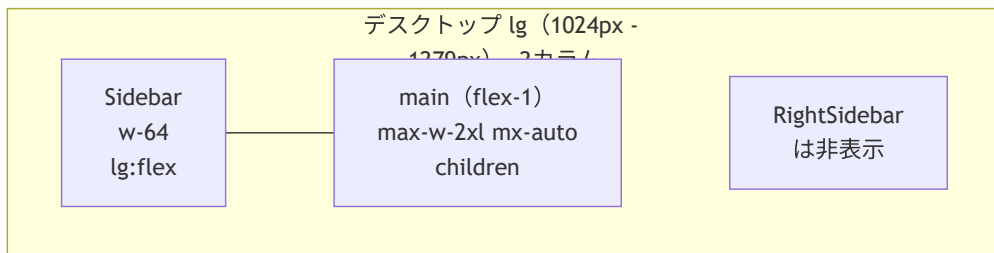
このセクションで学ぶこと

- BON-LOGの実際の3カラムレイアウト（`app/(main)/layout.tsx`）の構造
- Flexboxを使ったサイドバー + メインコンテンツの配置
- スティッキーサイドバーの実装方法と注意点
- レスポンシブ対応（デスクトップ3カラム / モバイル1カラム + ナビ）
- 各コンポーネント（Sidebar、RightSidebar、MobileNav、Header）の役割

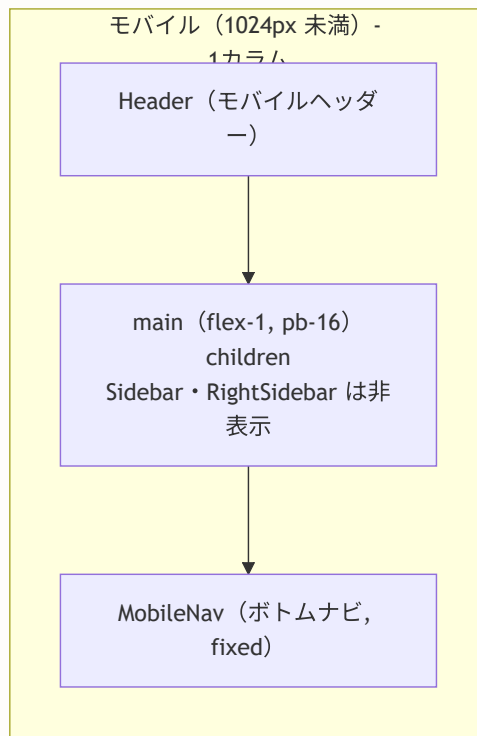
レイアウト全体像



lg: 1024px



< 1024px



メインレイアウトの実装 (BON-LOG実際のコード)

```
// app/(main)/layout.tsx (BON-LOGの実際のコード)
export default async function MainLayout({
  children,
}: {
  children: React.ReactNode
}) {
  const session = await auth()
  if (!session?.user) {
    redirect('/login')
  }

  const isPremium = await isPremiumUser(session.user.id)

  return (
    <div className="min-h-screen bg-background asanoha-pattern">
      {/* モバイルヘッダー (lg未満で表示) */}
      <Header userId={session.user.id} isPremium={isPremium} />

      <div className="flex">
        {/* 左サイドバー (lg以上で表示) */}
        <Sidebar userId={session.user.id} isPremium={isPremium} />

        {/* メインコンテンツ */}
        <main id="main-content" className="flex-1 min-h-screen pb-16 lg:pb-0" tabIndex={-1}>
          <div className="max-w-2xl mx-auto px-4 py-4 lg:py-6">
            {children}
          </div>
        </main>

        {/* 右サイドバー (xl以上で表示) */}
        <RightSidebar />
      </div>

      {/* モバイルボトムナビ (lg未満で表示) */}
      <MobileNav userId={session.user.id} isPremium={isPremium} />
      <KeyboardShortcutsProvider userId={session.user.id} />
      <Toaster />
    </div>
  )
}
```

画面表示 このレイアウトの各画面サイズでの見え方:

モバイル（1024px未満）：

- 画面上部: モバイルヘッダー（ロゴ + メニューボタン + 通知アイコン）
- 中央: メインコンテンツが画面幅いっぱいに表示。下部にボトムナビ分の余白（`pb-16` = 64px）
- 画面下部: ボトムナビ（ホーム / 検索 / 通知 / プロフィール）が固定表示
- 背景全体に生成り色 + 麻の葉模様（`asanoha-pattern`）

デスクトップ（1024px以上）：

- 左: サイドバー（幅256px）が出現。ナビゲーションリンク、プレミアムバッジ等
- 中央: メインコンテンツ（最大幅672px = `max-w-2xl` で中央配置）
- モバイルヘッダーとボトムナビは非表示

ワイドデスクトップ（1280px以上）：

- 右: 右サイドバー（幅320px）も出現。おすすめユーザー、トレンド等の補助情報

Flexboxによる3カラム配置

レイアウトの核心は `<div className="flex">` で囲まれた3つの要素です。

```
<div class="flex">
  <aside class="w-64 ... ">左サイドバー</aside>
  <main class="flex-1 ... ">メインコンテンツ</main>
  <aside class="w-80 ... ">右サイドバー</aside>
</div>
```

Flexboxの動作



画面幅1280pxの場合: Sidebar: 256px / main: 1280 - 256 - 320 = 704px / RightSidebar: 320px

`flex-1` は `flex: 1 1 0%` の短縮形で、残りの空間をすべて占有する。サイドバーの幅は固定で、メインコンテンツが伸縮する。

スティッキーサイドバー

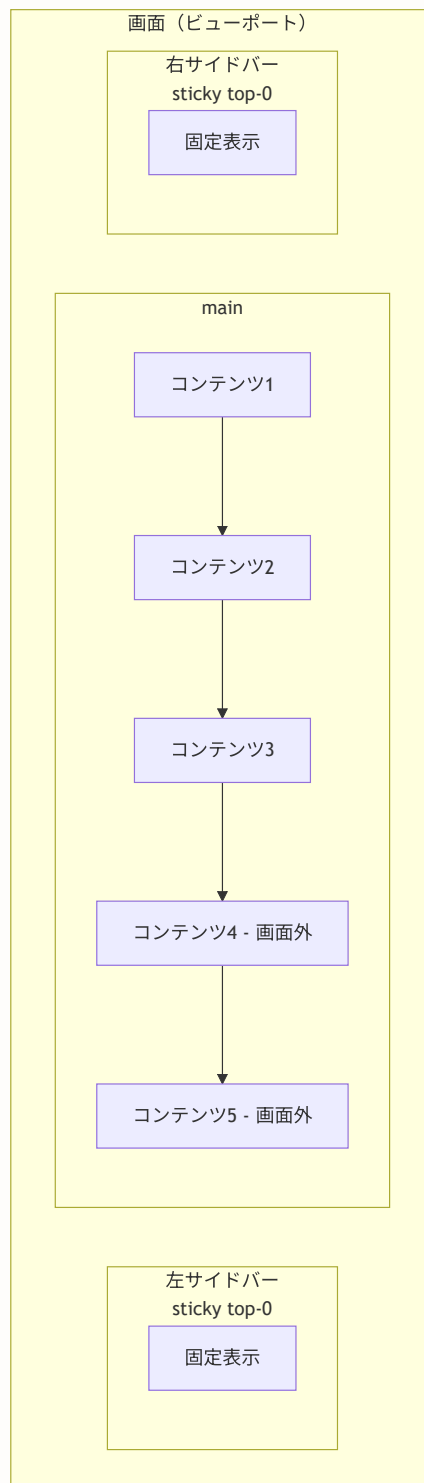
両サイドバーには `sticky top-0 h-screen` が適用されています。これにより、メインコンテンツをスクロールしてもサイドバーは画面に固定されます。

```

<!-- 左サイドバー（BON-LOGの実際のクラス） -->
<aside class="sticky top-0 h-screen w-64 border-r bg-card/95 backdrop-blur-sm hidden lg:flex" >
  <!-- ... -->
</aside>

<!-- 右サイドバー（BON-LOGの実際のクラス） -->
<aside class="sticky top-0 h-screen w-80 border-l bg-card/95 backdrop-blur-sm hidden xl:flex" >
  <!-- ... -->
</aside>
```

スティッキーの仕組み



sticky positioning の動作:

- **sticky**: 通常フローに残りつつ、スクロール時は画面上端に固定
- **h-screen**: ビューポートの高さいっぱい広がる
- **overflow-y-auto**: 右サイドバーの内容が長い場合はサイドバー内部でスクロール可能
- メインコンテンツはスクロール可能、サイドバーは固定

stickyの注意点

```
// sticky が動作するための条件:  
  
// 1. 親要素にoverflow: hidden/auto が設定されていないこと  
//   × <div className="overflow-hidden"> ← stickyが効かなくなる  
//     <aside className="sticky top-0">  
  
// 2. top, bottom, left, right のいずれかが指定されていること  
//   × <aside className="sticky">           ← 位置が指定されていない  
//   ○ <aside className="sticky top-0">      ← top-0を指定  
  
// 3. h-screen で高さを明示すること (サイドバーの場合)  
//   h-screen がないと、コンテンツの高さに合わせてサイドバーも伸びてしまう
```

レスポンシブブレイクポイントの使い分け

画面幅	表示される要素	レイアウト
< 1024px	Header + main + MobileNav	1カラム + ボトムナビ
>= 1024px (lg)	Sidebar + main	2カラム、左サイドバーが表示
>= 1280px (xl)	Sidebar + main + RightSidebar	3カラム、右サイドバーも表示

各コンポーネントの表示条件

```

<!-- 左サイドバー: lg以上で表示 -->
<aside class="hidden lg:flex ... ">

<!-- 右サイドバー: xl以上で表示 -->
<aside class="hidden xl:flex ... ">

<!-- モバイルヘッダー: lg未満で表示 (Headerコンポーネント内で制御) -->

<!-- モバイルボトムナビ: lg未満で表示 (MobileNavコンポーネント内で制御) -->

```

メインコンテンツの幅制限

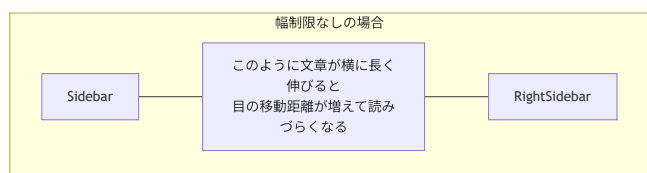
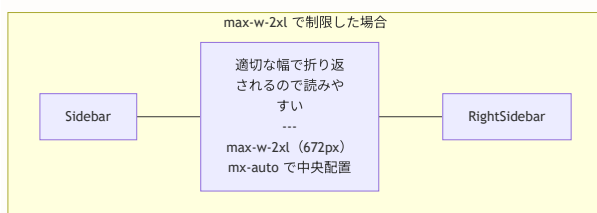
```

<main class="flex-1 min-h-screen pb-16 lg:pb-0">
  <div class="max-w-2xl mx-auto px-4 py-4 lg:py-6">
    {children}
  </div>
</main>

```

クラス	役割
<code>flex-1</code>	Flexbox内で残りの幅をすべて占有
<code>min-h-screen</code>	最低でも画面の高さを確保
<code>pb-16 lg:pb-0</code>	モバイルではボトムナビ分のパディング、デスクトップでは不要
<code>max-w-2xl</code>	コンテンツの最大幅を672pxに制限 (読みやすさのため)
<code>mx-auto</code>	コンテンツを中央に配置
<code>px-4</code>	左右にパディング (モバイルで端にくっつかない)

なぜmax-w-2xlで幅を制限するのか



背景とvisual effects

```

<!-- ルート要素: 背景色 + 麻の葉模様 -->
<div class="min-h-screen bg-background asanoha-pattern">

<!-- サイドバー: 半透明の背景 + ブラー -->
<aside class="bg-card/95 backdrop-blur-sm shadow-washi">
<!--          ↑ 95%の不透明度    ↑ 背後をぼかす    ↑ 和風シャドウ -->

```

画面表示 ページ全体の背景には生成り色の上に麻の葉の幾何学模様がうっすらと描かれている。サイドバーは和紙色の半透明（95%不透明）で、背後の麻の葉模様がほんのわずかに透けて見える。`backdrop-blur-sm`により透けた部分がぼやけ、すりガラスのような和の趣のある効果生まれる。和風シャドウ（`shadow-washi`）で柔らかい影も付き、サイドバーとメインコンテンツの境界が自然に区別される。

`bg-card/95` は「カード背景色を95%の不透明度で適用」を意味し、`backdrop-blur-sm`と組み合わせることで、背後の麻の葉模様がうっすら透けて見える、すりガラスのような効果を実現しています。

Grid レイアウトとの比較

3カラムレイアウトはCSS Gridでも実現できますが、BON-LOGではFlexboxを採用しています。

```

<!-- Flexbox版（BON-LOGの採用方式） -->
<div class="flex">
  <aside class="w-64 hidden lg:flex">左</aside>
  <main class="flex-1">中央</main>
  <aside class="w-80 hidden xl:flex">右</aside>
</div>

<!-- Grid版（代替実装） -->
<div class="grid grid-cols-1 lg:grid-cols-[256px_1fr] xl:grid-cols-[256px_1fr_320px]">
  <aside class="hidden lg:block">左</aside>
  <main>中央</main>
  <aside class="hidden xl:block">右</aside>
</div>

```

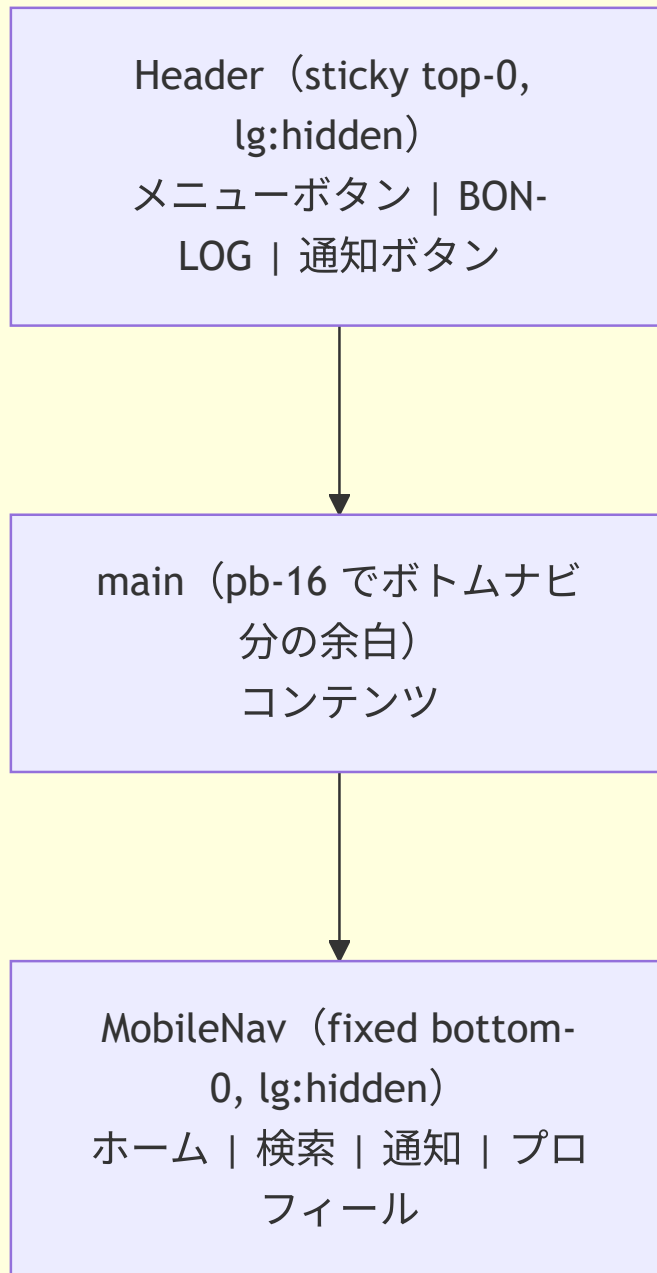
	Flexbox	Grid
サイドバーの固定幅指定	<code>w-64</code> で直接指定	<code>grid-cols-[256px_1fr]</code> で指定
レスポンス対応	<code>hidden lg:flex</code> で要素ごとに制御	<code>grid-cols</code> を切り替え
記述のシンプルさ	シンプル	やや複雑
高さの制御	<code>h-screen</code> で個別指定	<code>align-items</code> で一括制御

BON-LOGでは、各カラムの表示/非表示をレスポンスに制御する必要があるため、要素単位で `hidden` / `flex` を切り替えられるFlexboxがより適しています。

モバイル用ナビゲーション

モバイルでは左右のサイドバーが非表示になるため、ボトムナビゲーション（MobileNav）とヘッダー（Header）でナビゲーションを提供します。

モバイル画面レイアウト



pb-16の重要性

```
<main class="flex-1 min-h-screen pb-16 lg:pb-0">
```

`pb-16` (`padding-bottom: 4rem = 64px`) は、モバイルのボトムナビゲーションが `fixed bottom-0` で画面下部に固定されるため、メインコンテンツの最下部がナビに隠れないようにするためのパディングです。
`lg:pb-0` で、デスクトップ（ボトムナビなし）ではこのパディングを除去しています。

理解度チェック

▶ Q1: 左サイドバーが `lg:flex` で、右サイドバーが `xl:flex` である理由は何ですか？

▶ Q2: `sticky` と `fixed` の違いは何ですか？サイドバーになぜ `sticky` を使うのですか？

▶ Q3: `bg-card/95` と `backdrop-blur-sm` を組み合わせる効果は何ですか？

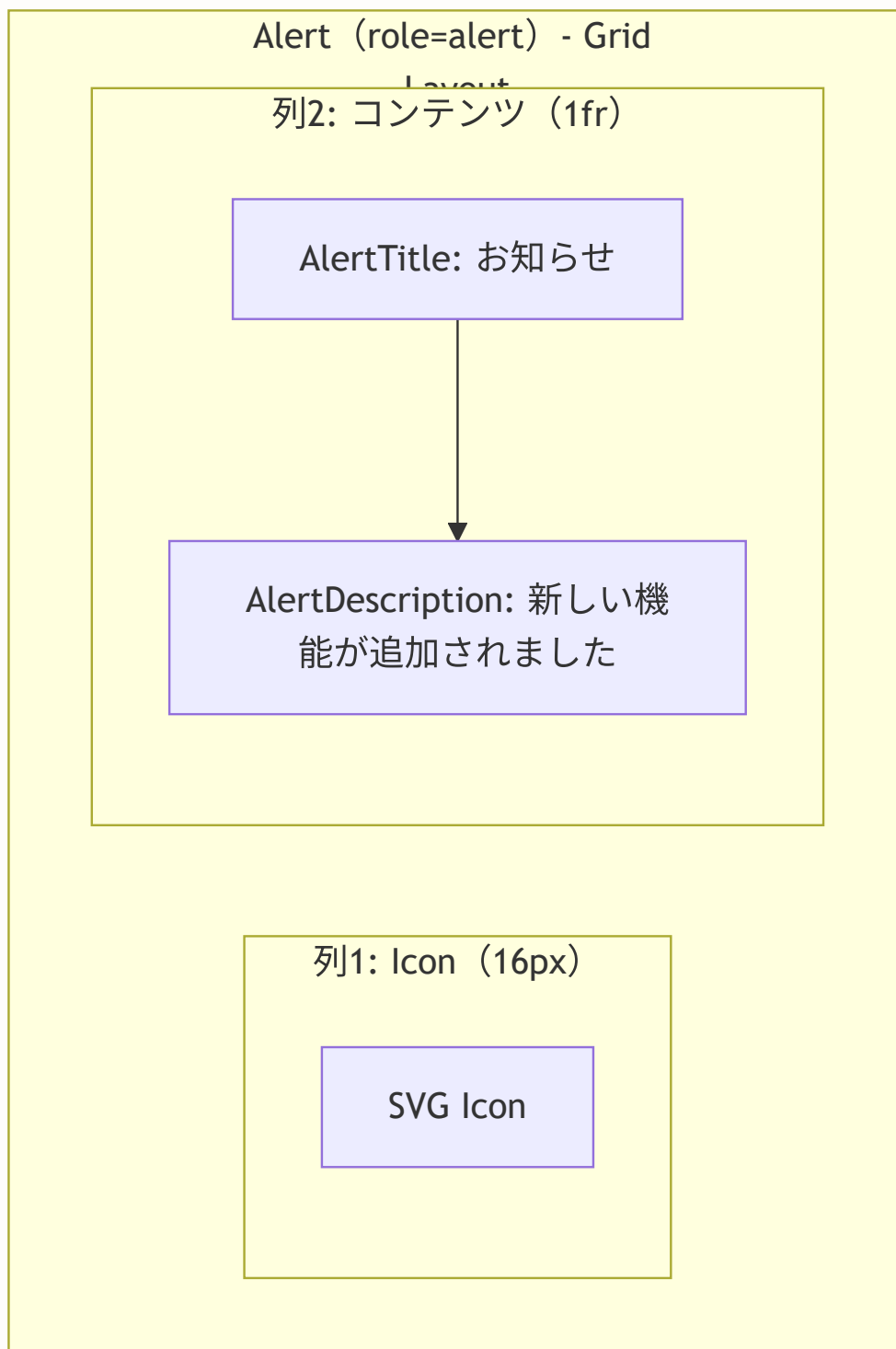
6.16 CVA応用: 実際のコンポーネントを深く読む

このセクションで学ぶこと

- BON-LOGの実際のAlertコンポーネントでのCVA活用パターン
- Inputコンポーネントで `cn()` のみを使うケースとCVAを使うケースの違い
- `data-slot` 属性を使った高度なCSS連携
- `[a&]` や `has-[>svg]` などの高度なTailwindセレクトの読み方
- 自作コンポーネントでCVAを設計する際の考え方

Alertコンポーネント: CVA + Grid レイアウトの実例

BON-LOGの `components/ui/alert.tsx` は、CVAとCSSグリッドを組み合わせた実践的な例です。

**Grid Layout:**

- SVGアイコンあり: `grid-cols-[16px_1fr]`
- SVGアイコンなし: `grid-cols-[0_1fr]`
- Title/Description は常に `col-start-2`

Alertのベーススタイルを1行ずつ読む

```

/* alertVariantsのベーススタイル（実際のコード）を分解して解説 */

relative          /* 相対配置：子要素のabsolute配置の基準点になる */
w-full            /* 幅100%：親コンテナいっぱい広がる */
rounded-lg        /* 角丸：大きめの丸み（0.5rem = 8px） */
border            /* ボーダー：1pxの枠線 */
px-4 py-3         /* 内側余白：左右16px、上下12px */
text-sm          /* フォントサイズ：14px */

/* Gridレイアウト関連 */
grid              /* display: grid を有効化 */

has-[>svg]:grid-cols-[calc(var(--spacing)*4)_1fr]
/* ↑ 直接の子要素にSVGアイコンがある場合のグリッド列定義
   第1列：16px（アイコン用）
   第2列：1fr（残りの幅すべて） */

grid-cols-[0_1fr]
/* ↑ SVGがない場合のデフォルト
   第1列：0px（幅なし）
   第2列：1fr（全幅） */

has-[>svg]:gap-x-3 /* SVGがある場合、列間に12pxのスペース */
gap-y-0.5         /* 行間に2pxのスペース */
items-start       /* グリッドアイテムを上揃え */

/* SVGアイコンのスタイル */
[&svg]:size-4      /* 直接の子SVGは 16x16px */
[&svg]:translate-y-0.5 /* SVGを2px下にずらす（テキストとの視覚的調整） */
[&svg]:text-current /* SVGの色はテキスト色に合わせる */

```

has-[>svg] セレクタの仕組み

このセレクタはCSS `:has()` 擬似クラスのTailwind表現で、「直接の子要素にSVGがあるかどうか」で条件分岐します。

条件	構造例	適用されるスタイル
SVGアイコンあり	<pre> <Alert> <svg ... </svg> <AlertTitle> <AlertDescription> </Alert> </pre>	<code>has-[>svg]:grid-cols-[16px_1fr]</code> が適用、 <code>has-[>svg]:gap-x-3</code> が適用
SVGアイコンなし	<pre> <Alert> <AlertTitle> <AlertDescription> </Alert> </pre>	<code>grid-cols-[0_1fr]</code> が適用（デフォルト）、アイコン列の幅が0なので実質1カラム

このように、同じコンポーネントが「アイコンあり」「アイコンなし」の両方に対応できるのは、`has-
[>svg]` を使った条件付きスタイルのおかげです。従来なら `icon` という props を追加して if 分岐する必要がありましたが、CSS だけで解決しています。

Alertのバリエーション定義

```
// components/ui/alert.tsx (BON-LOGの実際のコード)
const alertVariants = cva(
  "relative w-full rounded-lg border px-4 py-3 text-sm grid ...",
  {
    variants: {
      variant: {
        // default: カード背景色で情報を表示
        default: "bg-card text-card-foreground",

        // destructive: エラーや危険情報を赤系で強調
        destructive:
          "text-destructive bg-card [>svg]:text-current *:data-[slot=alert-description]:t
          //                                     ↑ アイコンも赤に   ↑ 説明文は少し薄い赤に
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)
```

`*:data-[slot=alert-description]` の読み方

セレクト `*:data-[slot=alert-description]:text-destructive/90` の分解:

パーツ	意味
<code>*</code>	任意の子孫要素
<code>:data-[slot=alert-description]</code>	<code>data-slot="alert-description"</code> 属性を持つ要素
<code>:text-destructive/90</code>	テキスト色: destructiveの90%不透明度

AlertDescriptionコンポーネントは `data-slot="alert-description"` を持っているので:

```
<div data-slot="alert-description" class=" ... ">
  説明文テキスト
</div>
```

この要素に `text-destructive/90` が適用される。

この `data-slot` パターンは shadcn/ui が採用している設計パターンで、コンポーネントの内部要素にセマンティックな名前を付け、親のバリエーションからスタイルを適用できるようにしています。

[a&] セレクタの意味

Badgeコンポーネントのバリエーションに登場する `[a&]:hover:bg-primary/90` の読み方を解説します。

セレクタ `[a&]:hover:bg-primary/90` の分解:

パーツ	意味
<code>a</code>	<code><a></code> タグ
<code>&</code>	現在の要素自身 (Badge)
<code>:hover</code>	ホバー状態のとき
<code>:bg-primary/90</code>	背景色: primaryの90%不透明度

意味: 「この要素がaタグとしてレンダリングされている場合のホバースタイル」

```
<!-- spanとしてレンダリング → ホバースタイルなし -->
<Badge>新着</Badge>

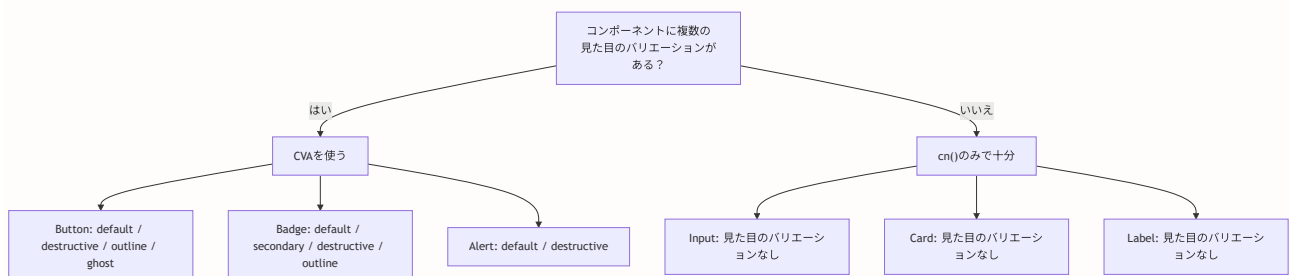
<!-- aタグとしてレンダリング → ホバースタイルあり -->
<Badge asChild>
  <a href="/news">新着</a>
</Badge>
```

Inputコンポーネント: CVAなしのcn()パターン

BON-LOGの `components/ui/input.tsx` は、CVAなしで `cn()` のみを使用しています。Inputコンポーネントにはバリエーションが不要だからです。

```
// components/ui/input.tsx (BON-LOGの実際のコード)
function Input({ className, type, ...props }: React.ComponentProps<"input">) {
  return (
    <input
      type={type}
      data-slot="input"
      className={cn(
        // ベーススタイル
        "file:text-foreground placeholder:text-muted-foreground/70 ... ",
        // フォーカス時
        "focus-visible:border-primary focus-visible:ring-primary/20 ... ",
        // ホバー時
        "hover:border-border hover:bg-card/80",
        // エラー時
        "aria-invalid:ring-destructive/20 ... ",
        // 外部からのオーバーライド
        className
      )}
    />
  )
}
```

CVAを使うかどうかの判断基準



Inputの高度なTailwindクラスを読み解く

Inputコンポーネントには、初心者には馴染みのない高度なクラスが多く含まれています。

```

/* ファイル入力関連 */
file:text-foreground /* <input type="file"> のテキスト色 */
file:inline-flex /* ファイル選択ボタンをインラインフレックスに */
file:h-7 /* ファイル選択ボタンの高さ: 28px */
file:border-0 /* ファイル選択ボタンのボーダーなし */
file:bg-transparent /* ファイル選択ボタンの背景: 透明 */
file:text-sm /* ファイル選択ボタンのフォント: 14px */
file:font-medium /* ファイル選択ボタンのフォント太さ: 500 */

/* プレースホルダー */
placeholder:text-muted-foreground/70
/* ↑ プレースホルダーテキストを薄いグレーに（70%不透明度） */

/* テキスト選択時 */
selection:bg-primary /* 選択部分の背景: プライマリカラー（松葉色） */
selection:text-primary-foreground /* 選択部分のテキスト: 白 */

/* バリデーションエラー（aria-invalid="true" が設定された場合） */
aria-invalid:ring-destructive/20 /* 赤いリングを薄く表示 */
aria-invalid:border-destructive /* ボーダーを赤に変更 */

/* ダークモード専用 */
dark:bg-input/30 /* ダークモードでの背景色 */
dark:aria-invalid:ring-destructive/40
/* ↑ ダークモードではエラーリングを少し濃くして視認性を確保 */

```

md:text-sm の重要性

```

// Inputの最後のクラス
"text-base md:text-sm"

```

画面サイズ	クラス	フォントサイズ	理由
モバイル (md未満)	text-base	16px	iOSのSafariは16px未満のフォントサイズのinputにフォーカスすると自動的にズームする。ユーザー体験を損なうため、16pxにすることでこの自動ズームを防止する。
デスクトップ (md以上)	md:text-sm	14px	デスクトップではズームの問題がないため、他のUIコンポーネントと揃えた14pxのフォントサイズを使用して統一感を出す。

自作コンポーネントでのCVA設計手順

BON-LOGで独自のCVAコンポーネントを作る際のステップバイステップガイドです。

```
// === Step 1: バリエーションの洗い出し ===
// まず、コンポーネントに必要な見た目のバリエーションをリストアップする
// 例: 通知バッジ
// - type: info / success / warning / error
// - size: sm / default / lg

// === Step 2: ベーススタイルを定義 ===
// 全バリエーションに共通するスタイルを抽出
import { cva, type VariantProps } from 'class-variance-authority'
import { cn } from '@lib/utils'

const notificationBadgeVariants = cva(
  // ベーススタイル: 全バリエーションに共通
  'inline-flex items-center gap-1.5 rounded-full font-medium border transition-colors',
  {
    // === Step 3: バリエーションごとのスタイルを定義 ===
    variants: {
      type: {
        info: 'bg-ai/10 text-ai border-ai/20', // 藍色ベース
        success: 'bg-matcha/10 text-matcha border-matcha/20', // 抹茶色ベース
        warning: 'bg-kincha/10 text-kincha border-kincha/20', // 金茶色ベース
        error: 'bg-aka/10 text-aka border-aka/20', // 朱色ベース
      },
      size: {
        sm: 'px-2 py-0.5 text-xs',
        default: 'px-3 py-1 text-sm',
        lg: 'px-4 py-1.5 text-base',
      },
    },
  },

  // === Step 4: デフォルト値を設定 ===
  {
    defaultVariants: {
      type: 'info',
      size: 'default',
    },
  }
)

// === Step 5: コンポーネントを実装 ===
type NotificationBadgeProps = React.ComponentProps<'span'> &
  VariantProps<typeof notificationBadgeVariants>

export function NotificationBadge({
  type,
  size,
  className,
  children,
  ...props
}: NotificationBadgeProps) {
  return (
    <span
```

```

    data-slot="notification-badge"
    className={cn(notificationBadgeVariants({ type, size }), className)}
    { ... props }
  >
    {children}
  </span>
)
}

// ≡ Step 6: 使用する ≡
// <NotificationBadge type="success">保存しました</NotificationBadge>
// <NotificationBadge type="error" size="lg">エラーが発生しました</NotificationBadge>
// <NotificationBadge type="warning" className="mt-2">注意</NotificationBadge>

```

理解度チェック

▶ Q1: `has- [>svg]:grid-cols-[16px_1fr]` はどのような場合に適用されますか？

▶ Q2: Inputコンポーネントが`text-base (16px)`を使用する理由は何ですか？

▶ Q3: CVAを使うコンポーネントと`cn()`のみで十分なコンポーネントの違いは何ですか？

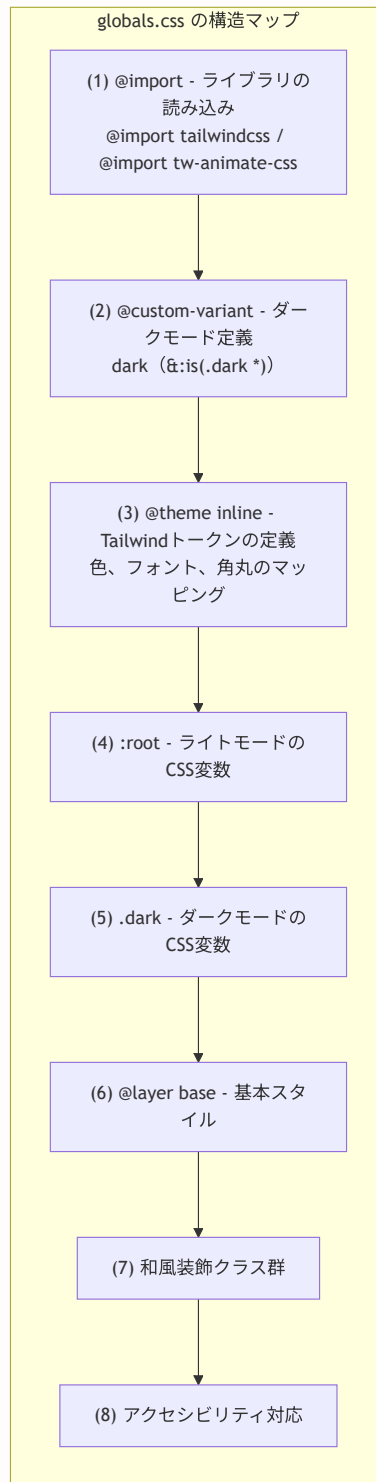
6.17 Tailwind CSS v4 詳細: BON-LOGの `globals.css` を完全に読む

このセクションで学ぶこと

- `@import` ディレクティブによるTailwindとアニメーションの読み込み
- `@custom-variant` によるダークモードの制御方法
- `@theme inline` ブロック内の全トークン定義の読み方
- `:root` と `.dark` でのCSS変数切り替えの仕組み
- `radius` (角丸) のサイズスケール設計

globals.css の全体構造

BON-LOGの `app/globals.css` は、Tailwind CSS v4の構成に従って以下の構造になっています。



(1) @import ディレクティブ

```
/* app/globals.css (BON-LOGの実際のコード) */
@import "tailwindcss";
@import "tw-animate-css";
```

Tailwind CSS v4では、従来の `@tailwind base; @tailwind components; @tailwind utilities;` の3行が `@import "tailwindcss"` の1行に統合されました。

バージョン	記述方法
Tailwind CSS v3 (従来)	<code>@tailwind base;</code> (リセットCSS) / <code>@tailwind components;</code> (コンポーネント) / <code>@tailwind utilities;</code> (ユーティリティクラス)
Tailwind CSS v4 (統合)	<code>@import "tailwindcss";</code> (全部入り)

`tw-animate-css` は、フェードイン、スライドイン、アコーディオンなどのアニメーションユーティリティを追加するプラグインです。shadcn/uiのDialogやDropDownMenuで使われるアニメーションに必要です。

(2) @custom-variant ダークモード

```
/* app/globals.css (BON-LOGの実際のコード) */
@custom-variant dark (&:is(.dark *));
```

この1行で「ダークモードの検出方法」を定義しています。

`@custom-variant dark (&:is(.dark *))` の意味:

- `dark:` プレフィックスが付いたクラスは、祖先要素に `.dark` クラスがある場合に適用される
- 例: `dark:bg-gray-900` は `.dark` 配下の要素にのみ `bg-gray-900` が適用

```
<html class="dark">      <!-- .dark がここにある -->
  <body>
    <div class="dark:bg-gray-900">
      <!-- .dark の子孫なので dark:bg-gray-900 が適用 -->
    </div>
  </body>
</html>
```

方式	記述	特徴
クラス方式 (BON-LOG採用)	<code>@custom-variant dark (&:is(.dark *))</code>	ユーザーがトグルスイッチで切り替え可能。 JavaScriptで <code><html></code> に <code>.dark</code> クラスを付け外しする
OS設定方式 (v3デフォルト)	<code>@custom-variant dark (prefers-color-scheme: dark)</code>	OSの設定に従う (ユーザーが手動切替できない)

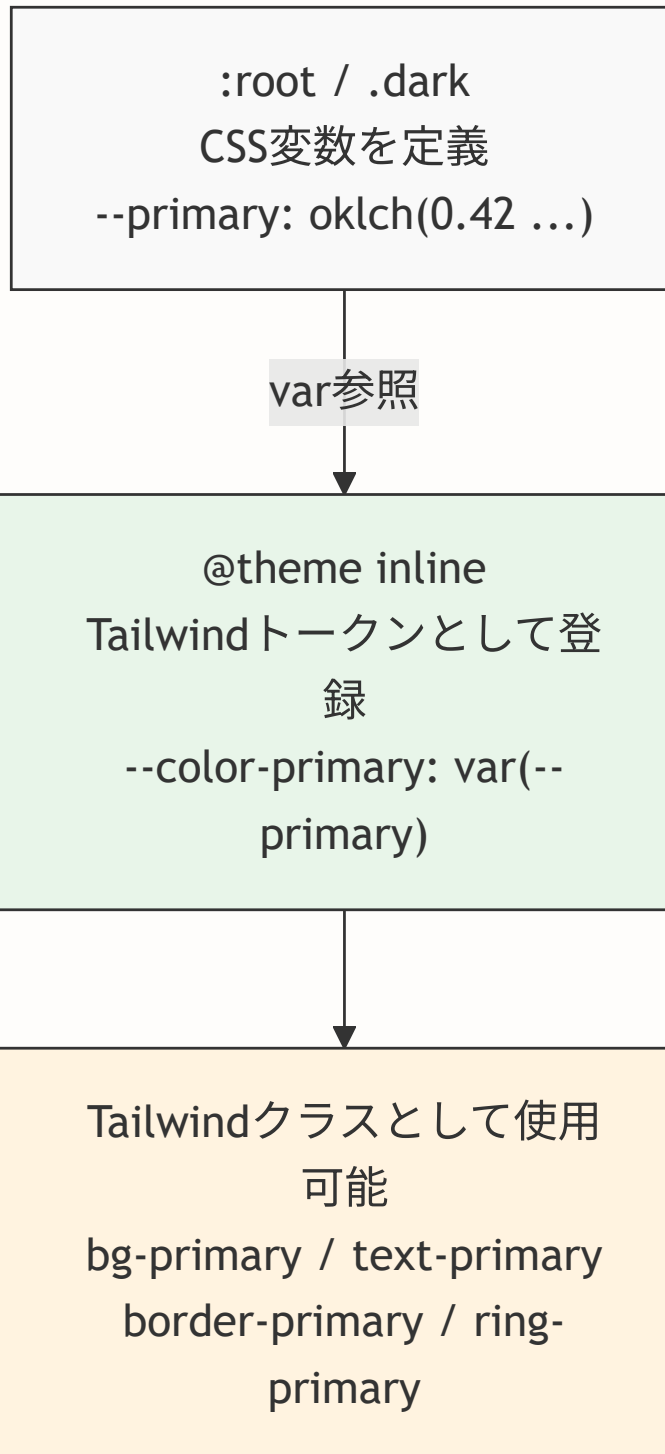
(3) @theme inline トークン定義

```

/* app/globals.css (BON-LOGの実際のコード) */
@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-noto-sans-jp);
  --font-mono: var(--font-geist-mono);
  --font-serif: var(--font-shippori-mincho);
  --color-bonsai-green: var(--bonsai-green);
  --color-bonsai-brown: var(--bonsai-brown);
  /* ... */
}

```

`@theme inline` はTailwind CSS v4の核心的な機能です。ここで定義した値が、Tailwindのユーティリティクラスとして使えるようになります。



inline キーワードの意味: CSS変数を参照 (`var(...)`) する場合に使用。出力CSSにトークン値が直接含まれない。ライト/ダークでCSS変数の値だけ切り替えれば、全てのTailwindクラスが自動的に更新される。

フォントファミリーのマッピング

```
@theme inline {
  --font-sans: var(--font-noto-sans-jp);    /* 本文用: Noto Sans JP */
  --font-mono: var(--font-geist-mono);      /* コード用: Geist Mono */
  --font-serif: var(--font-shippori-mincho); /* 装飾用: しっぽり明朝 */
}
```

フォントファミリー	フォント名	用途	適用クラス	表示例
font-sans	Noto Sans JP	読みやすいゴシック体。本文、ボタン、ラベルに使用	<code>class="font-sans"</code>	盆栽の手入れには、適切な水やりが重要です。
font-serif	しっぽり明朝	和風の雰囲気を出す明朝体。見出し、引用に使用	<code>class="font-serif"</code>	盆栽の手入れには、適切な水やりが重要です。
font-mono	Geist Mono	等幅フォント。コードブロック、数値表示に使用	<code>class="font-mono"</code>	<code>const bonsai = await getPost(id)</code>

角丸 (radius) のスケールシステム

```
@theme inline {
  --radius-sm: calc(var(--radius) - 4px); /* 0rem = 0px */
  --radius-md: calc(var(--radius) - 2px); /* 0.125rem = 2px */
  --radius-lg: var(--radius);             /* 0.25rem = 4px */
  --radius-xl: calc(var(--radius) + 4px); /* 0.5rem = 8px */
  --radius-2xl: calc(var(--radius) + 8px); /* 0.75rem = 12px */
  --radius-3xl: calc(var(--radius) + 12px); /* 1rem = 16px */
  --radius-4xl: calc(var(--radius) + 16px); /* 1.25rem = 20px */
}

:root {
  --radius: 0.25rem; /* 基準値: 4px */
}
```

角丸スケールの視覚比較 (`--radius = 0.25rem (4px)` の場合)

クラス	サイズ	見た目	用途
<code>rounded-sm</code>	0px	直角	-
<code>rounded-md</code>	2px	わずかな丸み	-
<code>rounded-lg</code>	4px	標準的な丸み	ボタンなどで使用
<code>rounded-xl</code>	8px	大きめの丸み	カードで使用
<code>rounded-2xl</code>	12px	かなり丸い	モーダルで使用

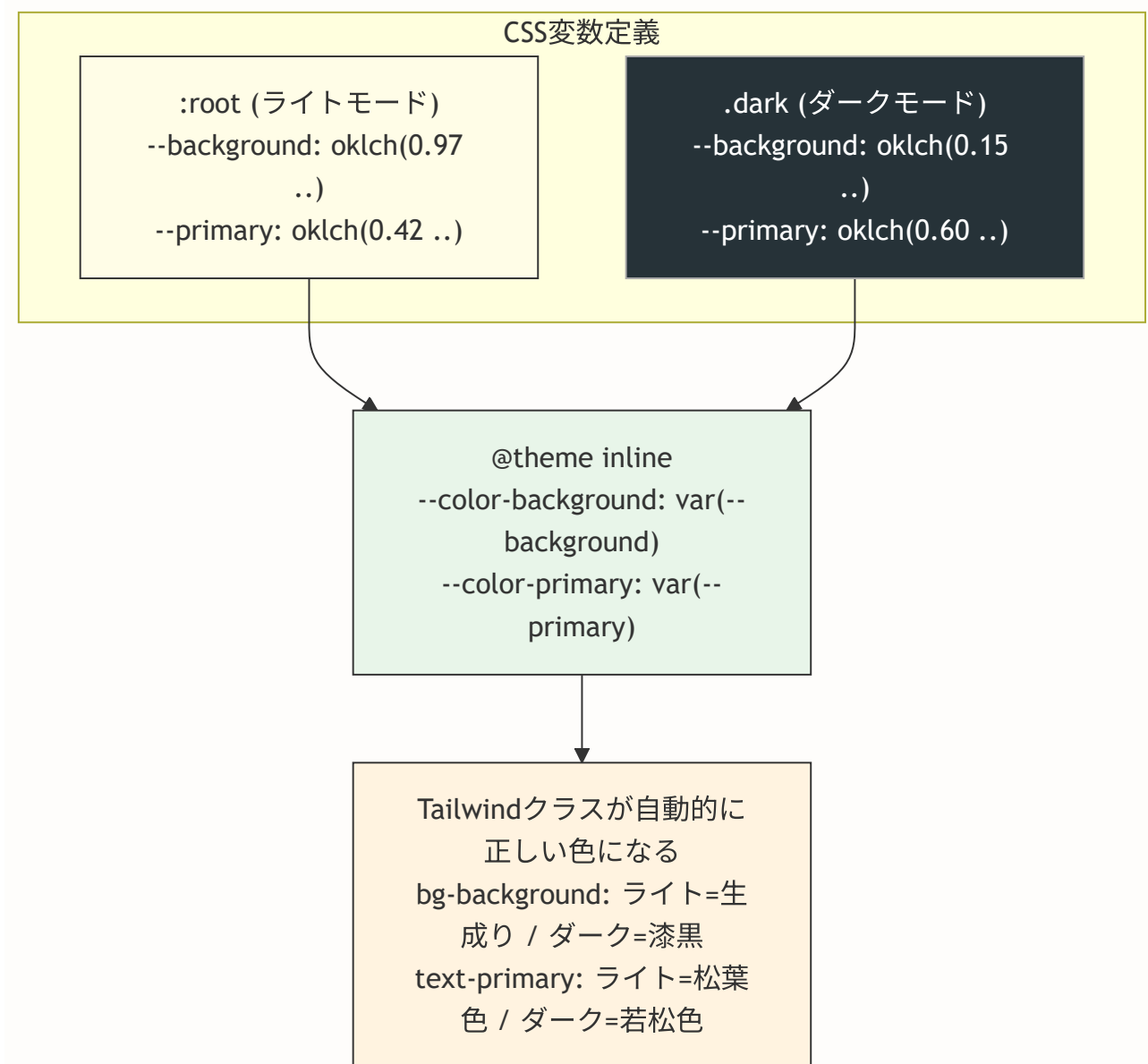
BON-LOGでは控えめな角丸（0.25rem基準）を採用。和風デザインは直線的な美しさを重視するため、デフォルト（0.5rem）より小さい値にしている。

(4)(5) ライトモードとダークモードの切り替え

`:root`（ライトモード）と `.dark`（ダークモード）で同じCSS変数名に異なる値を設定することで、テーマを切り替えます。

```
/* ライトモード */
:root {
  --primary: oklch(0.42 0.08 155); /* 松葉色（暗い緑） */
  --background: oklch(0.97 0.008 85); /* 生成り（明るい） */
}

/* ダークモード */
.dark {
  --primary: oklch(0.60 0.10 155); /* 若松色（明るい緑） */
  --background: oklch(0.15 0.015 50); /* 漆黒（暗い） */
}
```



コンポーネントのコードは一切変更不要。CSS変数の値が切り替わるだけで全体のテーマが変わる。

理解度チェック

▶ Q1: @theme inline の「inline」キーワードの役割は何ですか？

▶ Q2: BON-LOGの角丸が控えめ（0.25rem基準）に設定されている理由は何ですか？

▶ Q3: @custom-variant dark (&:is(.dark *)) がOSのダークモード設定 (prefers-color-scheme) より優れている点は？

6.18 和風タイポグラフィとカラーの深掘り

このセクションで学ぶこと

- oklch() カラー空間のパラメータ（明度・彩度・色相）の実践的な調整方法
- ライトモードからダークモードへの色の変換テクニック
- 和風テクスチャクラス（washi-texture、asanoha-pattern等）のSVG実装の仕組み
- 和風シャドウ（shadow-washi）の多層構造
- 縦書きテキスト、印鑑風装飾などの和風タイポグラフィ

oklch()の3つのパラメータを理解する

BON-LOGのカラーパレットは全てoklch()カラー空間で定義されています。3つのパラメータを直感的に理解しましょう。

oklch(L, C, H) - 3つのパラメータ

パラメータ	名前	範囲	説明
L	Lightness（明度）	0.0（真っ黒）～ 1.0（真っ白）	色の明るさ
C	Chroma（彩度）	0.0（無彩色）～ 0.4（最大彩度）	色の鮮やかさ
H	Hue（色相）	0～360	色の種類を角度で指定

色相環と角度:

角度	色	角度	色
0°	赤	180°	青緑
30°	オレンジ	210°	シアン
90°	黄	270°	紫
150°	緑	330°	ピンク

BON-LOGの主要色の色相:

色名	H（色相）	系統
松葉色	155	緑系
栗皮茶	55	黄～茶系
錆朱	45	オレンジ系
朱色	25	赤系
藍	250	青系
抹茶	140	緑系
金茶	70	黄系

ライトモードからダークモードへの変換パターン

```

/* 松葉色の変換例 */
/* ライトモード */
--primary: oklch(0.42 0.08 155); /* 明度0.42（暗め）、彩度0.08 */

/* ダークモード */
--primary: oklch(0.60 0.10 155); /* 明度0.60（明るめ）、彩度0.10 */

```

ダークモード色変換のルール:

要素	ライトモード	ダークモード	変換方向
背景	明るい (L=0.97)	暗い (L=0.15)	明度を反転 (明→暗)
テキスト	暗い (L=0.22)	明るい (L=0.92)	明度を反転 (暗→明)
プライマリ	暗め (L=0.42)	明るめ (L=0.60)	明度を少し上げる + 彩度も少し上げる

変換パターン:

1. 背景色: 明度を反転 (明→暗)
2. テキスト色: 明度を反転 (暗→明)
3. アクセント色: 明度を少し上げる + 彩度も少し上げる (暗い背景上で見えやすくするため)
4. 色相(H)は変えない (同じ色味を保持)

具体例 (松葉色 → 若松色):

パラメータ	ライト	ダーク	差分
L (明度)	0.42	0.60	+0.18
C (彩度)	0.08	0.10	+0.02
H (色相)	155	155	変化なし

和風テクスチャの実装解説

和紙テクスチャ（washi-texture）の仕組み

```
/* app/globals.css (BON-LOGの実際のコード) */
.washi-texture {
  background-image:
    url("data:image/svg+xml,%3Csvg ... %3E
      %3Cfilter id='noise'%3E
        %3CfeTurbulence
          type='fractalNoise' /* フラクタルノイズ */
          baseFrequency='0.8' /* ノイズの細かさ */
          numOctaves='4' /* ノイズの複雑さ */
          stitchTiles='stitch' /* タイルの継ぎ目を滑らかに */
        %3E
      %3C/filter%3E
      %3Crect ... filter='url(%23noise)' opacity='0.03' %3E
    %3C/svg%3E");
}
```

和紙テクスチャの仕組み (SVGフィルター `feTurbulence` を使用)

パラメータ	値	説明
<code>baseFrequency</code>	<code>0.8</code>	数値が大きいほど模様が細かい
<code>numOctaves</code>	<code>4</code>	数値が大きいほど模様が複雑
<code>opacity</code>	<code>0.03</code>	ほぼ見えないレベルの薄さ。和紙の繊維感を微かに表現

このテクスチャはインラインSVGのData URIとして埋め込まれているため、外部ファイルの読み込みが不要。パフォーマンスに影響を与えずに質感を追加できる。

和風シャドウ（shadow-washi）の多層設計

```
/* app/globals.css (BON-LOGの実際のコード) */
.shadow-washi {
  box-shadow:
    0 1px 2px rgba(34, 34, 34, 0.04), /* 第1層: 最も近い影 */
    0 2px 4px rgba(34, 34, 34, 0.04), /* 第2層: 中間の影 */
    0 4px 8px rgba(34, 34, 34, 0.04); /* 第3層: 遠い影 */
}
```

多層シャドウの視覚効果

方式	影の数	印象
通常の <code>box-shadow</code>	1層	ベタツとした平面的な印象
<code>shadow-washi</code> (3層の影)	3層	自然で柔らかい浮遊感。各層の不透明度が0.04と非常に薄く、和紙が重なったような繊細な表現

なぜ `rgba(34, 34, 34, ...)` を使うのか？ 純粋な黒（0,0,0）よりも温かみのある影になる。
`#222222` は墨色に近い色で、和風デザインと調和する。

和風タイポグラフィの活用

縦書きテキスト

```
/* app/globals.css (BON-LOGの実際のコード) */
.vertical-text {
  writing-mode: vertical-rl; /* 右から左へ縦書き */
  text-orientation: upright; /* 文字を正立（回転させない） */
}
```

```

<!-- 使用例: 和風の見出し -->
<div class="flex items-center gap-4">
  <div class="vertical-text font-serif text-primary text-lg tracking-widest">
    盆栽日記
  </div>
  <div class="flex-1">
    <!-- 通常の横書きコンテンツ -->
    <p>今日は五葉松の葉刈りを行いました。</p>
  </div>
</div>

```

レイアウト

盆
裁
日
記

(vertical-text
font-serif)

今日は五葉松の葉刈りを行
いました。
春先の手入れとして、昨年
伸びた葉を...

`vertical-text` + `font-serif` (しっぽり明朝) の組み合わせで和の雰囲気強く演出する。

印鑑風装飾 (hanko)

```
/* app/globals.css (BON-LOGの実際のコード) */
.hanko {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 2.5rem;          /* 40px */
  height: 2.5rem;         /* 40px */
  border: 2px solid var(--aka); /* 朱色のボーダー */
  border-radius: 50%;      /* 完全な円 */
  color: var(--aka);       /* 朱色のテキスト */
  font-size: 0.75rem;     /* 12px */
  font-weight: 700;       /* 太字 */
}
```

```
<!-- 使用例: ユーザー認証マーク -->
<span class="hanko">認</span>

<!-- 使用例: 投稿の承認マーク -->
<span class="hanko">済</span>
```

印鑑風装飾の表示イメージ: 朱色の円形ボーダー + 漢字1文字で、まるで実物の印鑑を押したような見た目になる。

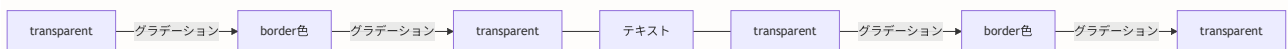
活用シーン:

- ユーザーの本人確認済みマーク
- 投稿の承認/確認状態の表示
- 和風のデコレーション要素

和風区切り線 (divider-bamboo)

```
/* app/globals.css (BON-LOGの実際のコード) */
.divider-bamboo {
  display: flex;
  align-items: center;
  gap: 1rem;
}
.divider-bamboo::before,
.divider-bamboo::after {
  content: '';
  flex: 1;
  height: 1px;
  background: linear-gradient(90deg, transparent, var(--border), transparent);
}
```

```
<!-- 使用例 -->
<div class="divider-bamboo text-sm text-muted-foreground">
  本日の投稿
</div>
```



両端が透明にフェードアウトする繊細なデザイン。従来の `<hr>` タグよりも和風の雰囲気がある。 `flex: 1` でテキストの左右を均等に埋める。

理解度チェック

▶ Q1: `oklch()` の色相(H)を変えずに明度(L)と彩度(C)だけを変えるとどうなりますか？

▶ Q2: `shadow-washi` が3層の影を使う理由は何ですか？

6.19 3カラムレイアウト詳細: 各コンポーネントの実装を深く読む

このセクションで学ぶこと

- Sidebarコンポーネントのナビゲーションアイテム構築パターン
- アクティブ状態のスタイリング（パス判定 + 条件付きクラス）
- RightSidebarのServer Component + 並列データ取得パターン
- MobileNavの「もっと見る」メニューの状態管理
- Headerのスティッキー配置とモバイル専用表示

Sidebarのナビゲーションスタイリング

BON-LOGの `components/layout/Sidebar.tsx` では、アクティブ状態の判定と条件付きスタイリングが組み合わされています。

```
// components/layout/Sidebar.tsx (BON-LOGの実際のコード)
{allNavItems.map((item) => {
  // パス判定: 完全一致 または 前方一致（子ルート含む）
  const isActive = pathname === item.href
    || pathname.startsWith(item.href + '/')
  const Icon = item.icon

  return (
    <li key={item.href}>
      <Link
        href={item.href}
        className={`flex items-center gap-3 px-4 py-3 rounded
          transition-all duration-200 ${
            isActive
              ? 'bg-primary/10 text-primary font-medium border-l-2 border-primary'
              : 'text-foreground hover:bg-muted/70 hover:translate-x-1'
            }`
      >
        {/* ... */}
      </Link>
    </li>
  )
})}
```

アクティブ状態のスタイリング詳解

ナビゲーションのアクティブ状態:

状態	クラス	効果
非アクティブ	<code>text-foreground</code>	通常のテキスト
	<code>hover:bg-muted/70</code>	ホバーで背景色
	<code>hover:translate-x-1</code>	ホバーで右に4px移動
アクティブ	<code>bg-primary/10</code>	薄い松葉色の背景
	<code>text-primary</code>	松葉色テキスト
	<code>font-medium</code>	中太字
	<code>border-l-2 border-primary</code>	左ボーダー（「現在地」を強調）

`hover:translate-x-1` の効果: マウスを乗せるとアイテムが右に4px移動する。微かなアニメーションで「触れている」感覚を演出し、`transition-all duration-200` でスムーズに動く。

パス判定ロジックの詳細

```
const isActive = pathname === item.href
|| pathname.startsWith(item.href + '/')
```

パス判定の2段階チェック（`item.href = '/notifications'` の場合）

pathname	完全一致	前方一致	結果
<code>/notifications</code>	true	false	Active
<code>/notifications/</code>	false	true	Active
<code>/notifications/123</code>	false	true	Active
<code>/notif</code>	false	false	Inactive
<code>/settings</code>	false	false	Inactive

なぜ `/` を付けて `startsWith` するのか？ `/notifications` で `startsWith` すると、`/notifications-settings` のような別パスもマッチしてしまう。`/` を付けることで確実に子ルートのみマッチする。

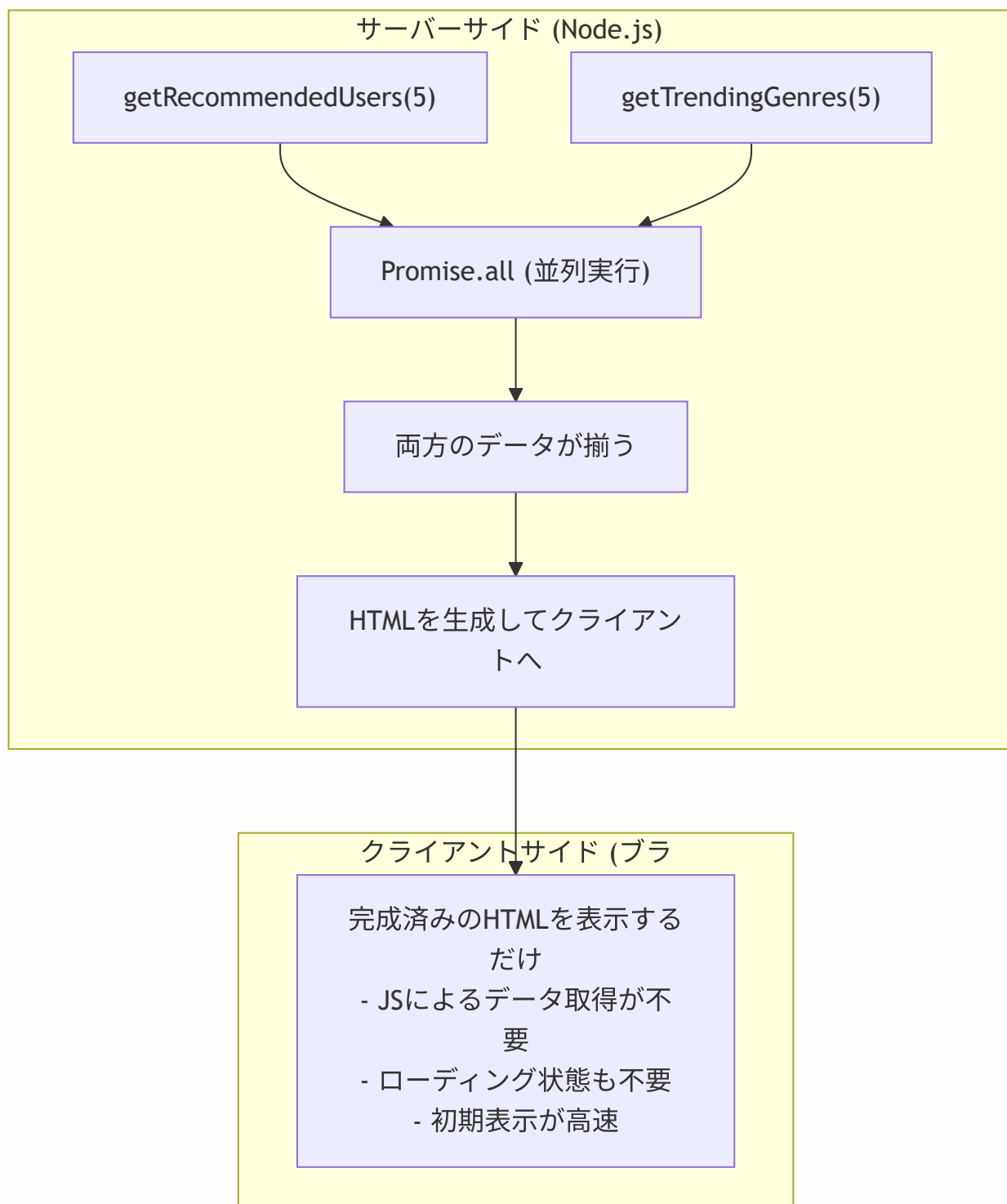
RightSidebarのServer Componentパターン

RightSidebarは **Server Component** として実装されており、サーバーサイドでデータを取得して描画します。

```
// components/layout/RightSidebar.tsx (BON-LOGの実際のコード)
export async function RightSidebar() {
  // Promise.allで並列取得 → パフォーマンス最適化
  const [usersResult, genresResult] = await Promise.all([
    getRecommendedUsers(5),
    getTrendingGenres(5),
  ])

  const recommendedUsers = usersResult.users || []
  const trendingGenres = genresResult.genres || []

  return (
    <aside className="sticky top-0 h-screen w-80 border-l
      bg-card/95 backdrop-blur-sm hidden xl:flex flex-col
      p-4 overflow-y-auto shadow-washi">
      {/* おすすめユーザー */}
      {/* トレンドジャンル */}
      {/* 広告 */}
      {/* フッター */}
    </aside>
  )
}
```



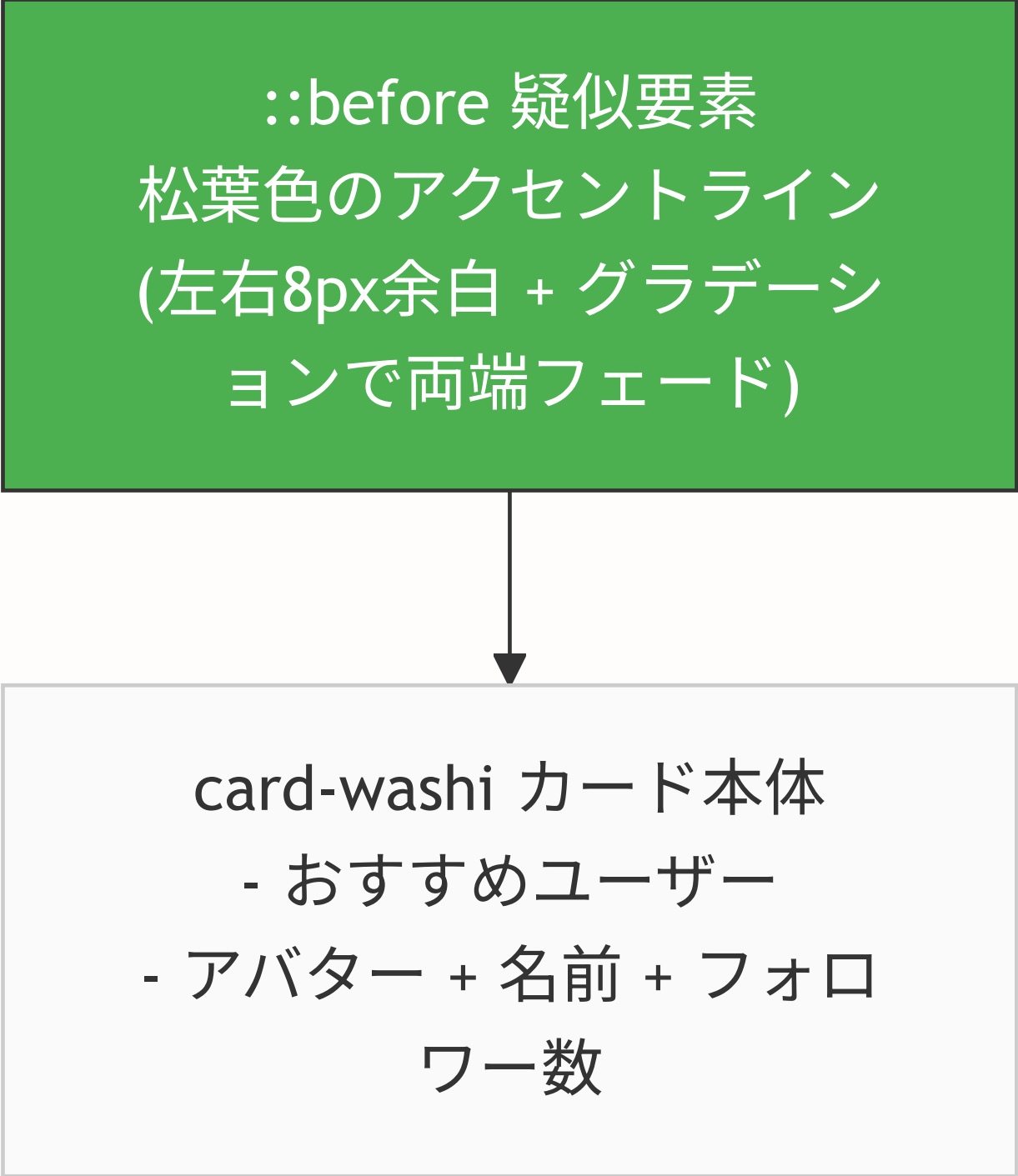
Server Component のメリット: データベースに直接アクセスでき、APIルート不要。ブラウザに JavaScriptを送信しないため軽量。

RightSidebarの card-washi クラス

```
<!-- おすすめユーザーセクション -->
<div className="card-washi rounded p-4 mb-4">
```

```
/* app/globals.css (BON-LOGの実際のコード) */
.card-washi {
  background: var(--card);
  border: 1px solid var(--border);
  position: relative;
}
.card-washi::before {
  content: '';
  position: absolute;
  top: -1px;
  left: 8px;
  right: 8px;
  height: 3px;
  background: linear-gradient(90deg,
    transparent 0%,
    var(--primary) 30%,
    var(--primary) 70%,
    transparent 100%
  );
  border-radius: 0 0 2px 2px;
}
```

::before 疑似要素
松葉色のアクセントライン
(左右8px余白 + グラデーションで両端フェード)



```
graph TD; A["::before 疑似要素  
松葉色のアクセントライン  
(左右8px余白 + グラデーションで両端フェード)"] --> B["card-washi カード本体  
- おすすめユーザー  
- アバター + 名前 + フォロ  
ワー数"]
```

card-washi カード本体
- おすすめユーザー
- アバター + 名前 + フォロ
ワー数

::before 疑似要素で上部にアクセントラインを配置。左右8pxの余白 + グラデーションで両端がフェードし、カードに和風の品格を添える装飾。

MobileNavの状態管理とスタイリング

MobileNavの「もっと見る」メニューは、Client Componentの状態管理とスタイリングが密接に関連しています。

```
// components/layout/MobileNav.tsx (BON-LOGの実際のコード)
export function MobileNav({ userId, isPremium }: MobileNavProps) {
  const pathname = usePathname()
  const [showMoreMenu, setShowMoreMenu] = useState(false)
  const menuRef = useRef<HTMLDivElement>(null)

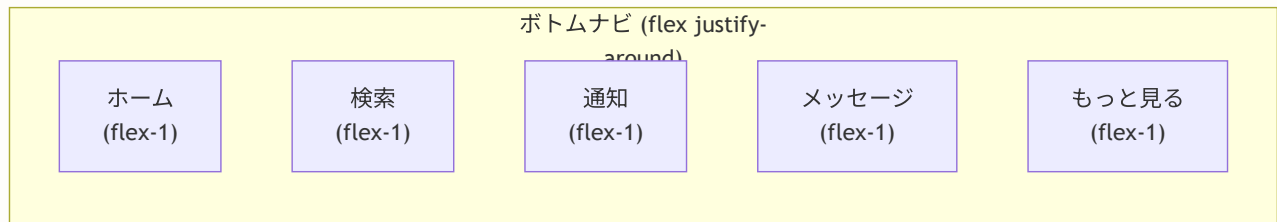
  // メニュー外クリックで閉じる
  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      if (menuRef.current && !menuRef.current.contains(event.target as Node)) {
        setShowMoreMenu(false)
      }
    }
    if (showMoreMenu) {
      document.addEventListener('mousedown', handleClickOutside)
    }
    return () => {
      document.removeEventListener('mousedown', handleClickOutside)
    }
  }, [showMoreMenu])

  return (
    <nav className="fixed bottom-0 left-0 right-0 bg-card/95
      backdrop-blur-sm border-t border-border/50 lg:hidden z-50
      shadow-washi">
      {/* ... */}
    </nav>
  )
}
```

MobileNavのレイアウト構成

MobileNav のレイアウト (モバイル画面下部)

プロパティ	値	効果
position	<code>fixed bottom-0 left-0 right-0</code>	画面下部に完全固定
背景	<code>bg-card/95 backdrop-blur-sm</code>	すりガラス効果の背景
重なり順	<code>z-50</code>	最前面に表示（他のコンテンツの上に）
高さ	<code>h-16</code> (64px)	ナビバーの高さ



「もっと見る」タップ時のドロップダウン: `absolute bottom-full right-0 w-48 bg-card rounded-lg shadow-lg border` で、ボタンの上に表示される。メニュー項目: プロフィール / マイ盆栽 / 盆栽園マップ / イベント / ブックマーク / 設定 / 利用規約 / プライバシー

HeaderのモバイルUI

```
// components/layout/Header.tsx (BON-LOGの実際のコード)
export function Header({ userId, isPremium }: HeaderProps) {
  return (
    <header className="sticky top-0 z-40 border-b border-border/50
      bg-card/95 backdrop-blur-sm lg:hidden shadow-washi">
      <div className="flex items-center justify-between h-14 px-4">
        {/* 左: ログ + プレミアムバッジ */}
        {/* 右: 設定 + ログアウト */}
      </div>
    </header>
  )
}
```

Header vs MobileNav の position 比較

コンポーネント	position	z-index	理由
Header	<code>sticky top-0</code>	<code>z-40</code>	コンテンツの一番上に表示。フロー内に残り、スクロール開始時に自然に画面上部に固定される
MobileNav	<code>fixed bottom-0</code>	<code>z-50</code>	常に画面下部に表示したいため、スクロール位置に関わらず固定。main要素に <code>pb-16</code> を付けてコンテンツが隠れるのを防止

理解度チェック

▶ Q1: RightSidebarがServer Componentである利点は何ですか？

▶ Q2: `pathname.startsWith(item.href + '/')` で `'/'` を付ける理由は何ですか？

▶ Q3: MobileNavのz-50とHeaderのz-40はなぜ異なる値なのか？

6.20 アニメーションとトランジションパターン

このセクションで学ぶこと

- BON-LOGの `globals.css` で定義されているアニメーション (@keyframes)
- Tailwindの `transition-all duration-200` パターンの使い分け
- `tw-animate-css` プラグインが提供するアニメーションクラス
- アクセシビリティ対応 (prefers-reduced-motion) の実装
- 実際のコンポーネントで使われているアニメーションの例
- 背景アニメーション (Canvas APIによる桜/紅葉/雪等のパーティクル描画)

BON-LOGで定義されているアニメーション

`globals.css` に定義されている @keyframes を一覧で解説します。

fade-in-up: 下からフェードイン

```
/* app/globals.css (BON-LOGの実際のコード) */
@keyframes fade-in-up {
  from {
    opacity: 0;          /* 完全に透明 */
    transform: translateY(10px); /* 10px下にずれた位置 */
  }
  to {
    opacity: 1;          /* 完全に不透明 */
    transform: translateY(0); /* 本来の位置 */
  }
}

.animate-fade-in-up {
  animation: fade-in-up 0.4s ease-out;
}
```

fade-in-up アニメーションの動き:

時間	状態	opacity	transform
0ms (開始)	見えない	0	translateY(10px) - 10px下にずれている
200ms (中間)	薄く見え始める	0.5	上に移動中
400ms (終了)	完全表示	1	translateY(0) - 定位置

使用場面: 投稿カードの初回表示、通知の出現。 `ease-out` により最初は速く、最後はゆっくりで自然な動きになる。

fade-in: スケールフェードイン

```
/* app/globals.css (BON-LOGの実際のコード) */
@keyframes fade-in {
  from {
    opacity: 0;
    transform: scale(0.95); /* 95%の大きさ */
  }
  to {
    opacity: 1;
    transform: scale(1); /* 100%の大きさ */
  }
}

.animate-fade-in {
  animation: fade-in 0.6s ease-out;
}
```

fade-in（ローディング画面などで使用）：

フェーズ	サイズ	見た目	scale	opacity
開始	小さい + 透明	見えない	0.95	0
中間	やや大きい + 半透明	薄く表示	0.975	0.5
終了	通常サイズ + 不透明	完全表示	1.0	1

gentle-bounce: やわらかバウンス

```
/* app/globals.css (BON-LOGの実際のコード) */
@keyframes gentle-bounce {
  0%, 100% {
    transform: translateY(0); /* 元の位置 */
  }
  50% {
    transform: translateY(-8px); /* 8px上に移動 */
  }
}

.animate-gentle-bounce {
  animation: gentle-bounce 2s ease-in-out infinite;
}
```

gentle-bounce（ローディングインジケータなどで使用）：

要素が基準位置から8px上に移動し、また戻る動きを2秒周期で永遠に繰り返す波形アニメーション。

設定	値	説明
duration	2s	1周期の長さ
timing	ease-in-out	上下の頂点で速度がゆるやかに
移動量	-8px	控えめな移動量（激しくない、和風の穏やかさ）
繰り返し	infinite	永遠に繰り返す

Tailwindトランジションの使い分け

BON-LOGのコンポーネントでは、いくつかのトランジションパターンが使われています。

トランジションパターン一覧

パターン	遷移対象	使用場面
transition-all duration-200	全プロパティを200msで遷移	ボタン、リンク全般。クリック、ホバー時
transition-colors	色のみを遷移	テキスト色、背景色のみ。フッターリンクなど
transition-[color, box-shadow]	色と影のみを遷移	Badge。フォーカス時
transition-all duration-300	全プロパティを300msで遷移	和風カスタムクラス。より落ち着いた動き

active:scale-[0.98] のフィードバック効果

```
/* Buttonコンポーネントのベーススタイルより */
active:scale-[0.98]
```

active:scale-[0.98] の効果

状態	scale	説明
通常状態	1.0	通常サイズ
クリック中	0.98	2%小さくなる

効果:

- 物理的な「押している」感覚を演出
- 2%の縮小はユーザーがかろうじて感じる程度
- 大きすぎると不自然、小さすぎると気づかない
- `duration-200` と組み合わせてスムーズに遷移

和風デザインとの調和: 派手なエフェクトではなく、控えめで上品なフィードバック。

アクセシビリティ: prefers-reduced-motion 対応

BON-LOGでは、アニメーションを好まないユーザーや、アニメーション酔いを起こすユーザーに配慮した実装がされています。

```

/* app/globals.css (BON-LOGの実際のコード) */
@media (prefers-reduced-motion: reduce) {
  /* 全てのアニメーションとトランジションを無効化 */
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }

  /* 特定のアニメーションクラスを無効化 */
  .animate-fade-in-up,
  .animate-fade-in,
  .animate-gentle-bounce {
    animation: none !important;
    opacity: 1 !important;
    transform: none !important;
  }
}

```

prefers-reduced-motion の仕組み

OS/ブラウザの設定場所:

OS	設定パス
Windows	設定 → アクセシビリティ → アニメーション効果
macOS	設定 → アクセシビリティ → 視差効果を減らす
iOS	設定 → アクセシビリティ → 視覚効果を減らす

ユーザーが「アニメーションを減らす」を有効にすると:

要素	通常モード	reduced-motionモード
投稿カード	フェードインしながら下から出現	即座に表示（アニメーションなし）
ボタン	クリック時に0.98倍に縮小するアニメーション	即座に反応（トランジションなし）
ローディング	要素がバウンドし続ける	静止した表示

`animation-duration: 0.01ms` にする理由: `0ms` にするとブラウザによっては `animationend` イベントが発火しない問題がある。`0.01ms` なら実質見えないが、イベントは正常に発火する。

理解度チェック

▶ Q1: `active:scale-[0.98]` の0.98という値の意図は何ですか？

▶ Q2: `prefers-reduced-motion` で `animation-duration` を `0ms` ではなく `0.01ms` にする理由は？

高コントラストモード対応

BON-LOGでは、視覚に制限のあるユーザーのために高コントラストモードにも対応しています。

```

/* app/globals.css (BON-LOGの実際のコード) */
@media (prefers-contrast: more) {
  :root {
    --border: oklch(0.40 0.02 50);
    --muted-foreground: oklch(0.35 0.02 50);
  }

  /* フォーカスリングをより目立たせる */
  *:focus-visible {
    outline-width: 3px;
    outline-offset: 3px;
  }
}

```

高コントラストモードの変更点

プロパティ	通常モード	高コントラストモード	効果
<code>--border</code>	oklch(0.85 ...) 薄いボーダー	oklch(0.40 ...) 濃いボーダー	明度 0.85 → 0.40 (大幅に暗く)。背景との区別がよりはっきりする
<code>--muted-foreground</code>	oklch(0.48 ...) 薄いグレーテキスト	oklch(0.35 ...) 濃いグレーテキスト	明度 0.48 → 0.35 (大幅に暗く)。補助テキストが読みやすくなる
<code>outline-width</code>	2px 標準	3px 太いフォーカス	フォーカスリングがより目立つ
<code>outline-offset</code>	-	3px	フォーカスリングを要素から離す

アニメーション実践例：投稿カードの表示

投稿カードが画面に表示される際のアニメーション実装例です。

```

````typescript
// components/post/PostCard.tsx (実践例)
'use client'

import { cn } from '@lib/utils'

type PostCardProps = {
 className?: string
 children: React.ReactNode
 index?: number // リスト内の順番 (遅延表示に使用)
}

export function PostCard({ className, children, index = 0 }: PostCardProps) {
 return (
 <article
 className={cn(
 // ベーススタイル
 'card-washi rounded-lg p-4',
 // アニメーション
 'animate-fade-in-up',
 // 外部からのカスタマイズ
 className
)}
 style={{
 // index に応じて表示を遅延させる (スタガーアニメーション)
 animationDelay: `${index * 50}ms`,
 animationFillMode: 'both', // 遅延中は非表示に保つ
 }}
 >
 {children}
 </article>
)
}

// 使用例：投稿リスト
function PostList({ posts }: { posts: Post[] }) {
 return (
 <div className="space-y-4">
 {posts.map((post, index) => (
 <PostCard key={post.id} index={index}>
 <p>{post.content}</p>
 </PostCard>
))}
 </div>
)
}

```

```
)
}
```

## スタガーアニメーション（時間差表示）の効果

カード番号	animationDelay	タイミング
1	0ms	即座にフェードイン
2	50ms	50ms後にフェードイン
3	100ms	100ms後にフェードイン
4	150ms	150ms後にフェードイン

各カードが順番にフェードインすることで、リストが「流れるように」表示される演出が生まれる。50msの間隔は人間が「順番に出ている」と感じる最小限の遅延。

## ホバーエフェクトの組み合わせパターン

BON-LOGで使われているホバーエフェクトのパターンを整理します。

```
// パターン1: 背景色の変化（最も基本的）
<div className="hover:bg-muted/50 transition-colors">
{/* → ホバーでうっすら灰色の背景が出現。リスト項目のホバーに */}

// パターン2: 背景色 + 移動（サイドバーナビゲーション）
<div className="hover:bg-muted/70 hover:translate-x-1 transition-all duration-200">
{/* → ホバーで背景色が変わり、要素が右に4pxスライド。ナビリンクの「選択感」を演出 */}

// パターン3: 影の変化（カード）
<div className="shadow-sm hover:shadow-washi-lg transition-all duration-200">
{/* → ホバーで影が大きくなり、カードが浮き上がるように見える */}

// パターン4: ボーダー色の変化（インプットの要素）
<div className="border border-border/50 hover:border-primary/30 transition-all duration-200">
{/* → ホバーで枠線が松葉色に変化。入力欄やセレクトボックス向き */}

// パターン5: 複合効果（リッチなホバー）
<div className="bg-card hover:bg-muted/30 shadow-sm hover:shadow-washi-lg
 border border-transparent hover:border-primary/20
 transition-all duration-200">
{/* → 背景・影・枠線が同時に変化する豪華なホバーエフェクト。重要なカードに */}

// パターン6: テキスト色の変化（リンク）

{/* → グレーのテキストがホバーで松葉色に変化。テキストリンクの基本パターン */}
```

## ホバーエフェクトの「和風原則」

原則	良い例	悪い例	理由
1. 変化は控えめに	<code>hover:bg-muted/50</code> （わずかな背景変化）	<code>hover:bg-primary</code> （急激な色変化）	和風デザインは控えめを重視
2. durationは200msが基本	<code>duration-200</code> （自然で心地よい速度）	<code>duration-75</code> （急すぎ） / <code>duration-500</code> （遅すぎ）	自然な心地よさ
3. 不透明度で調整	<code>hover:bg-muted/50</code> （50%不透明・控えめ）/ <code>hover:bg-muted/70</code> （70%・やや強め）	-	数値で微調整できるのがTailwindの利点
4. 色相は変えずに明度だけ変える	<code>hover:bg-primary/85</code> （同じ色の少し暗い版）	<code>hover:bg-blue-500</code> （全く異なる色）	色の一貫性を保つ

## 背景アニメーション（Canvas API）

BON-LOGでは、画面の奥に季節のエフェクト（桜の花びら、紅葉、雪、綿毛、水面の波紋、雨）が舞う**背景アニメーション**機能を提供しています。これはHTML5 Canvas APIを使ってJavaScriptで直接描画する手法で、CSSアニメーションとは異なるアプローチです。

### CSSアニメーション vs Canvas APIの使い分け

手法	適した場面	BON-LOGでの例
CSSアニメーション	ボタンのホバー効果、フェードイン、単純な繰り返し	<code>animate-fade-in-up</code> 、 <code>transition-all duration-200</code>
Canvas API	大量のパーティクル、物理シミュレーション、複雑な描画	桜の花びら（回転＋揺れ＋奥行き）、水面の波紋

**なぜCanvasを使うのか？** 桜の花びらは1画面に30～50個のパーティクルが独立して動き、それぞれが回転・揺れ・奥行きの3D風表現を持っています。これをCSSの `@keyframes` で実現しようとする、50個分のDOM要素と複雑なアニメーション定義が必要になり、パフォーマンスが大きく低下します。Canvas APIなら1つの `<canvas>` 要素上にJavaScriptで描画するため、DOM負荷が最小限です。

### アニメーションの種類

ユーザーは設定ページで以下の6種類＋オフから選べます：

値	表示名	描画内容
<code>sakura</code>	桜の花びら	ピンクの花びらが回転しながらゆっくり落ちる（デフォルト）
<code>momiji</code>	秋の紅葉	赤・オレンジ・黄色の葉が風に舞う
<code>snow</code>	冬の雪	白い雪が静かに降る
<code>dandelion</code>	春の綿毛	ふわふわした綿毛が漂う
<code>rain</code>	水面の波紋	水面に波紋が広がるエフェクト
<code>rain-drops</code>	雨	斜めに降る雨粒
<code>none</code>	オフ	アニメーション非表示

## 実装の全体像

背景アニメーション機能は3つのファイルで構成されています:

```
lib/sakura-petals-pref.ts ← 設定の読み書き (localStorage)
components/SakuraAnimation.tsx ← Canvas描画ロジック (Client Component)
components/settings/SakuraPetalsToggle.tsx ← 設定UI (Select)
```

### 1. 設定管理 ( lib/sakura-petals-pref.ts )

```
// lib/sakura-petals-pref.ts (実際のコード・簡略化)

// 選べるアニメーションの型 (Union型 - Ch03 参照)
export type AnimationType = 'sakura' | 'momiji' | 'snow' | 'dandelion' | 'rain' | 'rain-dr

// localStorage のキー名
export const BG_ANIMATION_STORAGE_KEY = 'bg-animation-type'

// 設定変更時に同一タブ内で通知するためのカスタムイベント名
export const BG_ANIMATION_CHANGE_EVENT = 'bg-animation-change'

// 設定の読み取り (localStorage から取得、なければデフォルト 'sakura')
export function getBgAnimationType(): AnimationType { /* ... */ }

// 設定の書き込み (localStorage に保存 + カスタムイベントを発火)
export function setBgAnimationType(type: AnimationType): void { /* ... */ }
```

**ポイント: カスタムイベントによる即時反映** `setBgAnimationType()` は localStorage に保存するだけでなく、`window.dispatchEvent(new CustomEvent(...))` でイベントを発火します。`SakuraAnimation` コンポーネントはこのイベントを `addEventListener` で購読しているので、設定を変更した瞬間にアニメーションが切り替わります。これは React の状態管理 (useState/Context) を使わず、ブラウザのネイティブイベントシステムで実現しています。

### 2. Canvas描画 ( components/SakuraAnimation.tsx )

```
// components/SakuraAnimation.tsx (構造の概要)
'use client'

export default function SakuraAnimation() {
 const canvasRef = useRef<HTMLCanvasElement>(null)
 const particlesRef = useRef<Particle[]>([]) // パーティクル（花びら等）の配列
 const [animType, setAnimType] = useState<AnimationType>('sakura')

 // 1. カスタムイベントを購読して animType を更新
 useEffect(() => { /* BG_ANIMATION_CHANGE_EVENT を addEventListener */ }, [])

 // 2. animType に応じて Canvas 描画ループを開始
 useEffect(() => {
 if (animType === 'none') return // オフなら何もしない

 // Canvas のサイズ設定 (devicePixelRatio対応)
 const resizeCanvas = () => { /* ... */ }

 // パーティクルの初期化
 const initParticles = () => { /* animType に応じた数・色・速度で生成 */ }

 // アニメーションループ (requestAnimationFrame)
 const animate = () => {
 ctx.clearRect(0, 0, w, h) // 画面をクリア
 for (const p of particles) {
 updateParticle(p) // 位置・回転を更新
 drawParticle(ctx, p) // Canvas に描画
 }
 animationFrameId = requestAnimationFrame(animate)
 }

 // クリーンアップ (コンポーネント破棄時にアニメーション停止)
 return () => cancelAnimationFrame(animationFrameId)
 }, [animType])

 if (animType === 'none') return null

 return (
 <canvas
 ref={canvasRef}
 className="pointer-events-none fixed inset-0 z-0"
 // ↑ pointer-events-none で画面操作を妨げない
 // ↑ fixed inset-0 z-0 で画面全体の最背面に配置
 />
)
}
```

**初心者向け: requestAnimationFrame とは？** ブラウザの画面更新タイミング（通常60fps = 1秒に60回）に合わせて関数を呼び出すAPIです。 `setInterval(fn, 16)` と似ていますが、タブが非表示の時は自動的に停止するためバッテリー消費を抑えられます。ゲームやアニメーションの描画ループで標準的に使われる手法です。

### 3. 設定UI ( `components/settings/SakuraPetalsToggle.tsx` )

```
// components/settings/SakuraPetalsToggle.tsx (実際のコード)
'use client'

export function BgAnimationSelect() {
 const [animType, setAnimType] = useState<AnimationType>('sakura')

 useEffect(() => {
 // マウント時に localStorage から現在の設定を読み込む
 const id = setTimeout(() => setAnimType(getBgAnimationType()), 0)
 return () => clearTimeout(id)
 }, [])

 const handleChange = (value: string) => {
 const type = value as AnimationType
 setAnimType(type) // UIの状態を更新
 setBgAnimationType(type) // localStorage保存 + イベント発火
 }

 return (
 <Select value={animType} onChange={handleChange}>
 <SelectTrigger id="bg-animation">
 <SelectValue placeholder="エフェクトを選択" />
 </SelectTrigger>
 <SelectContent>
 <SelectItem value="sakura">桜の花びら</SelectItem>
 <SelectItem value="momiji">秋の紅葉</SelectItem>
 <SelectItem value="snow">冬の雪</SelectItem>
 { /* ... 他のオプション */ }
 <SelectItem value="none">オフ（非表示）</SelectItem>
 </SelectContent>
 </Select>
)
}
```

## 学んだ技術のまとめ

技術	この機能での使い方	詳しく学べる章
Canvas API	パーティクル描画、 <code>requestAnimationFrame</code> ループ	—
localStorage	ユーザー設定の永続化	Ch04 (ブラウザAPI)
CustomEvent	コンポーネント間のイベント通知	Ch04 (DOM API)
Union型	<code>AnimationType</code> の定義	Ch03 (TypeScript)
useRef	Canvas要素・パーティクル配列の参照保持	Ch07 (React Hooks)
useEffect	イベント購読・アニメーションループの管理	Ch07 (React Hooks)

▶ 理解度チェック: 背景アニメーション

## 6.21 asChildパターンとSlotコンポーネント

### このセクションで学ぶこと

- Radix UIの `Slot` コンポーネントの仕組みと役割
- `asChild` パターンがReactのコンポーネント合成でどう機能するか
- BON-LOGのButton/Badgeでの実際のasChild使用例
- Next.jsのLinkコンポーネントとの組み合わせパターン

### asChildとは何か

`asChild` は、コンポーネントの「外見」はそのままに、レンダリングされるHTML要素を子要素に委譲するパターンです。

**asChild パターンの動作:**

asChild	入力	出力	説明
<b>false</b> (デフォルト)	<code>&lt;Button&gt;クリック&lt;/Button&gt;</code>	<code>&lt;button class="bg-primary ... "&gt;クリック&lt;/button&gt;</code>	Button自身が <code>&lt;button&gt;</code> をレンダリング
<b>true</b>	<code>&lt;Button asChild&gt;&lt;Link href="/profile"&gt;プロフィール&lt;/Link&gt;&lt;/Button&gt;</code>	<code>&lt;a href="/profile" class="bg-primary ... "&gt;プロフィール&lt;/a&gt;</code>	子要素にButtonの props/classを渡し、子要素 (Link = a要素) がそのまま描画される

## Slotコンポーネントの仕組み

`asChild` パターンの背後にはRadix UIの `Slot` コンポーネントがあります。

```
// components/ui/button.tsx (BON-LOGの実際のコード)
import { Slot } from "@radix-ui/react-slot"

function Button({
 className,
 variant = "default",
 size = "default",
 asChild = false,
 ...props
}: /* ... */) {
 // asChildがtrueならSlot、falseならbuttonを使用
 const Comp = asChild ? Slot : "button"

 return (
 <Comp
 data-slot="button"
 data-variant={variant}
 data-size={size}
 className={cn(buttonVariants({ variant, size, className }))}
 { ... props}
 />
)
}
```

### Slot コンポーネントの動作原理:

Slot は「自分自身をレンダリングせず、子要素に自分の props を受け渡す」コンポーネント。

Slot className='bg-primary'  
onClick=handler

propsを委譲

a href='/home'  
class='bg-primary'  
onclick='...'  
ホーム

Slot自身は消えて、子要素の `<a>` にpropsが渡される。 `className` は `cn()` でマージされるため、子要素の既存クラスとも安全に結合される。

## 実践的なasChild使用例

### ボタンスタイルのリンク

```
// Next.jsのLinkをButtonスタイルで表示
import { Button } from '@components/ui/button'
import Link from 'next/link'

// パターン1: リンクボタン
<Button asChild>
 <Link href="/feed">タイムラインへ</Link>
</Button>
// →
// タイムラインへ
//

// パターン2: アウトラインスタイルのリンク
<Button variant="outline" asChild>
 <Link href="/settings">設定</Link>
</Button>

// パターン3: アイコンボタンのリンク
<Button variant="ghost" size="icon" asChild>
 <Link href="/notifications">
 <BellIcon className="w-5 h-5" />
 </Link>
</Button>
```

## バッジスタイルのリンク

```
import { Badge } from '@components/ui/badge'
import Link from 'next/link'

// ジャンルタグをクリック可能にする
<Badge variant="secondary" asChild>
 <Link href="/search?genre=松柏類">松柏類</Link>
</Badge>
// →
// 松柏類
//
// ホバー時に [a&]:hover:bg-secondary/90 が適用される
```

## asChildを使う場面と使わない場面

### asChild 使い分けガイド

asChildを使う場面	asChildを使わない場面
ボタンスタイルのリンク (Link + Button)	通常のクリックハンドラ (onClick) を持つボタン
バッジスタイルのリンク (Link + Badge)	フォームの送信ボタン (type="submit")
button以外のセマンティクスが必要な場合	disabled属性が必要な場合
カスタムラッパーコンポーネントへのスタイル委譲	単純な表示用バッジ

**ルール:** 「HTML要素を変えたい場合」に `asChild`。「HTML要素はそのままでもいい場合」は `asChild` 不要。

## なぜ [a&]:hover:bg-primary/90 が機能するのか

Badgeのバリエーション定義にある `[a&]:hover:bg-primary/90` は、asChildパターンと連携して動作します。

```
// badgeVariantsのdefaultバリエーション
default: "border-transparent bg-primary text-primary-foreground [a&]:hover:bg-primary/90"
```

### `[a&]:hover:bg-primary/90` の解説

ケース	JSX	出力HTML	[a&]	ホバー
通常のBadge (span)	<code>&lt;Badge&gt;新着&lt;/Badge&gt;</code>	<code>&lt;span class=" ... "&gt;新着 &lt;/span&gt;</code>	不成立 (spanなので)	ホバースタイルなし
asChild=trueの Badge (a)	<code>&lt;Badge asChild&gt;&lt;a href=" ... "&gt;新着&lt;/a&gt; &lt;/Badge&gt;</code>	<code>&lt;a class=" ... "&gt;新 着&lt;/a&gt;</code>	成立 (aタグなので)	ホバー時に <code>bg-primary/90</code>

**設計意図:** spanの場合はホバー不要（クリックできないため）、aの場合はホバーが必要（クリックできることを示すため）。CSSだけで自動判定できる。

## data-slot属性の活用

BON-LOGの全コンポーネントは `data-slot` 属性を持っています。

```
<Comp data-slot="button" ... /> // Buttonコンポーネント
<Comp data-slot="badge" ... /> // Badgeコンポーネント
<div data-slot="alert" ... /> // Alertコンポーネント
<div data-slot="alert-title" ... /> // AlertTitleコンポーネント
<input data-slot="input" ... /> // Inputコンポーネント
```

### data-slot 属性の3つの用途

用途	例	説明
1. CSSセレクタとして	<code>::data-[slot=alert-description]:text-destructive</code>	AlertDescriptionのスタイルを親バリエーションから制御
2. テストでの要素特定	<code>screen.getByAttribute('data-slot', 'button')</code>	テストで確実に要素を見つけられる
3. デバッグツールでの識別	ブラウザの開発者ツールで要素を検査	どのコンポーネントかすぐに分かる

## 理解度チェック

- ▶ Q1: `asChild=true` にした場合、Slotコンポーネントは何をレンダリングしますか？
- ▶ Q2: Badge に `[a&]:hover:bg-primary/90` が必要な理由は何ですか？
- ▶ Q3: `asChild`パターンを使わずにLinkにボタンスタイルを適用する方法はありますか？

## 6.22 よくある質問（FAQ）

### スタイリング全般

- ▶ Q: Tailwind CSSのクラスが長すぎて見づらいです。どう整理すればいいですか？
- ▶ Q: `bg-primary/10` の `/10` は何を意味しますか？
- ▶ Q: `cn()`関数とテンプレートリテラル(``${}``)はどう使い分けますか？

### shadcn/uiとCVA

- ▶ Q: shadcn/uiのコンポーネントをカスタマイズする正しい方法は？
- ▶ Q: CVAのvariantを後から追加するにはどうすればいいですか？

## レスポンスとレイアウト

▶ Q: 3カラムレイアウトで右サイドバーの表示がlg:flexではなくxl:flexである理由は？

▶ Q: sticky と fixed の使い分けがよく分かりません。

## ダークモードとテーマ

▶ Q: oklch()カラーが難しいです。もっと簡単に色を決める方法がありますか？

▶ Q: 和風デザインパターン（asanoha-pattern等）を使いすぎると重くなりませんか？

## 開発のヒント

▶ Q: Tailwindのクラスを効率よく書くコツはありますか？

▶ Q: カスタムCSSクラス（.washi-textureなど）とTailwindのユーティリティクラスは併用できますか？

## 6.23 デザインシステム チートシート

### BON-LOGカラーパレット早見表

セマンティックカラー（意味に基づくカラー）

Tailwindクラス	CSS変数	ライト	ダーク
<code>bg-background</code>	<code>--background</code>	生成り	漆黒
<code>text-foreground</code>	<code>--foreground</code>	墨	生成り
<code>bg-card</code>	<code>--card</code>	和紙	墨色
<code>bg-primary</code>	<code>--primary</code>	松葉色	若松色
<code>bg-secondary</code>	<code>--secondary</code>	亜麻色	暗灰
<code>bg-muted</code>	<code>--muted</code>	白練	暗灰
<code>bg-accent</code>	<code>--accent</code>	錆朱	緋色
<code>bg-destructive</code>	<code>--destructive</code>	朱色	明朱

## 和風カスタムカラー

Tailwindクラス	CSS変数	色名
<code>bg-bonsai-green</code>	<code>--bonsai-green</code>	松葉色
<code>bg-bonsai-brown</code>	<code>--bonsai-brown</code>	栗皮茶
<code>bg-bonsai-beige</code>	<code>--bonsai-beige</code>	亜麻色
<code>bg-bonsai-cream</code>	<code>--bonsai-cream</code>	生成り
<code>bg-washi</code>	<code>--washi</code>	和紙
<code>bg-sumi</code>	<code>--sumi</code>	墨
<code>bg-aka</code>	<code>--aka</code>	朱
<code>bg-ai</code>	<code>--ai</code>	藍
<code>bg-matcha</code>	<code>--matcha</code>	抹茶
<code>bg-kincha</code>	<code>--kincha</code>	金茶

## よく使うスタイリングパターン集

### カードパターン

```
// パターン1: 基本カード
<div className="bg-card rounded border p-4 shadow-sm">
 コンテンツ
</div>

// パターン2: 和風カード（上部アクセントライン付き）
<div className="card-washi rounded p-4">
 コンテンツ
</div>

// パターン3: ホバーエフェクト付きカード
<div className="bg-card rounded border p-4 shadow-sm
 hover:shadow-washi-lg hover:border-primary/30
 transition-all duration-200">
 コンテンツ
</div>

// パターン4: クリック可能なカード
<Link href="/posts/123" className="block bg-card rounded border p-4
 shadow-sm hover:bg-muted/30 transition-all duration-200
 active:scale-[0.99]">
 コンテンツ
</Link>
```

## テキストパターン

```
// 見出し（和風）
<h1 className="text-2xl font-bold text-foreground font-serif">
 盆栽の世界
</h1>

// サブテキスト
<p className="text-sm text-muted-foreground">
 2024年1月15日に投稿
</p>

// テキスト切り詰め（1行）
<p className="truncate">
 長いテキストが自動的に省略されます ...
</p>

// テキスト切り詰め（2行）
<p className="line-clamp-2">
 2行で切り詰められるテキスト。
 この行まで表示されますが、
 これ以降は省略されます。
</p>

// リンクテキスト
<Link href="/about" className="text-primary hover:text-primary/80
 transition-colors">
 詳細を見る
</Link>
```

## フォームパターン

```
// ラベル + 入力フィールド
<div className="space-y-2">
 <label className="text-sm font-medium text-foreground">
 ニックネーム
 </label>
 <Input placeholder="表示名を入力" />
</div>

// エラー表示付き入力
<div className="space-y-2">
 <label className="text-sm font-medium text-foreground">
 メールアドレス
 </label>
 <Input
 type="email"
 aria-invalid={!!error}
 placeholder="example@email.com"
 />
 {error && (
 <p className="text-sm text-destructive">{error}</p>
)}
</div>

// 送信ボタン
<Button type="submit" className="w-full">
 投稿する
</Button>
```

## レイアウトパターン

```
// 横並び（アイテム間にスペース）
<div className="flex items-center gap-3">
 <Avatar />
 ユーザー名
</div>

// 横並び（両端に配置）
<div className="flex items-center justify-between">
 タイトル
 <Button size="sm">編集</Button>
</div>

// 縦並び（アイテム間にスペース）
<div className="space-y-4">
 <PostCard />
 <PostCard />
 <PostCard />
</div>

// 中央揃え（画面いっぱい）
<div className="flex items-center justify-center min-h-screen">
 <LoginForm />
</div>
```

## スタイリング設計原則

BON-LOGのスタイリングは以下の原則に基づいています。

### BON-LOG スタイリング5原則

#	原則	説明
1	侘び寂びの美学	派手な色やエフェクトを避け、自然で落ち着いた雰囲気を大切にする。色は日本の伝統色から選び、角丸は控えめに、影は薄く繊細に。
2	コンテンツファースト	装飾は控えめにし、盆栽の写真やテキストなどのコンテンツが主役になるデザインにする。背景パターンは opacity 0.03～0.05 で極めて薄く。
3	レスポンシブ段階表示	モバイル → タブレット → デスクトップで段階的に情報量を増やす。モバイルでは核心的な情報のみ、デスクトップでは補助情報も表示する。
4	アクセシビリティ最優先	フォーカスリング、reduced-motion対応、高コントラストモード、aria属性を必ず実装する。美しさよりも使いやすさを優先する。
5	コンポーネント指向	スタイルはコンポーネントに閉じ込める。グローバルCSSは最小限に抑え、CVA + cn() でバリエントを型安全に管理する。

### ### Tailwindクラス名 逆引き辞書

「こんなことをしたい」から、対応するTailwindクラスをすぐに見つけられる逆引き辞書です。

#### \*\*レイアウト\*\*

やりたいこと	Tailwindクラス
横並びにしたい	<code>`flex`</code>
縦並びにしたい	<code>`flex flex-col`</code>
中央揃え（横）	<code>`justify-center`</code>
中央揃え（縦）	<code>`items-center`</code>
両端揃え	<code>`justify-between`</code>
要素間にスペース	<code>`gap-2`</code> , <code>`gap-4`</code>
画面いっぱいの高さ	<code>`min-h-screen`</code>
固定幅	<code>`w-64`</code> , <code>`w-80`</code>
残りの幅を全部使う	<code>`flex-1`</code>
最大幅を制限	<code>`max-w-2xl`</code>
中央に配置	<code>`mx-auto`</code>

#### \*\*見た目\*\*

やりたいこと	Tailwindクラス
背景色	<code>`bg-card`</code> , <code>`bg-primary`</code>
テキスト色	<code>`text-foreground`</code>
ボーダー	<code>`border`</code> , <code>`border-border`</code>
角丸	<code>`rounded`</code> , <code>`rounded-lg`</code>
影	<code>`shadow-sm`</code> , <code>`shadow-washi`</code>
不透明度	<code>`opacity-50`</code> , <code>`bg-primary/10`</code>

#### \*\*テキスト\*\*

やりたいこと	Tailwindクラス
フォントサイズ	<code>`text-xs`</code> , <code>`text-sm`</code> , <code>`text-base`</code>
太字	<code>`font-bold`</code> , <code>`font-medium`</code>
テキスト切り詰め	<code>`truncate`</code> , <code>`line-clamp-2`</code>
行間	<code>`leading-relaxed`</code>
文字間隔	<code>`tracking-tight`</code>

#### \*\*インタラクション\*\*

やりたいこと	Tailwindクラス
ホバー効果	<code>`hover:bg-muted`</code>
フォーカス表示	<code>`focus-visible:ring-2`</code>
クリック効果	<code>`active:scale-[0.98]`</code>

```
| 無効状態 | `disabled:opacity-50` |
| アニメーション | `transition-all duration-200` |
```

#### \*\*レスポンス\*\*

```
| やりたいこと | Tailwindクラス |
|-----|-----|
| モバイルのみ | (プレフィックスなし) |
| タブレット以上 | `md:` |
| デスクトップ以上 | `lg:` |
| 大画面以上 | `xl:` |
| モバイルでは非表示 | `hidden lg:flex` |
| デスクトップでは非表示 | `lg:hidden` |
```

#### \*\*配置\*\*

```
| やりたいこと | Tailwindクラス |
|-----|-----|
| 画面上部に固定 | `sticky top-0` |
| 画面下部に固定 | `fixed bottom-0` |
| 要素を重ねる | `relative` + `absolute` |
| 前面に表示 | `z-50` |
```

#### ### コンポーネント別 使用Tailwindクラス一覧

BON-LOGの主要コンポーネントで使われているTailwindクラスのパターンを整理します。

#### \*\*コンポーネント: Sidebar\*\* (`components/layout/Sidebar.tsx`)

```
| 対象 | クラス | 効果 |
|-----|-----|-----|
| aside 要素 | `sticky top-0` | スクロール時の固定配置 |
| | `h-screen` | 画面高さいっぱい |
| | `w-64` | 固定幅256px |
| | `border-r` | 右ボーダー |
| | `bg-card/95` | 95%不透明の背景 |
| | `backdrop-blur-sm` | すりガラス効果 |
| | `hidden lg:flex` | lg以上でのみ表示 |
| | `flex-col` | 子要素を縦並びに |
| | `shadow-washi` | 和風シャドウ |
| ナビ (アクティブ) | `bg-primary/10` | 薄い松葉色の背景 |
| | `text-primary` | 松葉色のテキスト |
| | `font-medium` | 中太字 |
| | `border-l-2 border-primary` | 左ボーダーのインジケーター |
| ナビ (非アクティブ) | `text-foreground` | 通常のテキスト色 |
| | `hover:bg-muted/70` | ホバーで薄い背景 |
| | `hover:translate-x-1` | ホバーで右に4px移動 |
```

#### \*\*コンポーネント: RightSidebar\*\* (`components/layout/RightSidebar.tsx`)

対象	クラス	効果
aside 要素	`sticky top-0`	スクロール時の固定配置
	`h-screen`	画面高さいっぱい
	`w-80`	固定幅320px
	`border-l`	左ボーダー
	`bg-card/95`	95%不透明の背景
	`backdrop-blur-sm`	すりガラス効果
	`hidden xl:flex`	xl以上でのみ表示
	`flex-col`	子要素を縦並びに
	`p-4`	全方向16pxのパディング
	`overflow-y-auto`	内容が長い場合スクロール
	`shadow-washi`	和風シャドウ
おすすめユーザーリスト	`space-y-3`	アイテム間に12pxのスペース
	`rounded-full`	アバターを完全な円に
	`ring-2 ring-border`	アバターに枠線
	`truncate`	長いニックネームを切り詰め
フッター	`mt-auto`	下部に配置 (flexboxの余白を利用)
	`text-xs`	極小フォント
	`text-muted-foreground`	薄いグレーのテキスト

**\*\*コンポーネント: MobileNav\*\*** (`components/layout/MobileNav.tsx`)

対象	クラス	効果
nav 要素	`fixed bottom-0 left-0 right-0`	画面下部に完全固定
	`bg-card/95`	95%不透明の背景
	`backdrop-blur-sm`	すりガラス効果
	`border-t`	上ボーダー
	`border-border/50`	50%不透明のボーダー色
	`lg:hidden`	lg以上では非表示
	`z-50`	最前面に表示
	`shadow-washi`	和風シャドウ
ナビ (アクティブ)	`text-primary`	松葉色のアイコン・テキスト
	`bg-primary/10`	薄い松葉色の背景 (アイコン周り)
	`font-medium`	テキストを中太字に
ナビ (非アクティブ)	`text-muted-foreground`	薄いグレーのアイコン
	`hover:text-foreground`	ホバーで通常色に
「もっと見る」メニュー	`absolute bottom-full right-0`	ボタンの上に表示
	`w-48`	幅192px
	`bg-card`	カード背景色
	`rounded-lg`	角丸
	`shadow-lg`	大きめの影
	`border`	ボーダー

**\*\*コンポーネント: Header\*\*** (`components/layout/Header.tsx`)

対象	クラス	効果
header 要素	`sticky top-0`	スクロール時上部に固定

```
| | `z-40` | MobileNav(z-50)より下 |
| | `border-b` | 下ボーダー |
| | `border-border/50` | 50%不透明のボーダー |
| | `bg-card/95` | 95%不透明の背景 |
| | `backdrop-blur-sm` | すりガラス効果 |
| | `lg:hidden` | lg以上では非表示 |
| | `shadow-washi` | 和風シャドウ |
| 内部レイアウト | `flex items-center justify-between` | 左右に分割 |
| | `h-14` | 高さ56px |
| | `px-4` | 左右16pxのパディング |
```

### ### 和風CSSクラス一覧

`globals.css` で定義されている和風カスタムCSSクラスの完全なリファレンスです。

クラス名	用途	適用対象
`washi-texture`	和紙のような微細な質感	背景に使用
`asanoha-pattern`	麻の葉模様の背景パターン	メインレイアウト背景
`seigaiha-pattern`	青海波(波模様)の背景パターン	装飾的な背景
`ichimatsu-pattern`	市松模様の背景パターン	装飾的な背景
`washi-border`	上部にグラデーションラインを追加	セクション区切り
`suminagashi`	墨流し風グラデーション背景	装飾的な背景
`shadow-washi`	和紙のような3層の柔らかいシャドウ	カード、サイドバー
`shadow-washi-lg`	shadow-washiの大きいバージョン	ホバー時のカード
`hover-washi`	ホバー時の和風グラデーション効果	インタラクティブ要素
`btn-washi`	波紋エフェクト付きボタン	特別なボタン
`vertical-text`	縦書きテキスト	和風見出し
`divider-bamboo`	両端フェードの区切り線	セクション区切り
`ink-circle`	墨で描いたような回転する円	装飾的なアバター
`card-washi`	上部アクセントライン付きカード	サイドバーカード
`tag-washi`	和風のタグ/ラベル	カテゴリ表示
`input-washi`	和風の入力フィールド	フォーム入力
`avatar-washi`	和風のアバター枠	ユーザーアバター
`hanko`	印鑑風の円形装飾	認証マーク
`frame-washi`	和紙風の縁取り装飾	画像フレーム
`animate-fade-in-up`	下からフェードインするアニメーション	初回表示
`animate-fade-in`	スケールフェードインアニメーション	ローディング
`animate-gentle-bounce`	やわらかなバウンスアニメーション	ローディング

---

### ## まとめ

この章では、Tailwind CSSとshadcn/uiを使ったスタイリングを学びました。

#### \*\*重要ポイント\*\*

1. **\*\*ユーティリティファースト\*\***: 小さなクラスを組み合わせでスタイリング
2. **\*\*モバイルファースト\*\***: プレフィックスなしはすべてのサイズ、プレフィックス付きはそのサイズ以上
3. **\*\*カスタムテーマ\*\***: Tailwind CSS v4の`@theme inline`でCSS変数ベースのテーマを定義

4. **\*\*shadcn/ui\*\***: アクセシブルで高品質なコンポーネントをコピー&ペースト
5. **\*\*CVA\*\***: `cva()`でバリエントを型安全に管理し、`cn()`でクラスの競合を解決
6. **\*\*レスポンシブ\*\***: sm:, md:, lg:, xl:等のプレフィックスで段階的な3カラムレイアウト
7. **\*\*和風デザイン\*\***: oklch()カラーパレット、和紙テクスチャ、控えめなアニメーション
8. **\*\*アクセシビリティ\*\***: prefers-reduced-motion、高コントラストモード対応

```
<div class="mermaid-rendered">

</div>
```

次の章では、Prismaとデータベースについて学びます。